

Computing with Pictures

the Rise of String Diagrams in Computer Science

Fabio Zanasi



Part I

Algebraic Semantics: from Programs to Diagrams

From Logic to Computer Science

Programming Language Theory

From Logic to Computer Science

Turing Machines
(1936)



A. Turing



A. Church

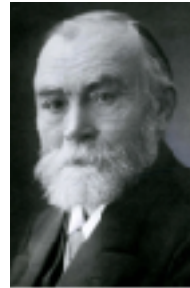
λ -calculus
(1930s)



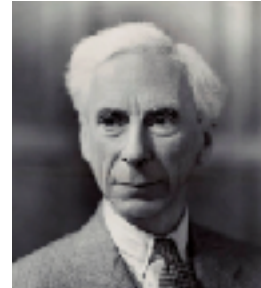
Programming Language Theory

From Logic to Computer Science

The *Begriffsschrift*
(1879) is the first
logical system in a
modern sense



G. Frege



B. Russell

Logical systems
informed by the
foundational crisis in
Mathematics
(early XX century)



Turing Machines
(1936)



A. Turing



A. Church

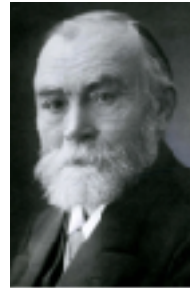
λ -calculus
(1930s)



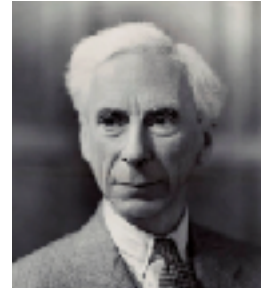
Programming Language Theory

From Logic to Computer Science

The *Begriffsschrift*
(1879) is the first
logical system in a
modern sense



G. Frege



B. Russell

Logical systems
informed by the
foundational crisis in
Mathematics
(early XX century)

The Road not taken

Existential Graphs
Calculus of Relations



C.S. Peirce



Turing Machines
(1936)



A. Turing



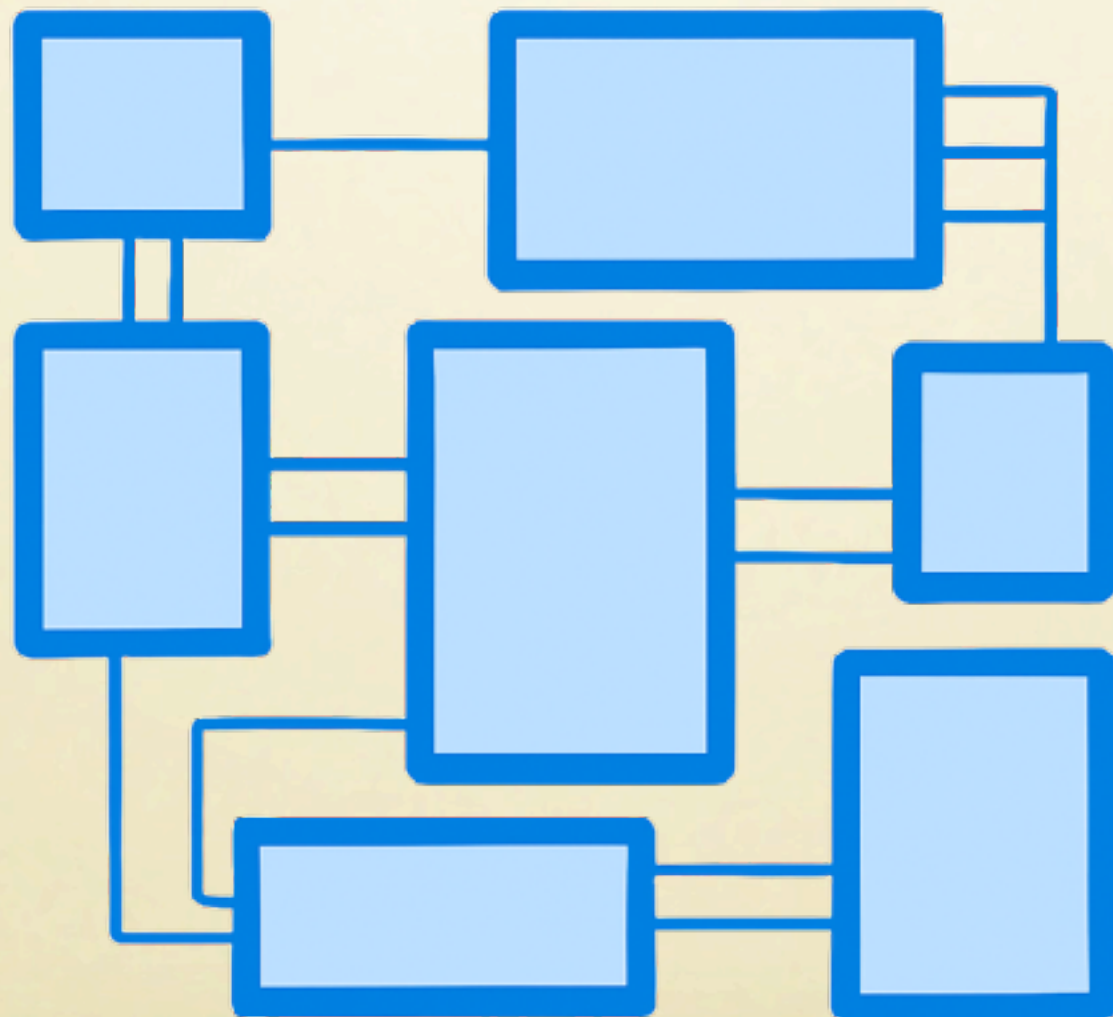
A. Church

λ -calculus
(1930s)

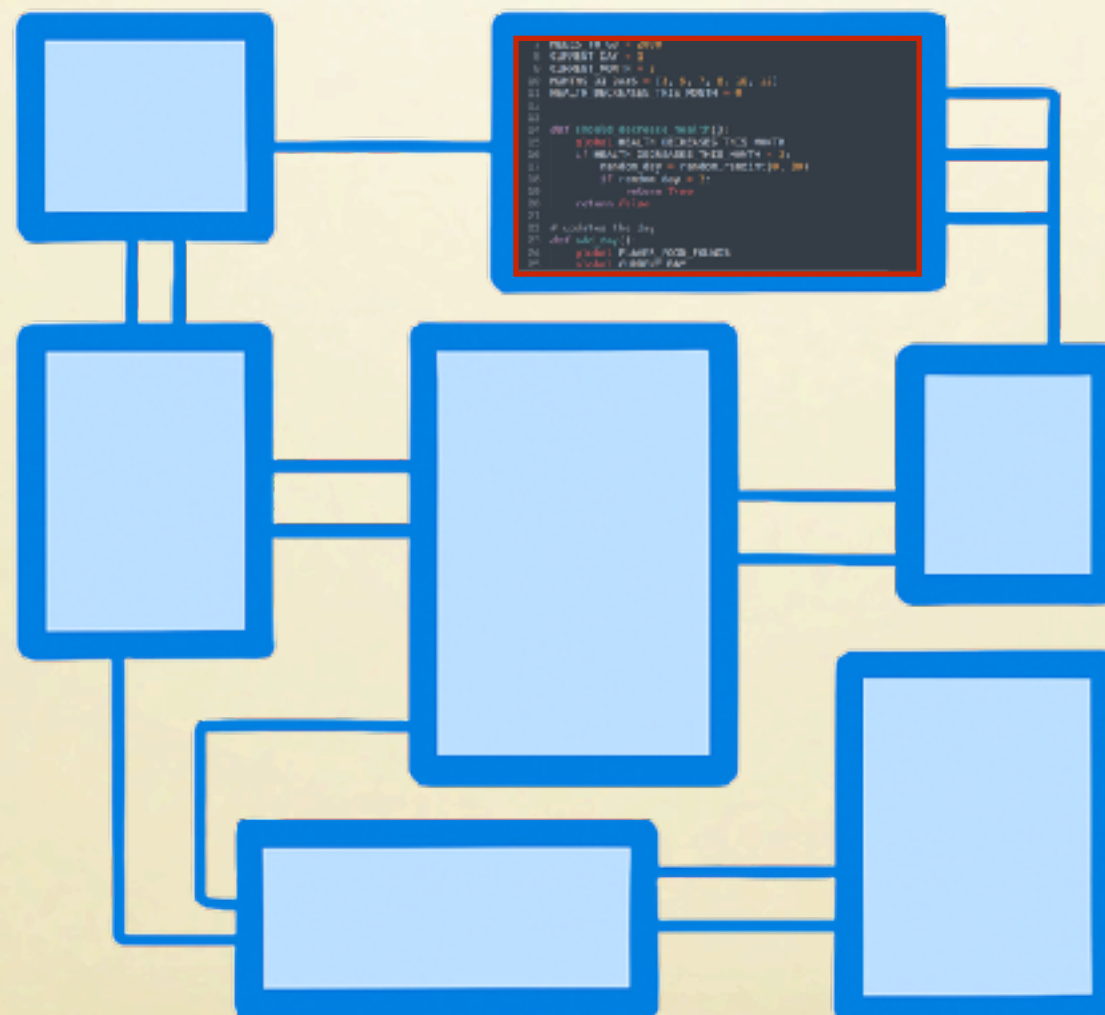


Programming Language Theory

Algebraic Semantics of Computation



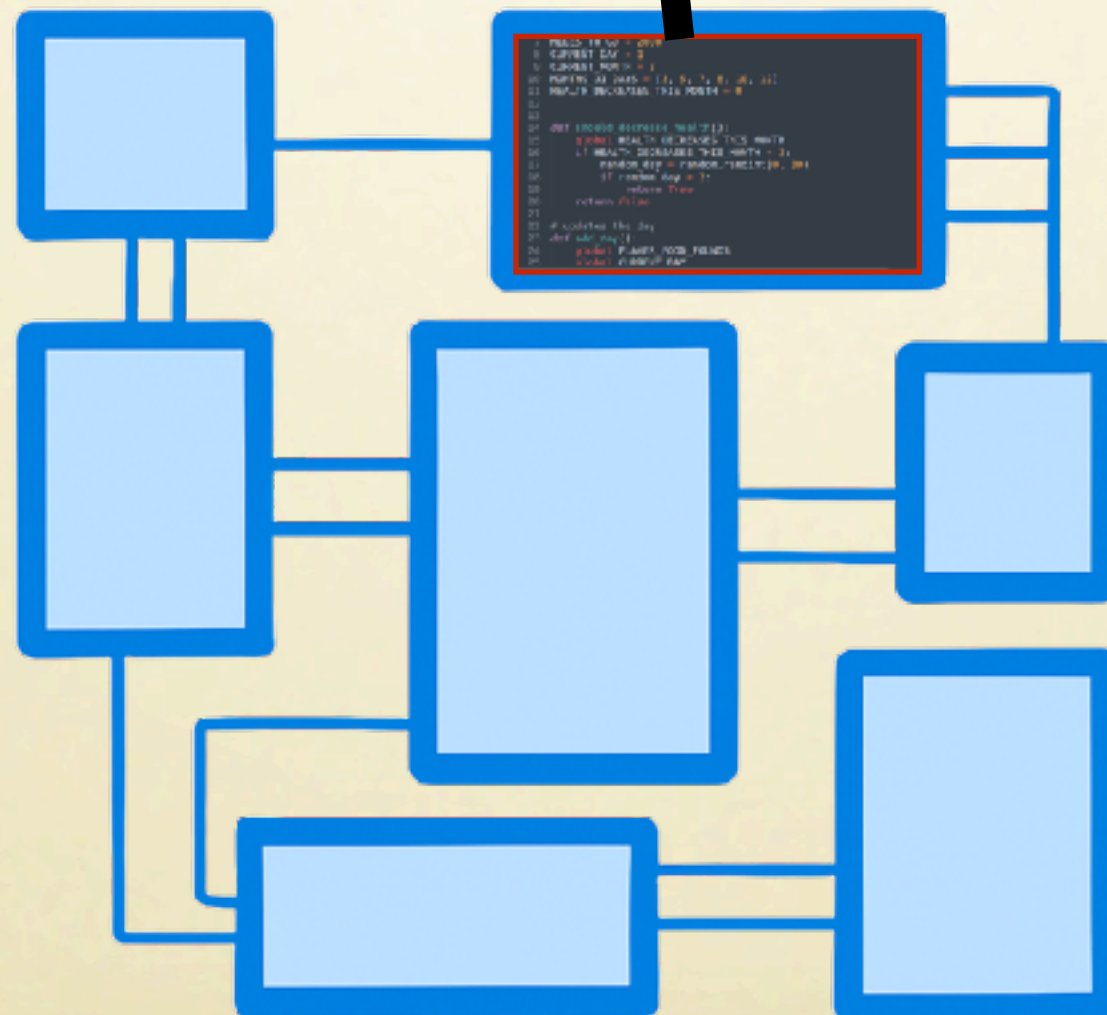
Algebraic Semantics of Computation



Algebraic Semantics of Computation

Program
Semantics

$s(s(t(x), y, z), t(z), z)$

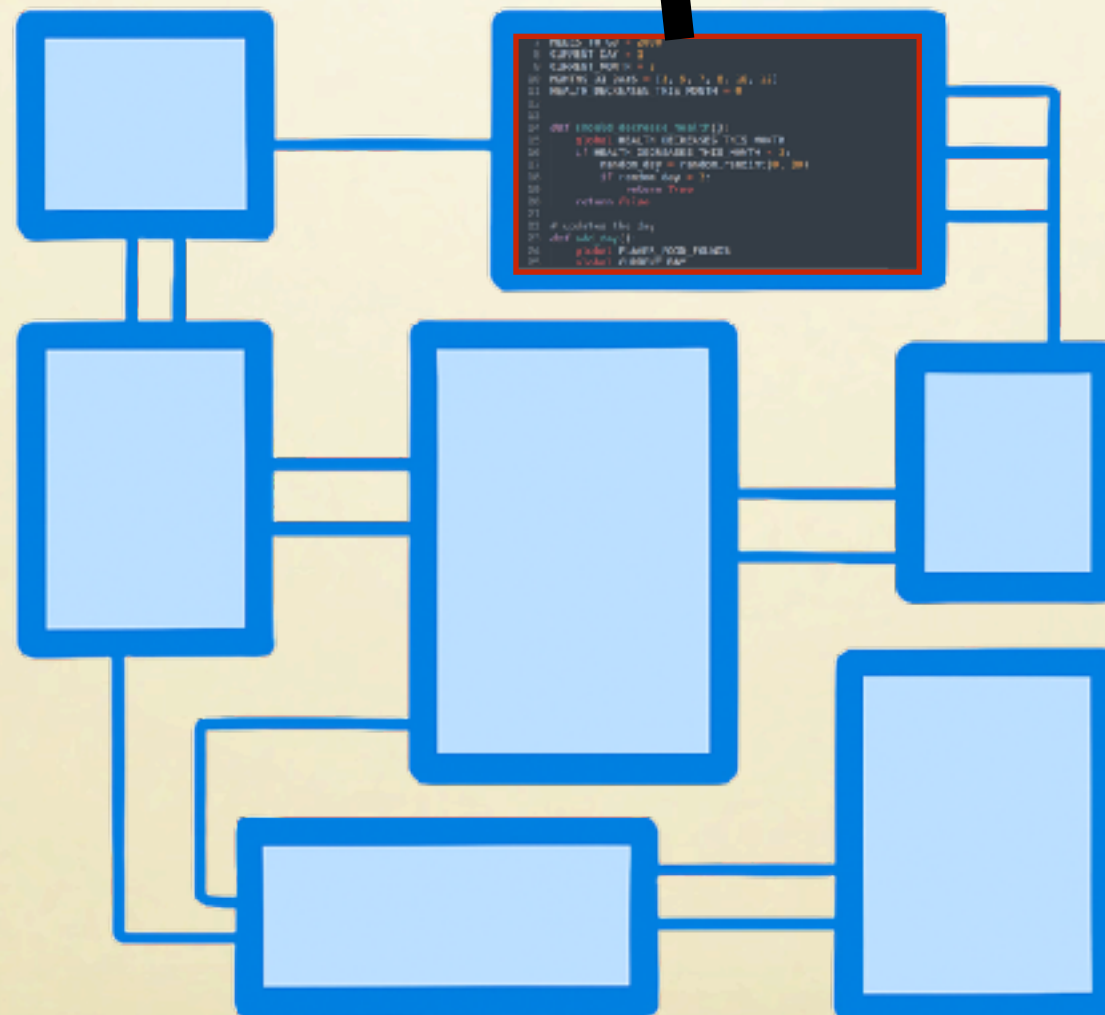


Algebraic Semantics of Computation

Program Semantics

$$s(t(x), y, z) = u(y, y)$$

$$s(s(t(x), y, z), t(z), z) = s(u(y, y), t(z), z)$$

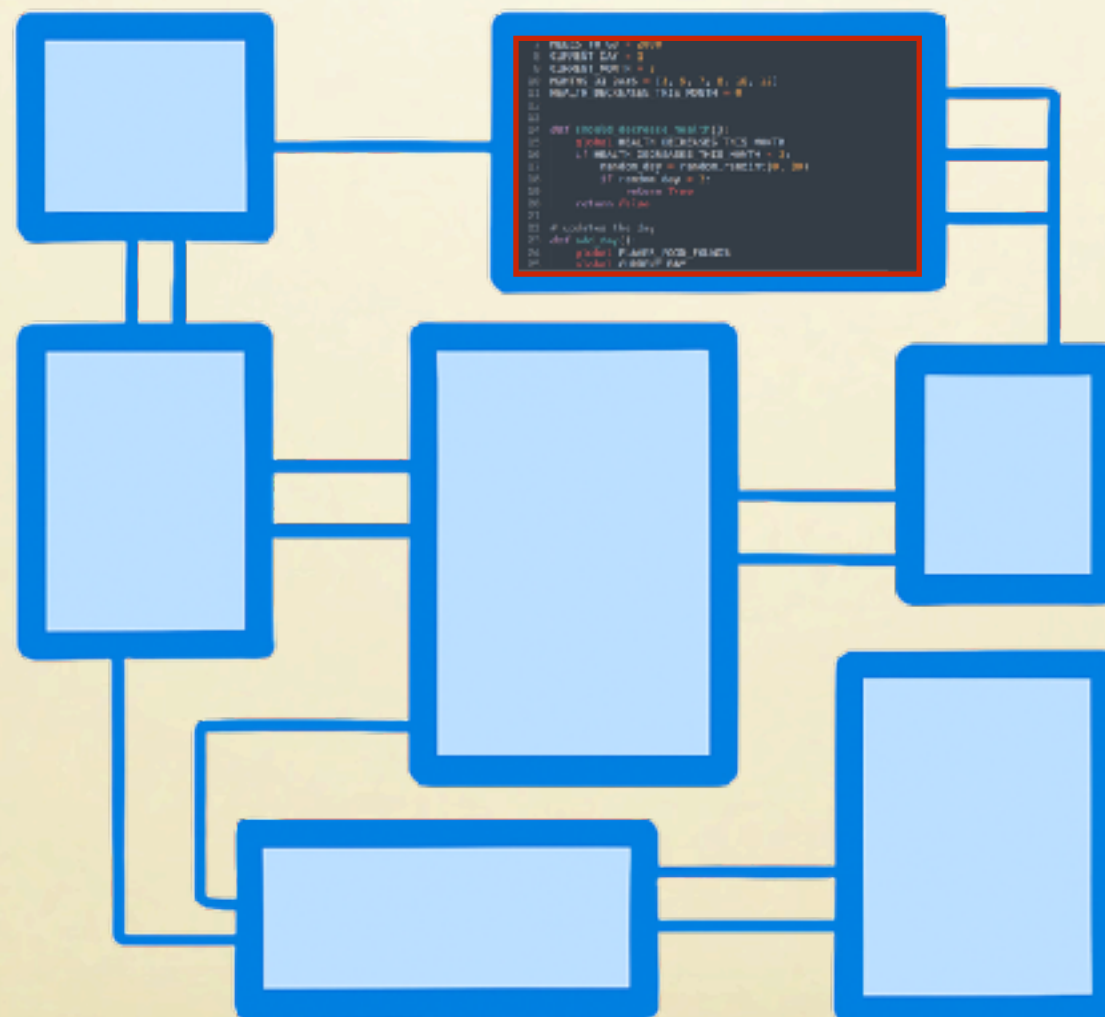


```

1  def partition(array, low, high):
2      pivot = array[high]
3      i = low - 1
4      for j in range(low, high):
5          if array[j] <= pivot:
6              i = i + 1
7              array[i], array[j] = array[j], array[i]
8      array[i + 1], array[high] = array[high], array[i + 1]
9      return i + 1
10
11 def quickSort(array, low, high):
12     if low < high:
13         pi = partition(array, low, high)
14         quickSort(array, low, pi - 1)
15         quickSort(array, pi + 1, high)
16
17 # DRIVER CODE
18 A = [99, 21, 31, 22, 11]
19 quickSort(A, 0, len(A) - 1)

```

Algebraic Semantics of Computation

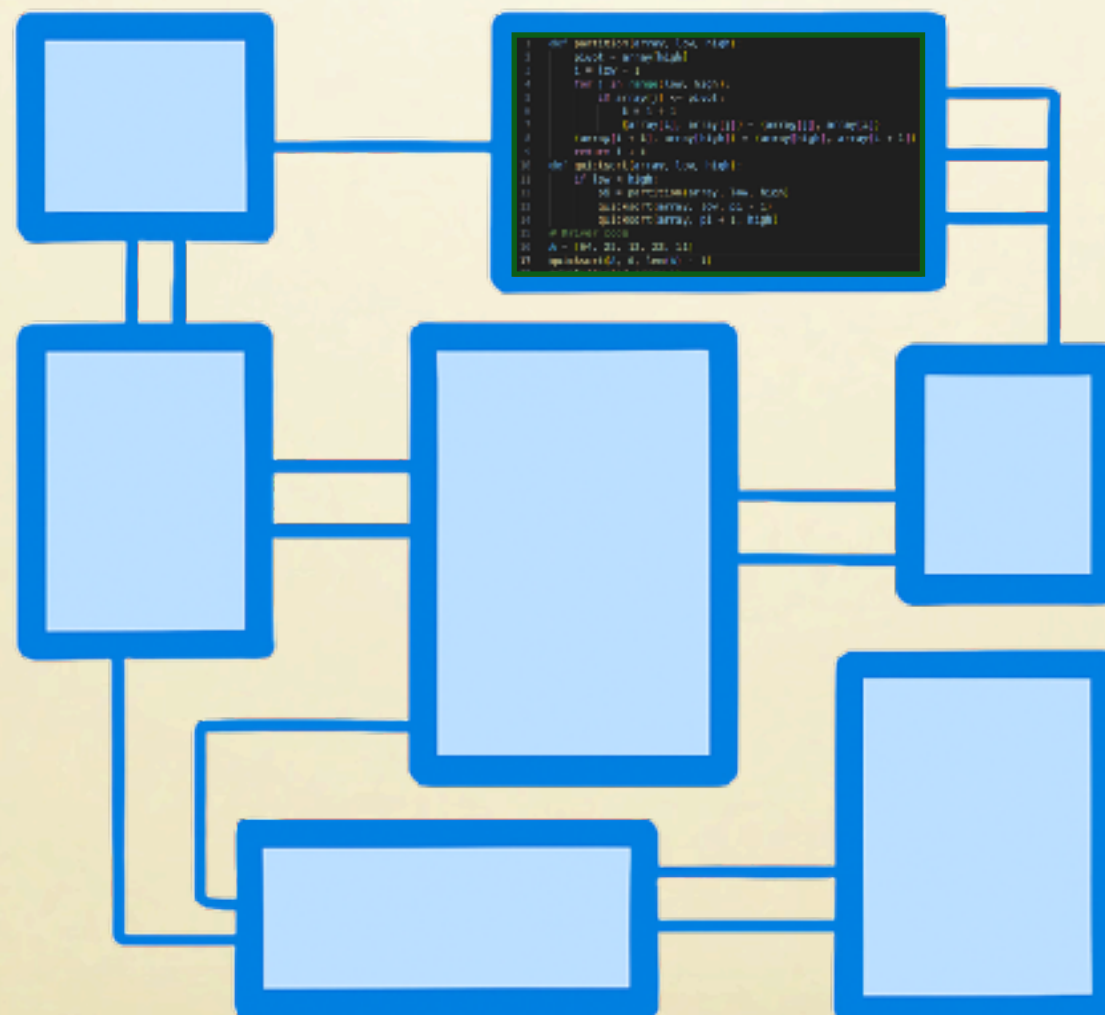


```

1  def partitionArray, low, high
2    pivot = arr[high]
3    i = low - 1
4    for j in range(low, high)
5      if arr[j] < pivot
6        i = i + 1
7        arr[j], arr[i] = arr[i], arr[j]
8    arr[i], arr[high] = arr[high], arr[i]
9    return i
10
11 def quickSortArray, low, high
12   if low < high
13     p = partitionArray, low, high
14     quickSortArray, low, p - 1
15     quickSortArray, p + 1, high
16
17 # DRIVER CODE
18 A = [10, 21, 3, 22, 11]
19 quickSortArray, 0, len(A) - 1

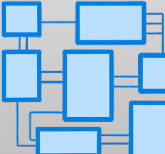
```

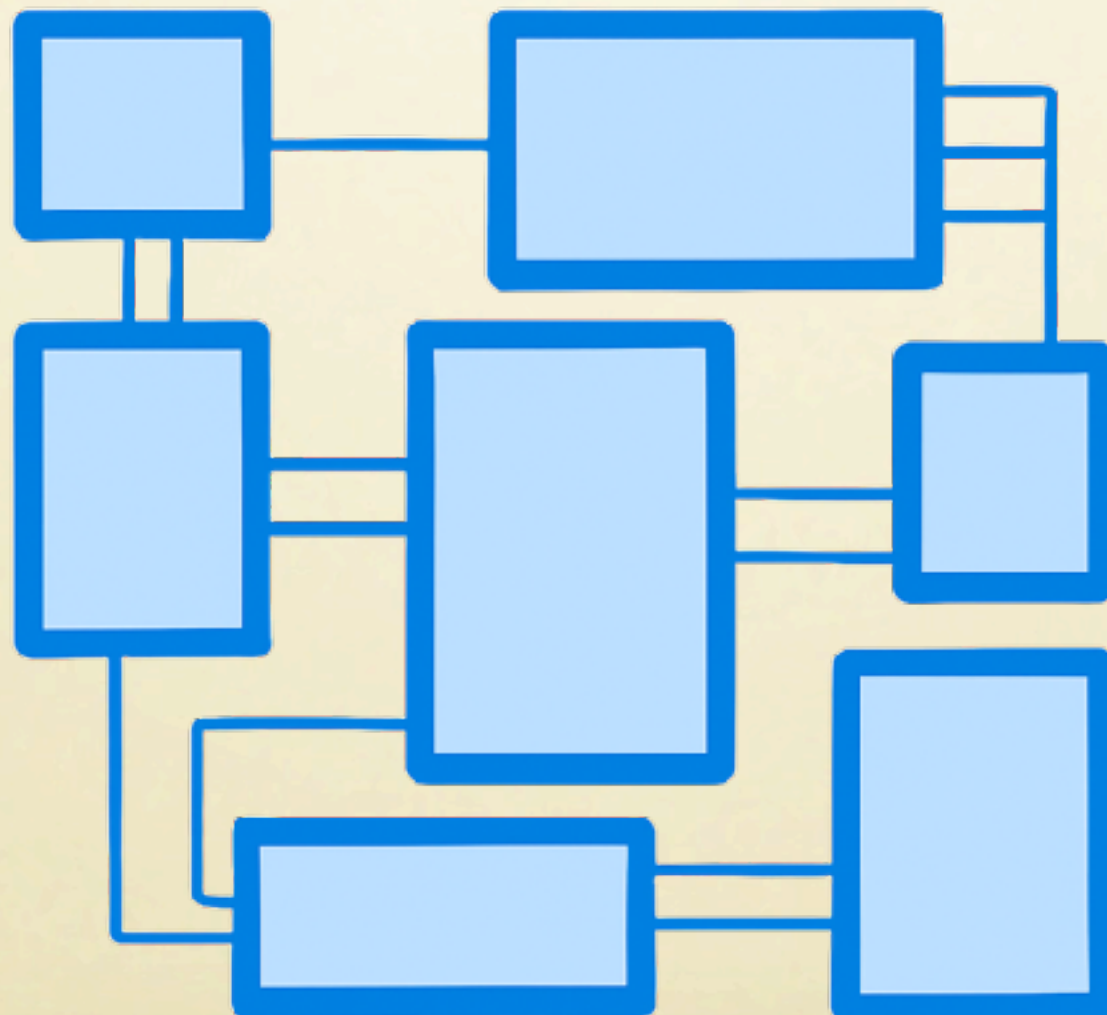
Algebraic Semantics of Computation



Algebraic Semantics of Computation

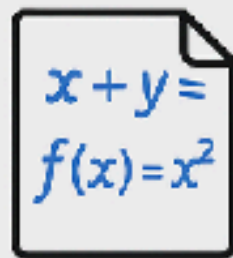
Compositionality

From the semantics of  I can derive the semantics of the whole system 



Why Algebraic Semantics Matters

Clear and Unambiguous
Description of
Program Behaviour



$x + y =$
 $f(x) = x^2$

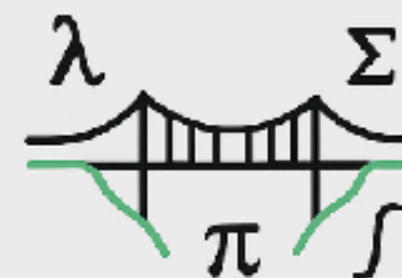
Language Design
and Optimisation



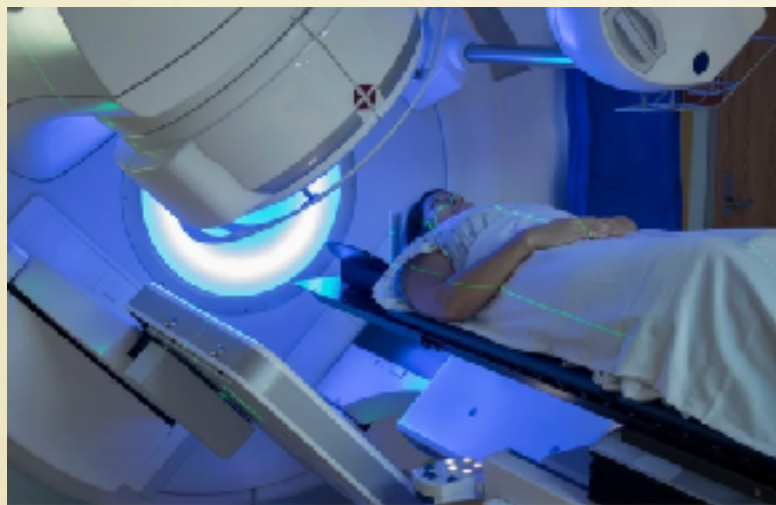
Foundation for
Verification



Bridge to other
domains



New Challenge: Non-Classical Computation



Can we also make **non-classical** forms of computation **easier to design**, **more transparent**, and **provably safe**?

The Limits of Algebraic Semantics

Non-classical computation demands a **resource sensitive analysis** beyond the expressiveness of classical algebraic semantics.

```
1  PREDICATE: 100 = 10000
2  CURRENT DAY = 1
3  CURRENT MONTH = 1
4  PREDICATE: 10000 = 10000
5  PREDICATE: 10000 = 10000
6  PREDICATE: 10000 = 10000
7  PREDICATE: 10000 = 10000
8  PREDICATE: 10000 = 10000
9  PREDICATE: 10000 = 10000
10 PREDICATE: 10000 = 10000
11 PREDICATE: 10000 = 10000
12 PREDICATE: 10000 = 10000
13 PREDICATE: 10000 = 10000
14 PREDICATE: 10000 = 10000
15 PREDICATE: 10000 = 10000
16 PREDICATE: 10000 = 10000
17 PREDICATE: 10000 = 10000
18 PREDICATE: 10000 = 10000
19 PREDICATE: 10000 = 10000
20 PREDICATE: 10000 = 10000
21 PREDICATE: 10000 = 10000
22 PREDICATE: 10000 = 10000
23 PREDICATE: 10000 = 10000
24 PREDICATE: 10000 = 10000
25 PREDICATE: 10000 = 10000
26 PREDICATE: 10000 = 10000
27 PREDICATE: 10000 = 10000
28 PREDICATE: 10000 = 10000
29 PREDICATE: 10000 = 10000
30 PREDICATE: 10000 = 10000
31 PREDICATE: 10000 = 10000
32 PREDICATE: 10000 = 10000
33 PREDICATE: 10000 = 10000
34 PREDICATE: 10000 = 10000
35 PREDICATE: 10000 = 10000
36 PREDICATE: 10000 = 10000
37 PREDICATE: 10000 = 10000
38 PREDICATE: 10000 = 10000
39 PREDICATE: 10000 = 10000
40 PREDICATE: 10000 = 10000
41 PREDICATE: 10000 = 10000
42 PREDICATE: 10000 = 10000
43 PREDICATE: 10000 = 10000
44 PREDICATE: 10000 = 10000
45 PREDICATE: 10000 = 10000
46 PREDICATE: 10000 = 10000
47 PREDICATE: 10000 = 10000
48 PREDICATE: 10000 = 10000
49 PREDICATE: 10000 = 10000
50 PREDICATE: 10000 = 10000
51 PREDICATE: 10000 = 10000
52 PREDICATE: 10000 = 10000
53 PREDICATE: 10000 = 10000
54 PREDICATE: 10000 = 10000
55 PREDICATE: 10000 = 10000
56 PREDICATE: 10000 = 10000
57 PREDICATE: 10000 = 10000
58 PREDICATE: 10000 = 10000
59 PREDICATE: 10000 = 10000
60 PREDICATE: 10000 = 10000
61 PREDICATE: 10000 = 10000
62 PREDICATE: 10000 = 10000
63 PREDICATE: 10000 = 10000
64 PREDICATE: 10000 = 10000
65 PREDICATE: 10000 = 10000
66 PREDICATE: 10000 = 10000
67 PREDICATE: 10000 = 10000
68 PREDICATE: 10000 = 10000
69 PREDICATE: 10000 = 10000
70 PREDICATE: 10000 = 10000
71 PREDICATE: 10000 = 10000
72 PREDICATE: 10000 = 10000
73 PREDICATE: 10000 = 10000
74 PREDICATE: 10000 = 10000
75 PREDICATE: 10000 = 10000
76 PREDICATE: 10000 = 10000
77 PREDICATE: 10000 = 10000
78 PREDICATE: 10000 = 10000
79 PREDICATE: 10000 = 10000
80 PREDICATE: 10000 = 10000
81 PREDICATE: 10000 = 10000
82 PREDICATE: 10000 = 10000
83 PREDICATE: 10000 = 10000
84 PREDICATE: 10000 = 10000
85 PREDICATE: 10000 = 10000
86 PREDICATE: 10000 = 10000
87 PREDICATE: 10000 = 10000
88 PREDICATE: 10000 = 10000
89 PREDICATE: 10000 = 10000
90 PREDICATE: 10000 = 10000
91 PREDICATE: 10000 = 10000
92 PREDICATE: 10000 = 10000
93 PREDICATE: 10000 = 10000
94 PREDICATE: 10000 = 10000
95 PREDICATE: 10000 = 10000
96 PREDICATE: 10000 = 10000
97 PREDICATE: 10000 = 10000
98 PREDICATE: 10000 = 10000
99 PREDICATE: 10000 = 10000
100 PREDICATE: 10000 = 10000
```



$s(y, t(z), t(z))$

The Limits of Algebraic Semantics

Non-classical computation demands a **resource sensitive analysis** beyond the expressiveness of classical algebraic semantics.

```
1 PRED: FRUIT = APPLE  
2 CURRENT DAY = 1  
3 CURRENT MONTH = 1  
4 MONTHS_AHEAD = (1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12)  
5 MONTHS_SECONDS_THIS_MONTH = 0  
6  
7  
8  
9  
10 GET_RANDOM_MONTH_YEAR():  
11   1. PRINT: "MONTH: " & MONTHS_AHEAD & " YEARS: " & CURRENT_YEAR  
12   2. MONTHS_AHEAD = MONTHS_AHEAD + 1  
13   3. random day = random.randint(1, 28)  
14   4. if random day < 1:  
15     return None  
16   5. return (MONTHS_AHEAD, CURRENT_YEAR)  
17  
18 # calculate the day  
19 def calc_day():  
20   1. PRINT: "MONTH: " & MONTHS_AHEAD & " YEARS: " & CURRENT_YEAR  
21   2. PRINT: "DAY: " & random.randint(1, 28)
```

→ $s(y, t(z), t(z))$

Is resource z **consumed** once or twice?

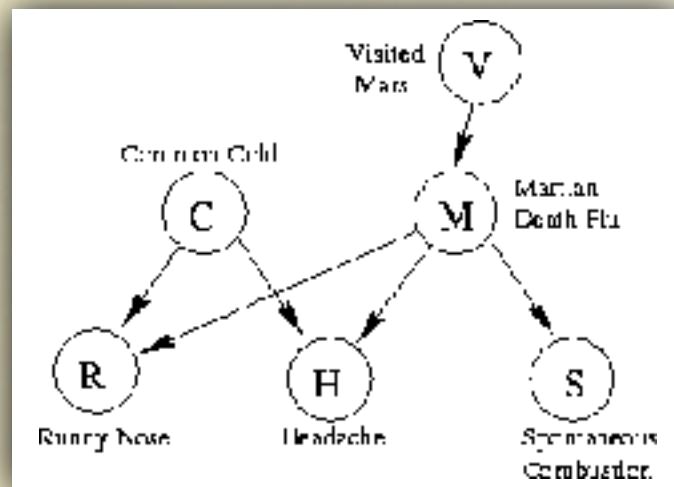
```
z <- Bernoulli(0.5)  
return (z, z)
```

VS

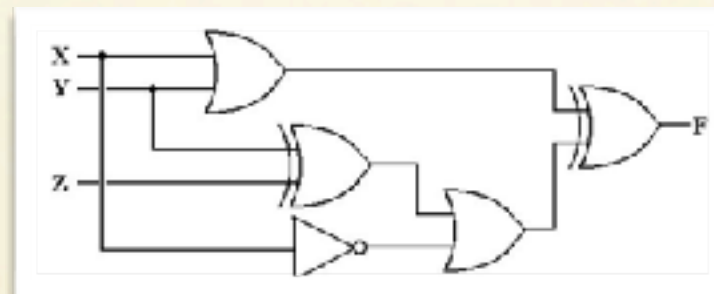
```
return (Bernoulli(0.5), Bernoulli(0.5))
```

Graphical Models of Computation

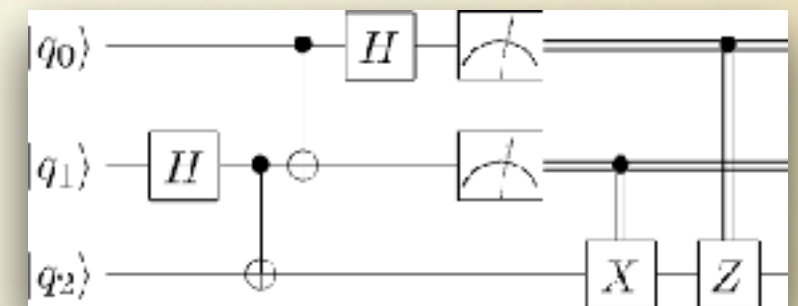
Bayesian networks



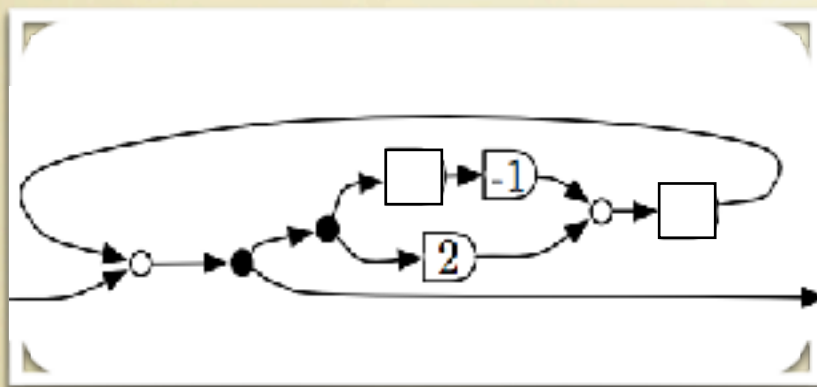
Digital Circuits



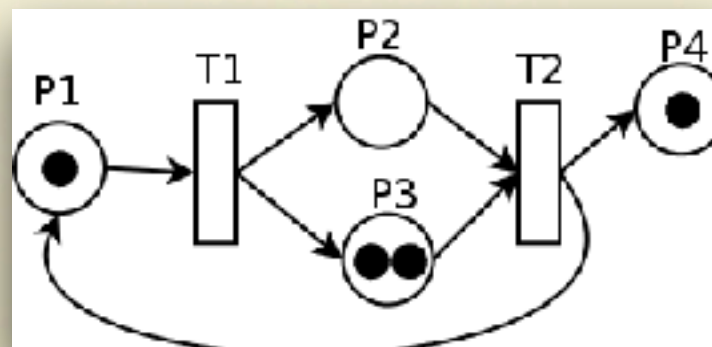
Quantum circuits



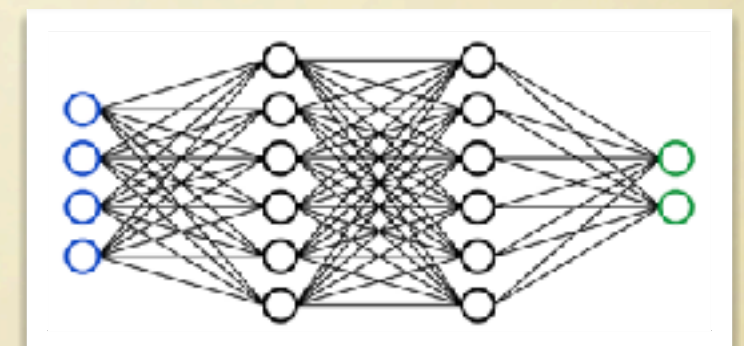
Signal flow graphs



Petri nets



Neural Networks



Back to Resource-Sensitivity

```
1 PRED: 1000 = 0.000
2 CURRENT DAY = 1
3 CURRENT MONTH = 1
4 MONTHS_AHEAD = (1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12)
5 MONTHS_AHEAD_THIS_MONTH = 0
6
7
8
9
10 GET INCOME DECREASE: 0.000(1)
11   1.000: HEALTH DECREASE: 100.000
12   1.000: DECREASE: 100.000
13   random day = random.randint(1, 12)
14   if random day < 1:
15     return True
16   return False
17
18
19
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100
```

→ $s(y, t(z), t(z))$

Is resource z **consumed** once or twice?

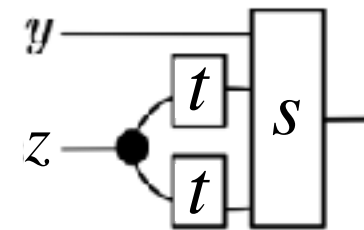
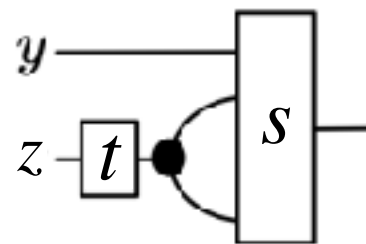
```
z <- Bernoulli(0.5)
return (z,z)
```

VS

```
return (Bernoulli(0.5), Bernoulli(0.5))
```

Back to Resource-Sensitivity

A graphical representation can tell the difference.



```
1. PROCS TO GO = 0.000
2. CURRENT DAY = 1
3. CURRENT MONTH = 1
4. MONTHS AT RISK = (1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12)
5. MONTHS SINCE LAST TIME = 0
6.
7.
8.
9.
10.
11.
12.
13.
14.
15.
16.
17.
18.
19.
20.
21.
22.
23.
24.
25.
26.
27.
28.
29.
30.
31.
32.
33.
34.
35.
36.
37.
38.
39.
40.
41.
42.
43.
44.
45.
46.
47.
48.
49.
50.
51.
52.
53.
54.
55.
56.
57.
58.
59.
60.
61.
62.
63.
64.
65.
66.
67.
68.
69.
70.
71.
72.
73.
74.
75.
76.
77.
78.
79.
80.
81.
82.
83.
84.
85.
86.
87.
88.
89.
90.
91.
92.
93.
94.
95.
96.
97.
98.
99.
100.
```

$s(y, t(z), t(z))$

Is resource z **consumed** once or twice?

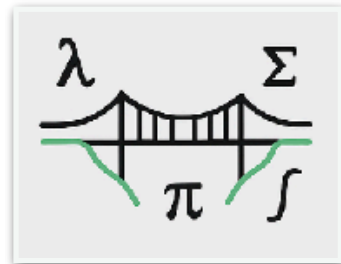
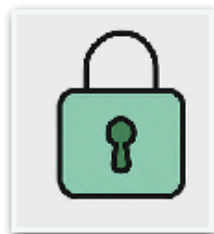
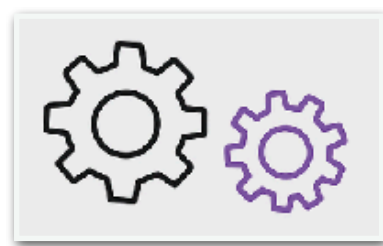
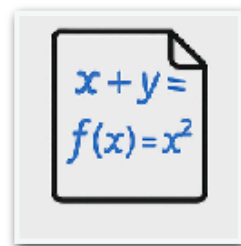
```
z <- Bernoulli(0.5)
return (z,z)
```

VS

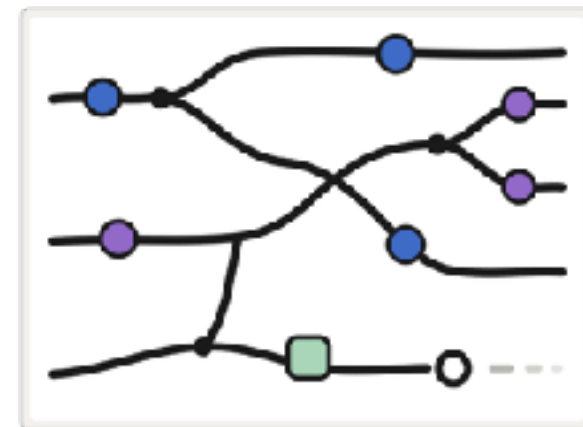
```
return (Bernoulli(0.5), Bernoulli(0.5))
```

The Best of Two Worlds?

Algebraic Semantics



Graphical Models



Challenge: a resource-sensitive graphical formalism that still enjoys the benefits of algebraic semantics

Enter String Diagrams

String Diagrams originated in **category theory** as a notation for morphisms of **monoidal categories**

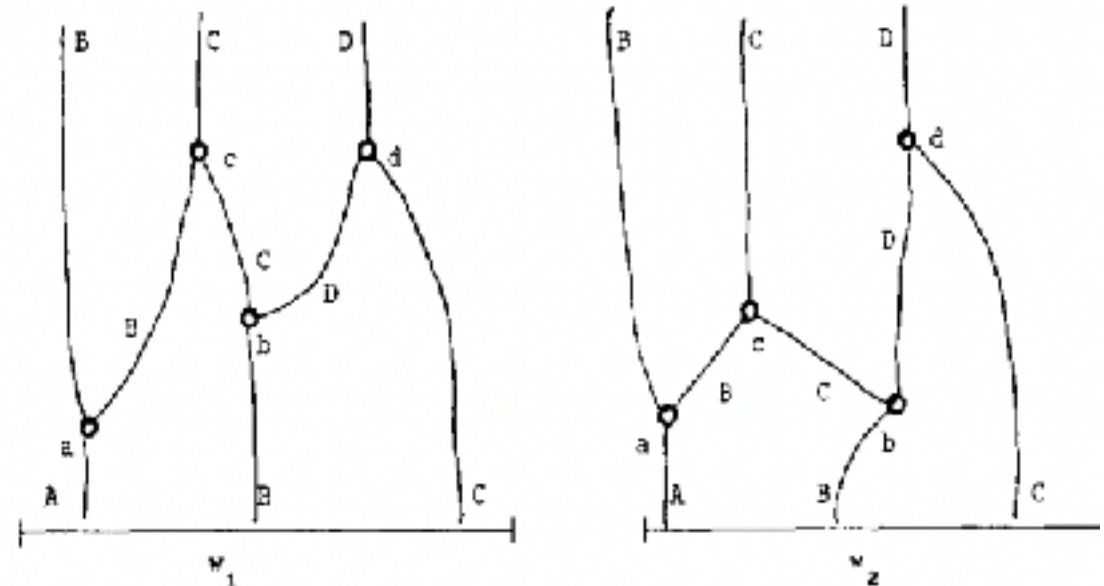
ADVANCES IN MATHEMATICS **88**, 55–112 (1991)

JOYAL AND STREET

The Geometry of Tensor Calculus, I

$$(B \otimes c \otimes d) \circ (B \otimes B \otimes b \otimes C) \circ (a \otimes B \otimes C) = w_1$$

$$(B \otimes C \otimes d) \circ (B \otimes c \otimes D \otimes C) \circ (a \otimes b \otimes C) = w_2$$



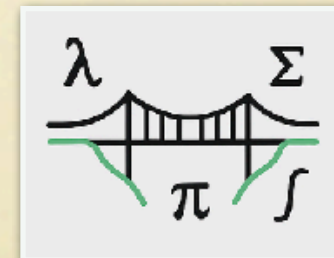
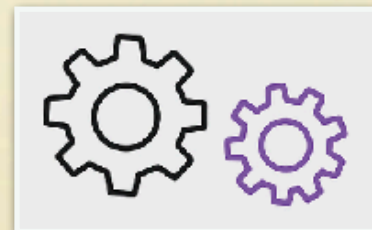
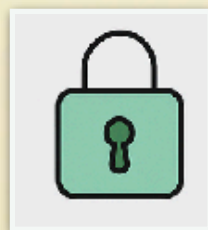
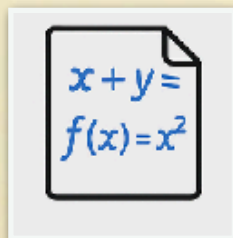
A Modern Take on String Diagrams

String Diagrams serve as a **syntax** for graphical models.

In the same sense as logic/programming languages

$$\begin{array}{lcl} b & ::= & \text{True} \mid x = y \mid x = n \mid \neg b \mid b \wedge b \mid b \vee b \\ p & ::= & \text{skip} \mid x := y \mid x := s(y) \mid x := n \mid \text{while } b \text{ do } p \mid p ; p \end{array}$$

The categorical roots of string diagrams enable **algebraic semantics** for graphical models.



String Diagrammatic Syntax



String Diagrammatic Syntax

basic wiring



String Diagrammatic Syntax

basic wiring

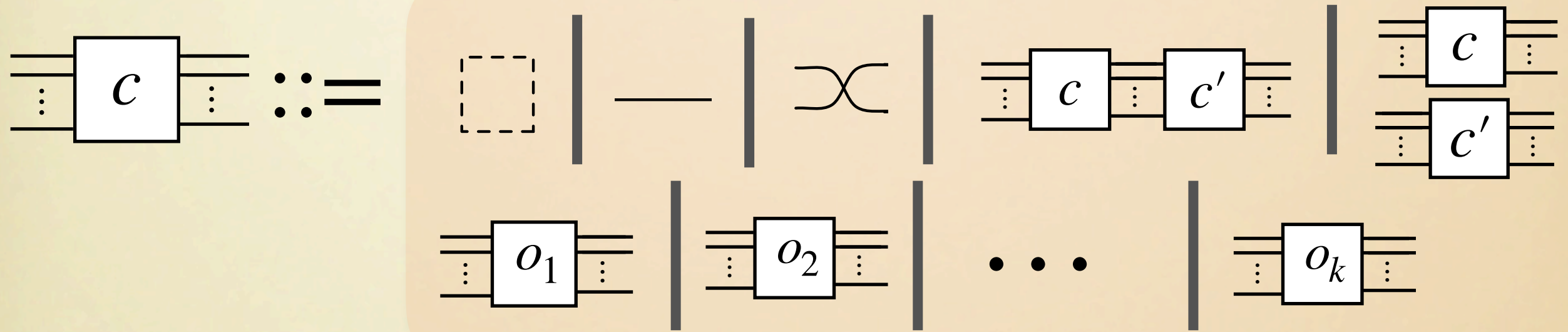
two forms of composition



String Diagrammatic Syntax

basic wiring

two forms of composition

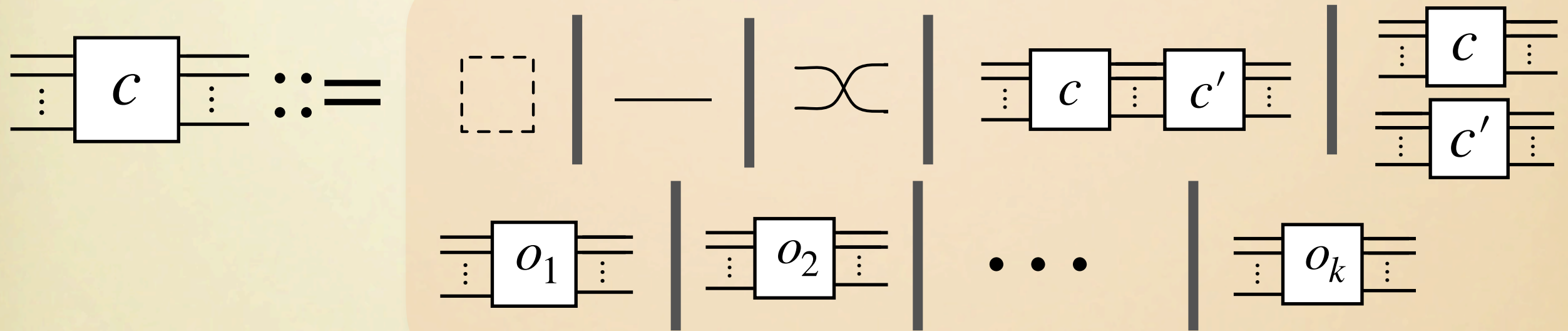


domain specific operations

String Diagrammatic Syntax

basic wiring

two forms of composition



domain specific operations

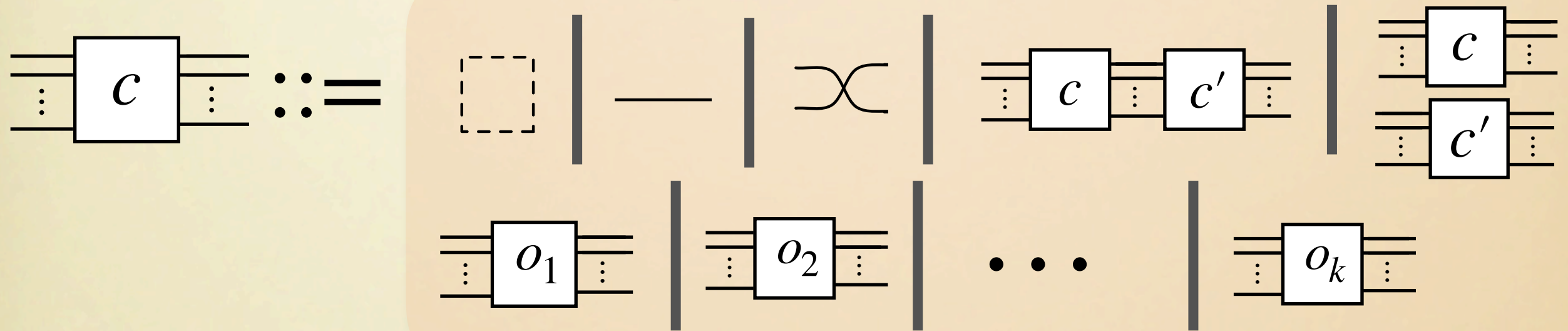
Examples



String Diagrammatic Syntax

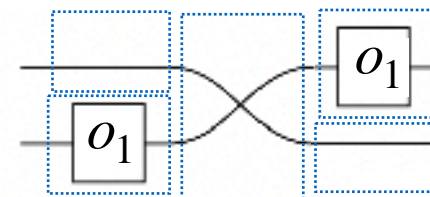
basic wiring

two forms of composition



domain specific operations

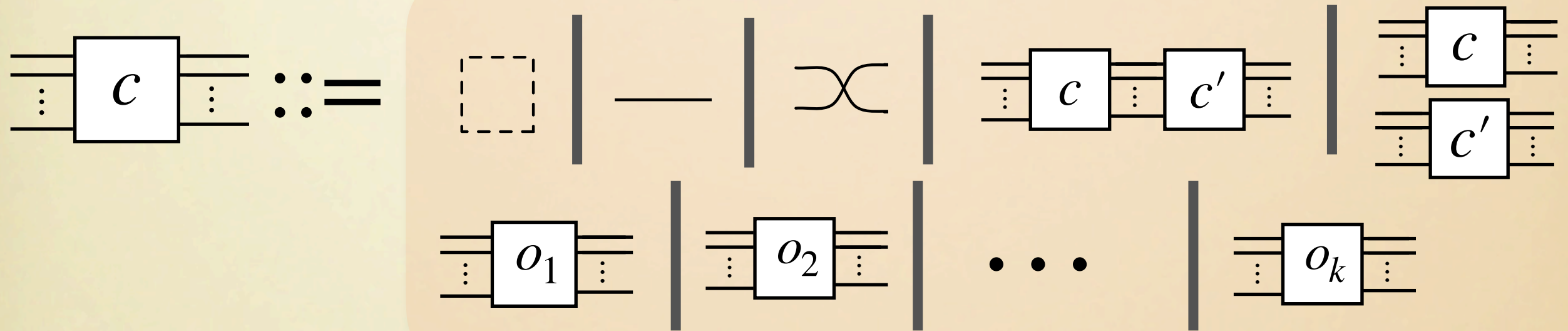
Examples



String Diagrammatic Syntax

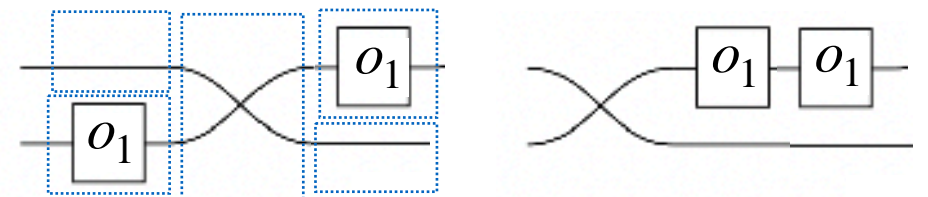
basic wiring

two forms of composition



domain specific operations

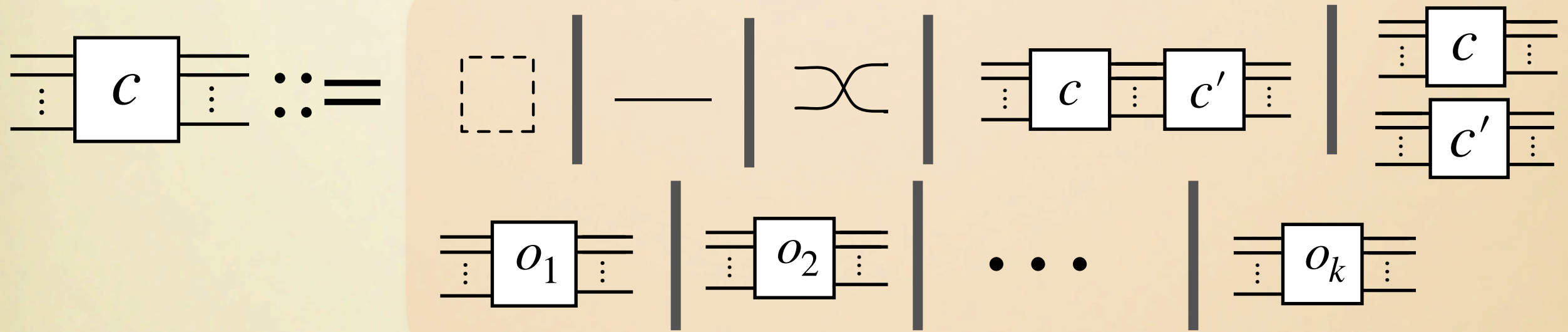
Examples



String Diagrammatic Syntax

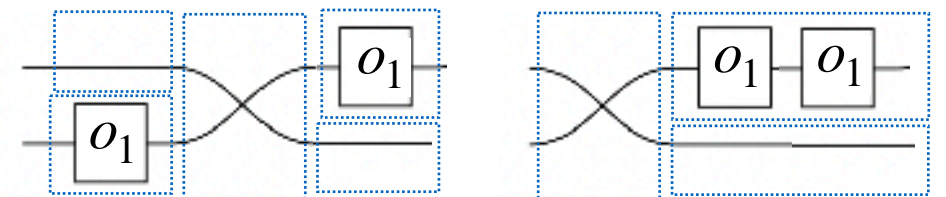
basic wiring

two forms of composition



domain specific operations

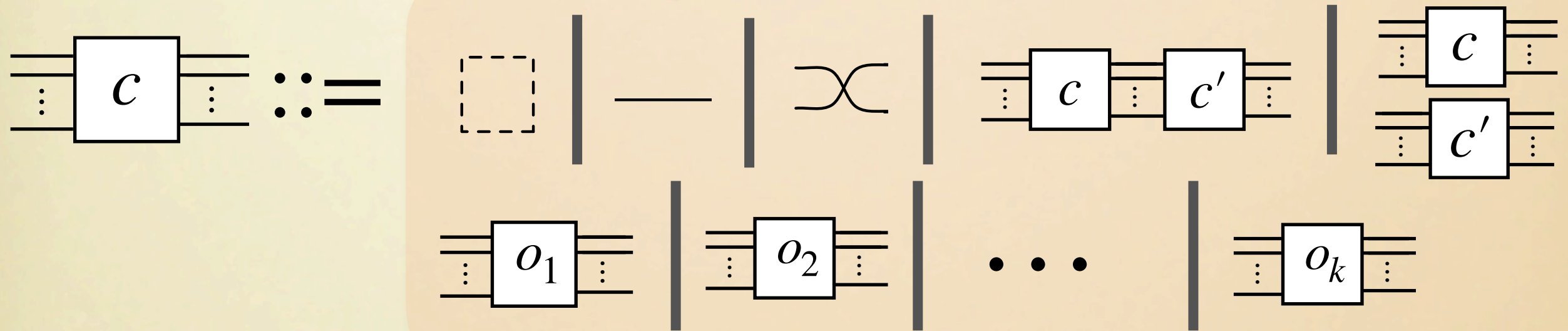
Examples



String Diagrammatic Syntax

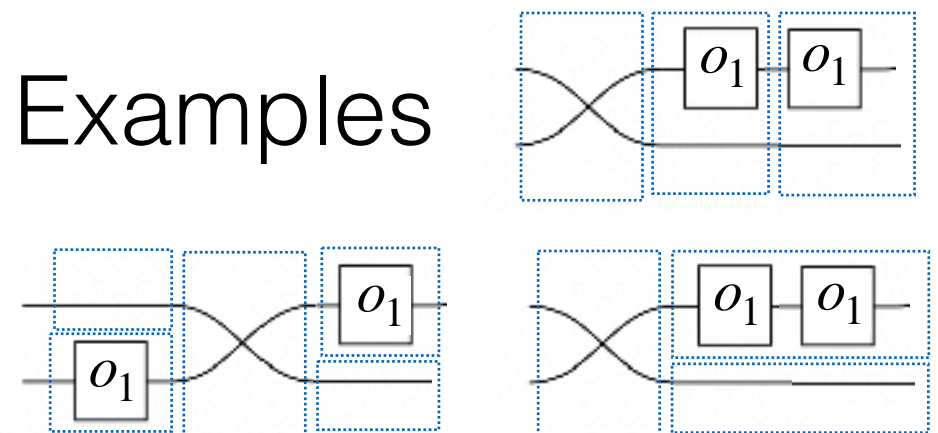
basic wiring

two forms of composition



domain specific operations

Examples

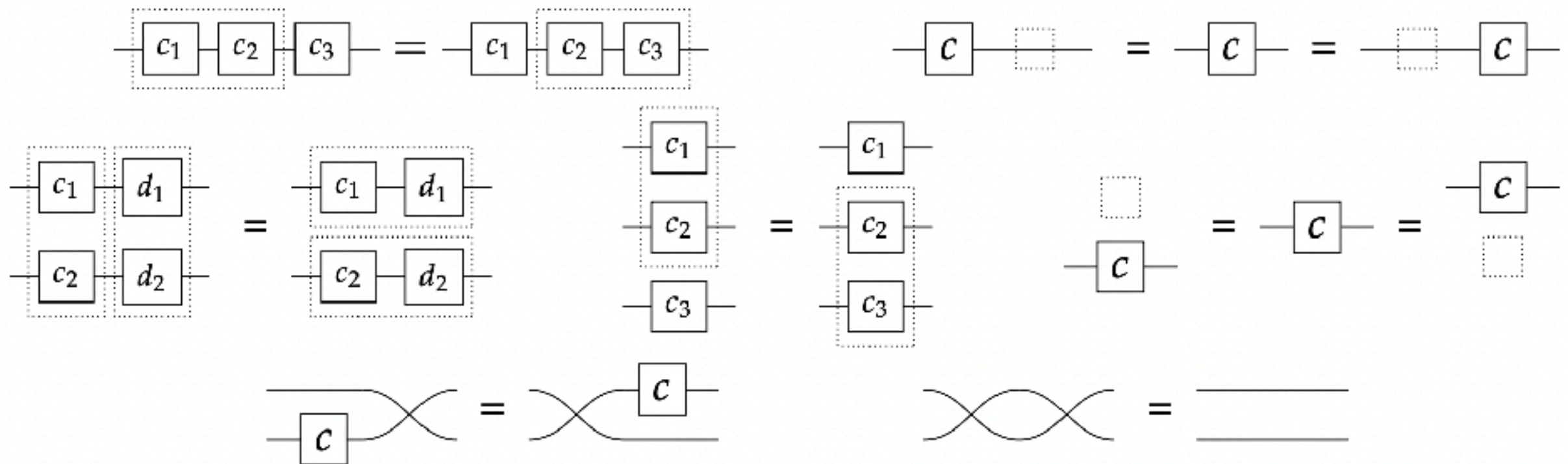


String Diagrammatic Syntax

String diagrammatic syntax is defined
modulo certain equations:

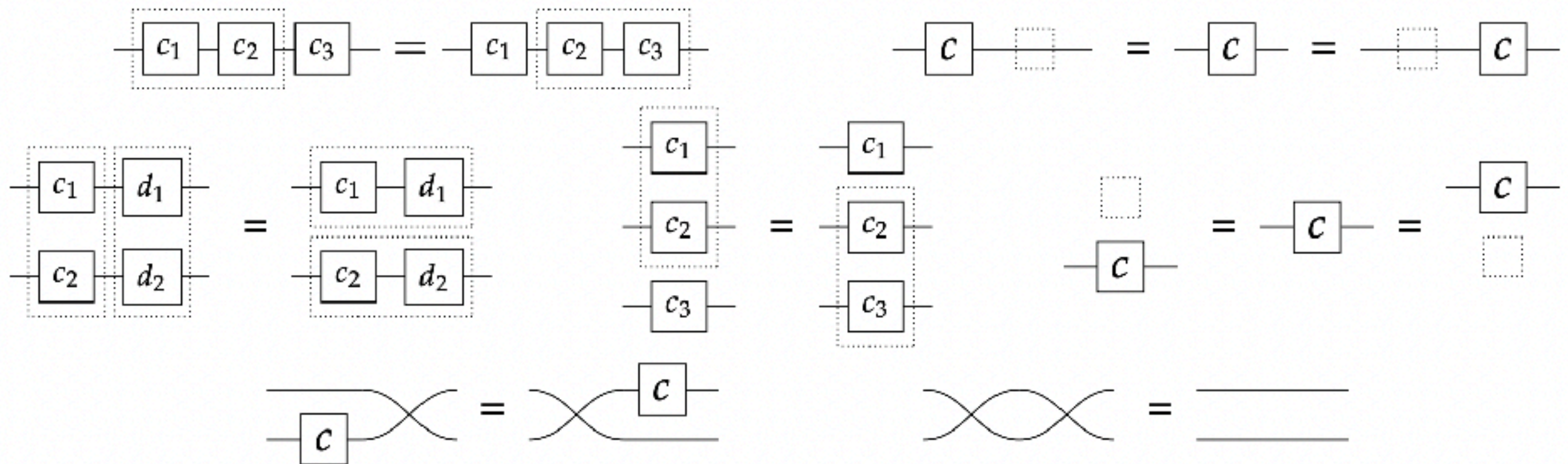
String Diagrammatic Syntax

String diagrammatic syntax is defined modulo certain equations:



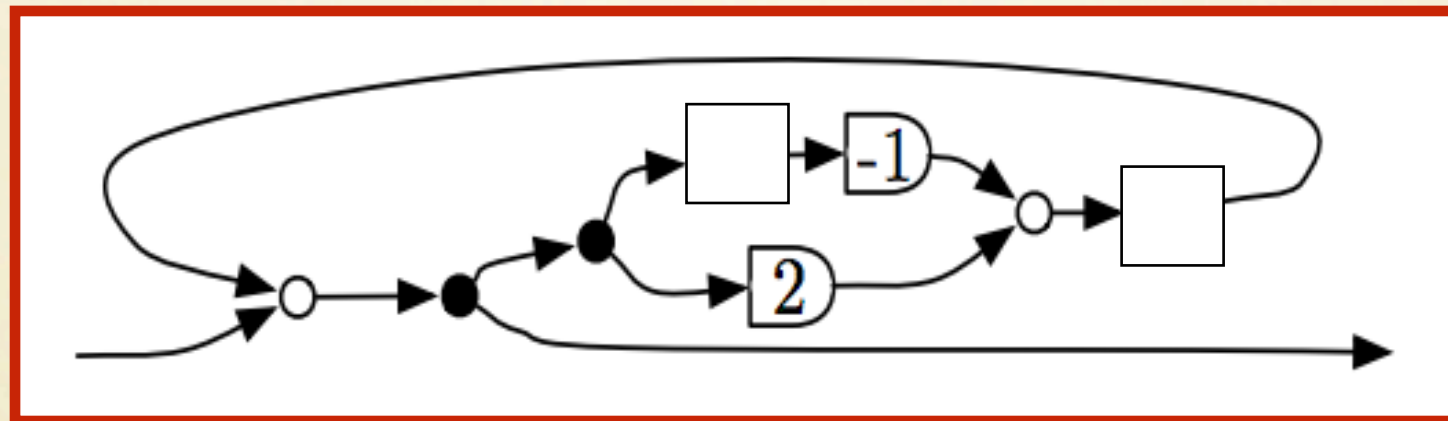
String Diagrammatic Syntax

String diagrammatic syntax is defined modulo certain equations:



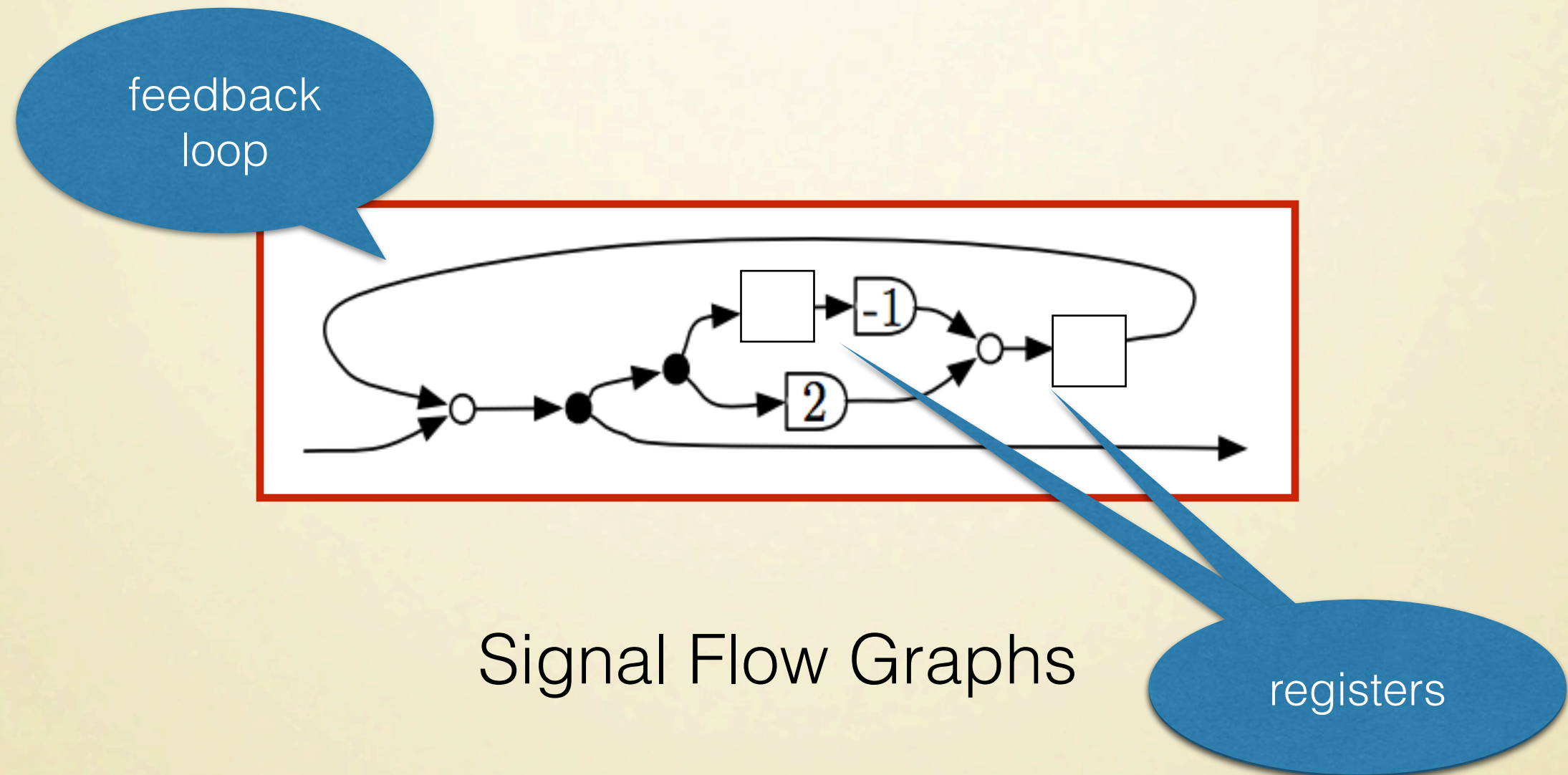
These equations allow us to reason formally about string diagrams interchangeably as **syntax** and as **graphs**.

Example: Signal Flow Graphs



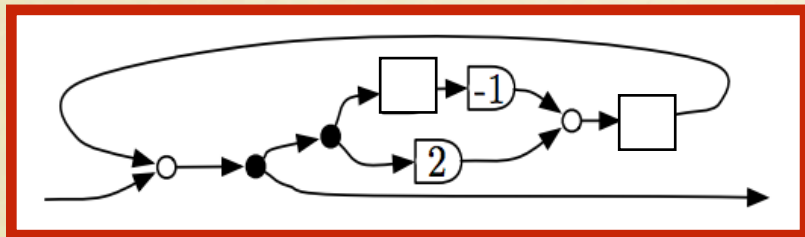
Signal Flow Graphs
(Shannon 1942)

Example: Signal Flow Graphs

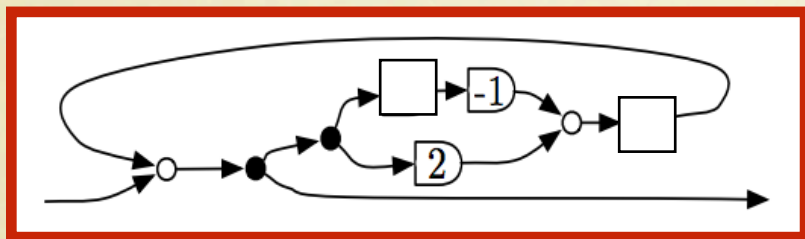
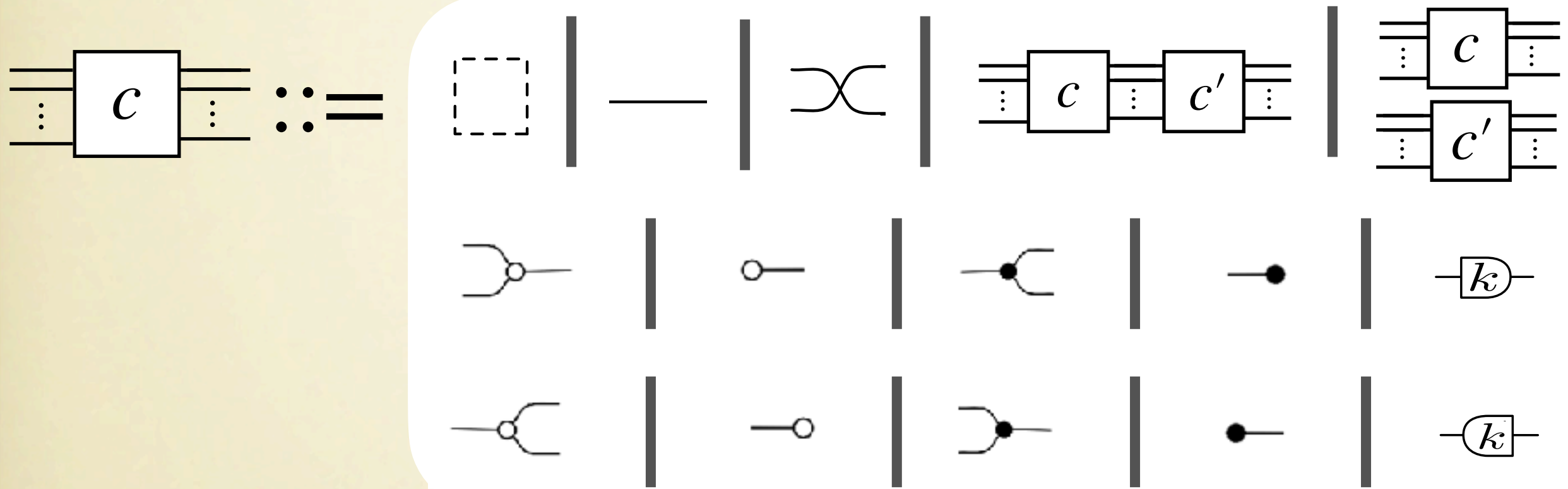


Signal Flow Graphs
(Shannon 1942)

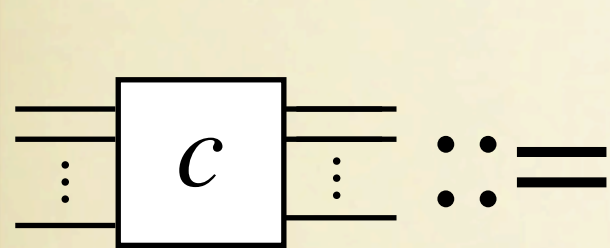
Example: Signal Flow Graphs



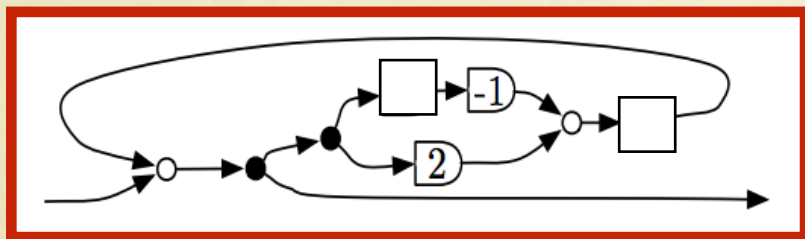
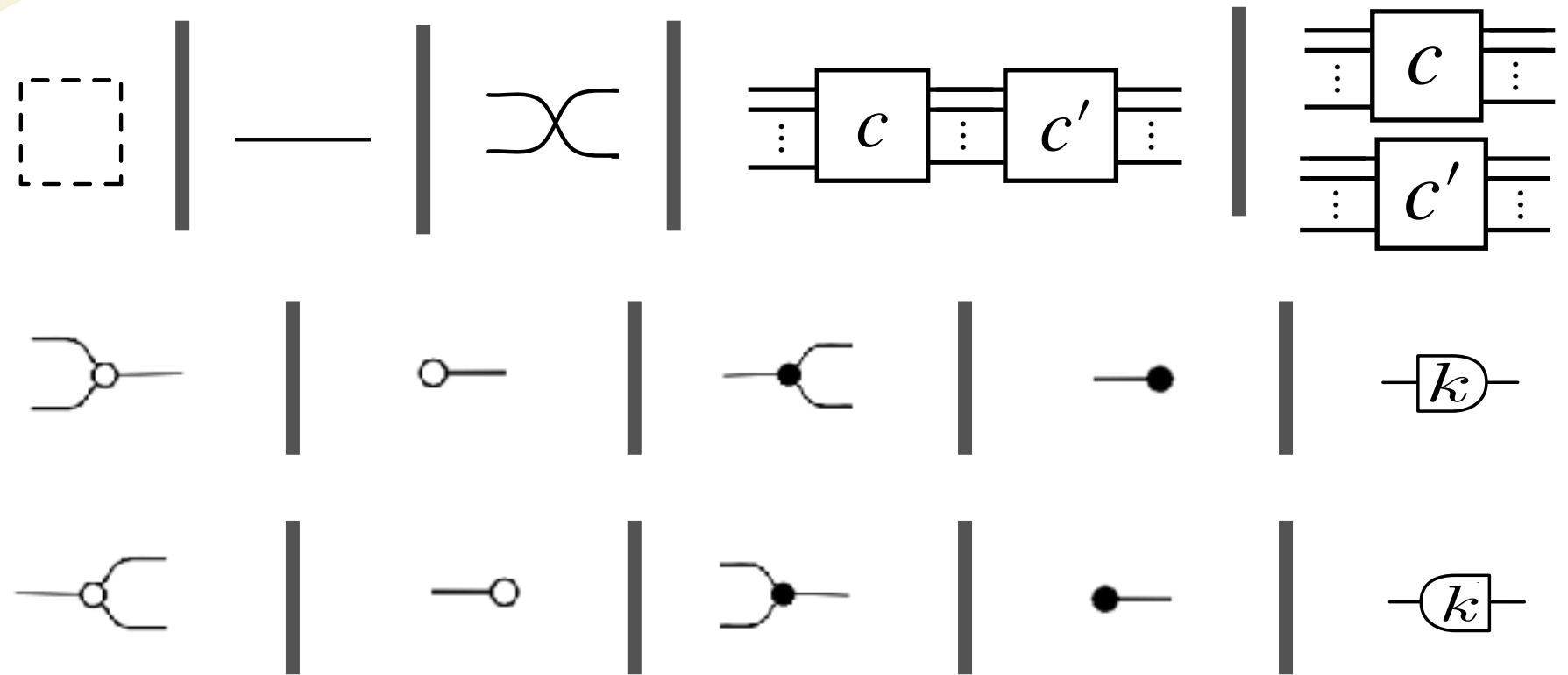
Example: Signal Flow Graphs



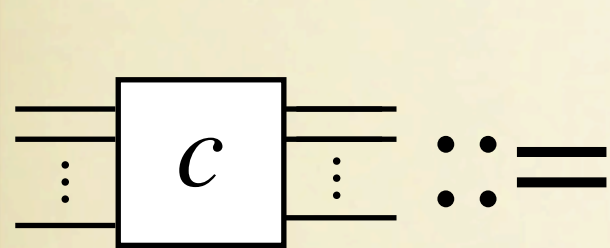
Example: Signal Flow Graphs



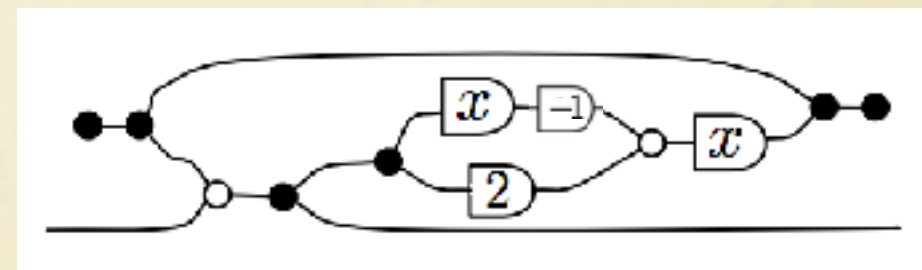
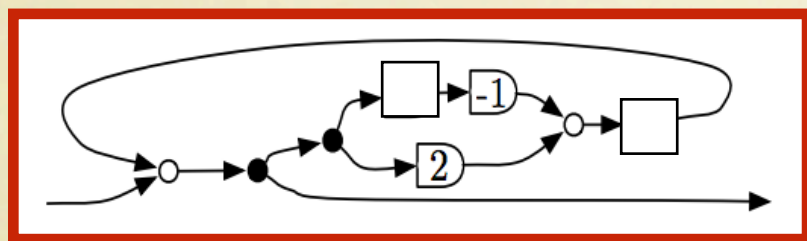
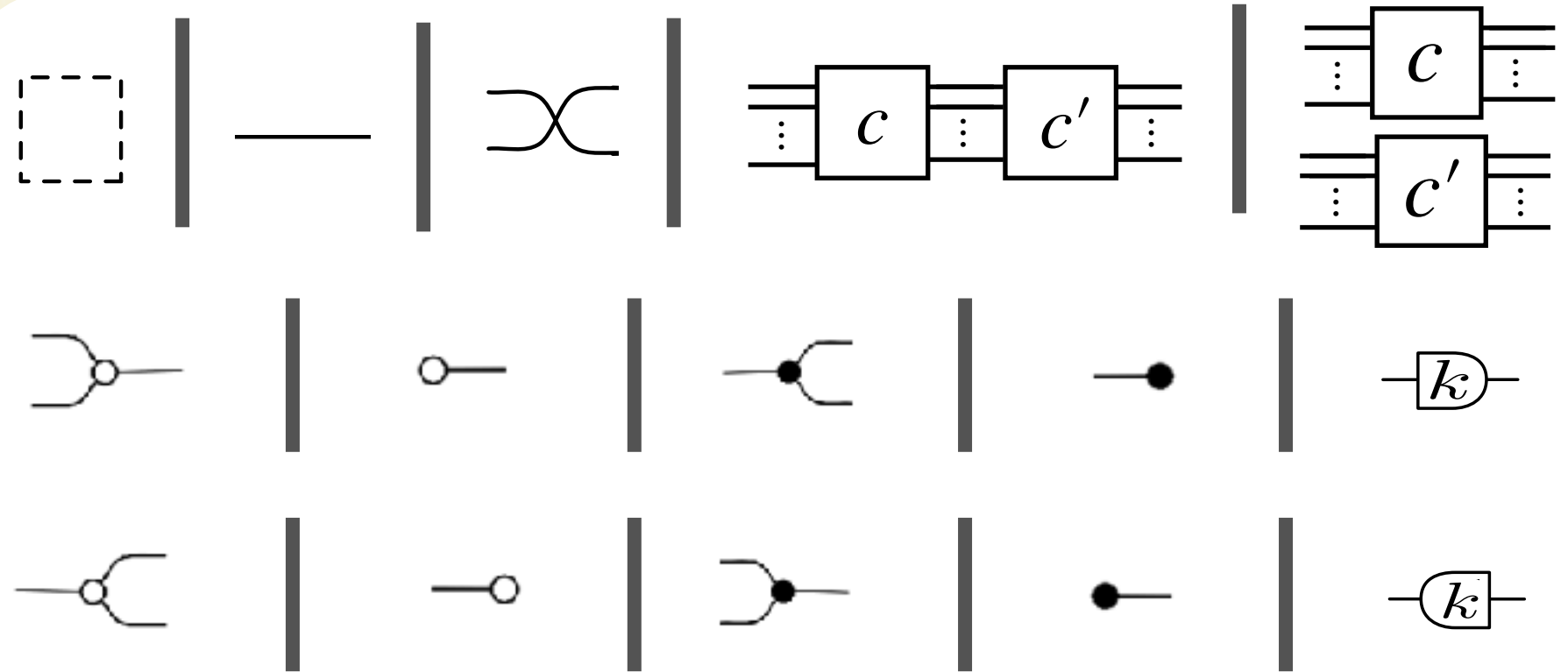
$k \in R$
In this case,
 $R = \mathbb{R}[x]$



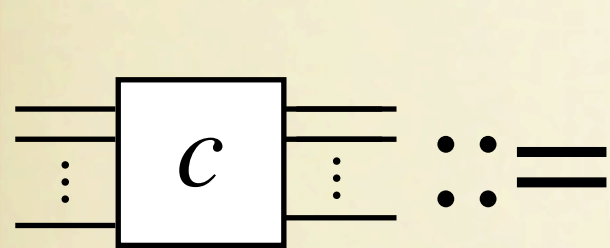
Example: Signal Flow Graphs



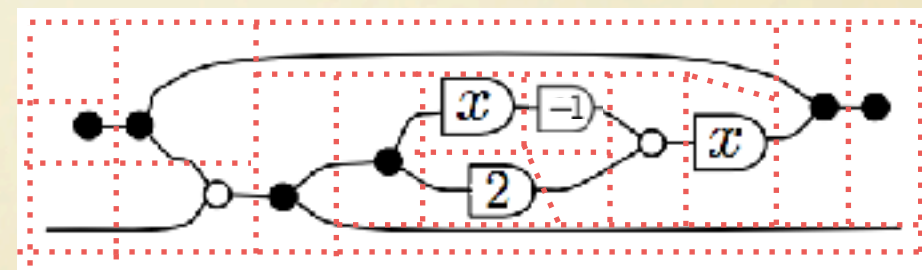
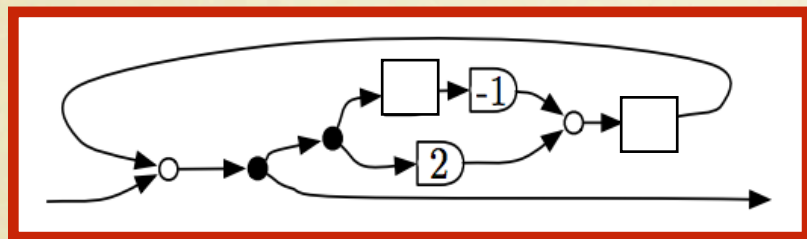
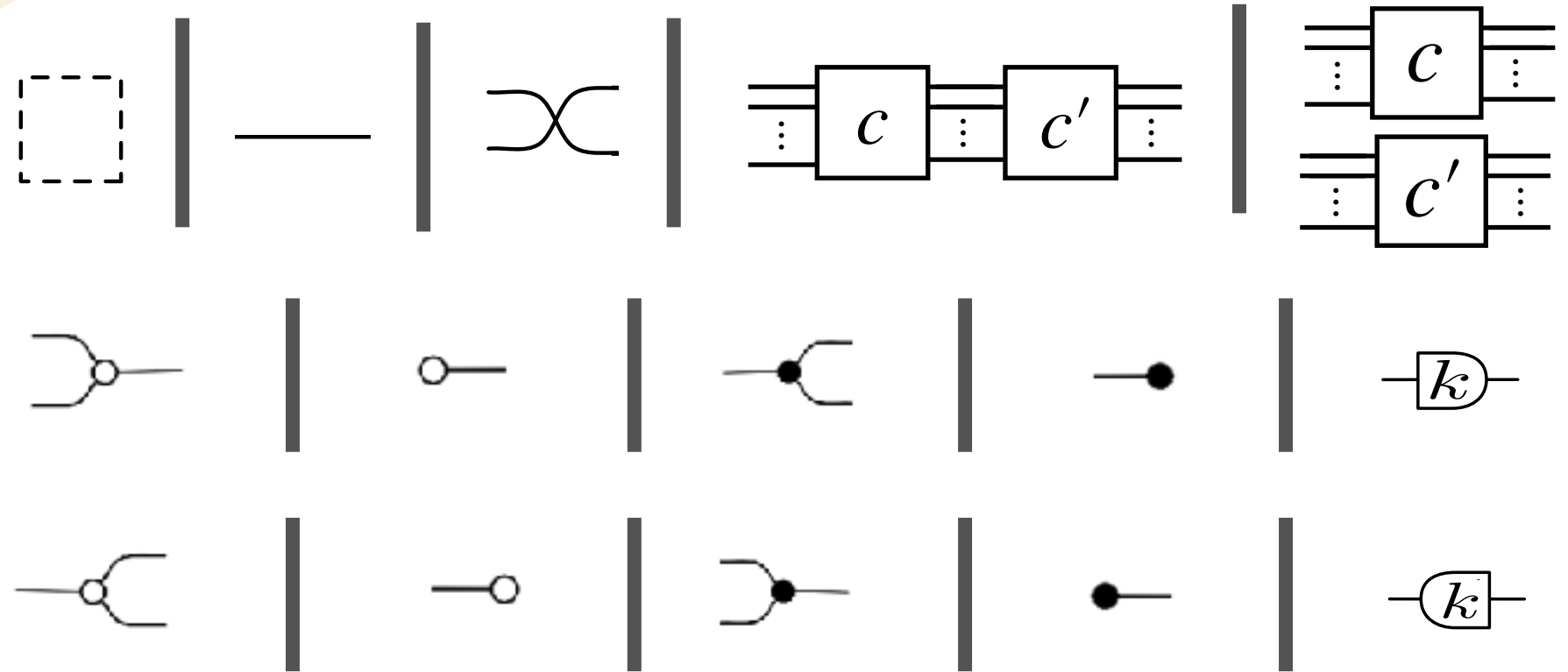
$k \in R$
In this case,
 $R = \mathbb{R}[x]$



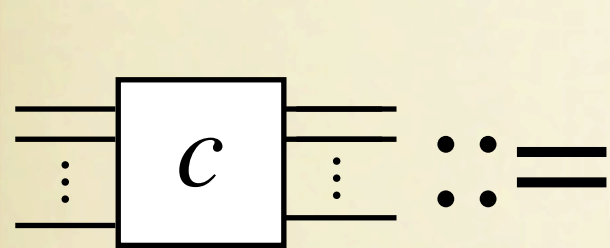
Example: Signal Flow Graphs



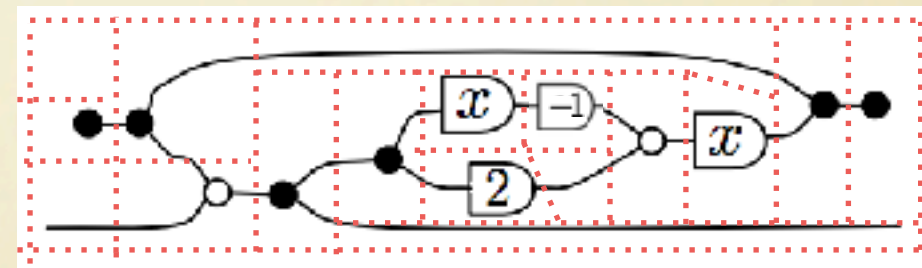
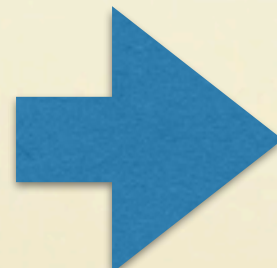
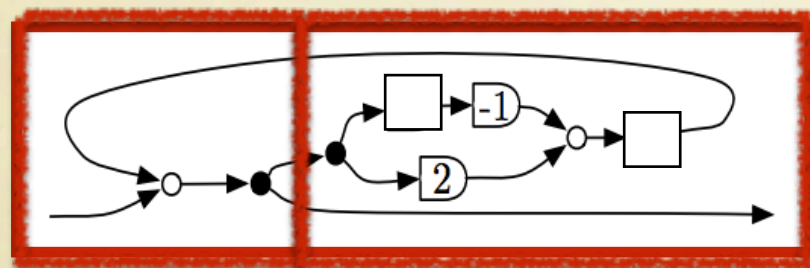
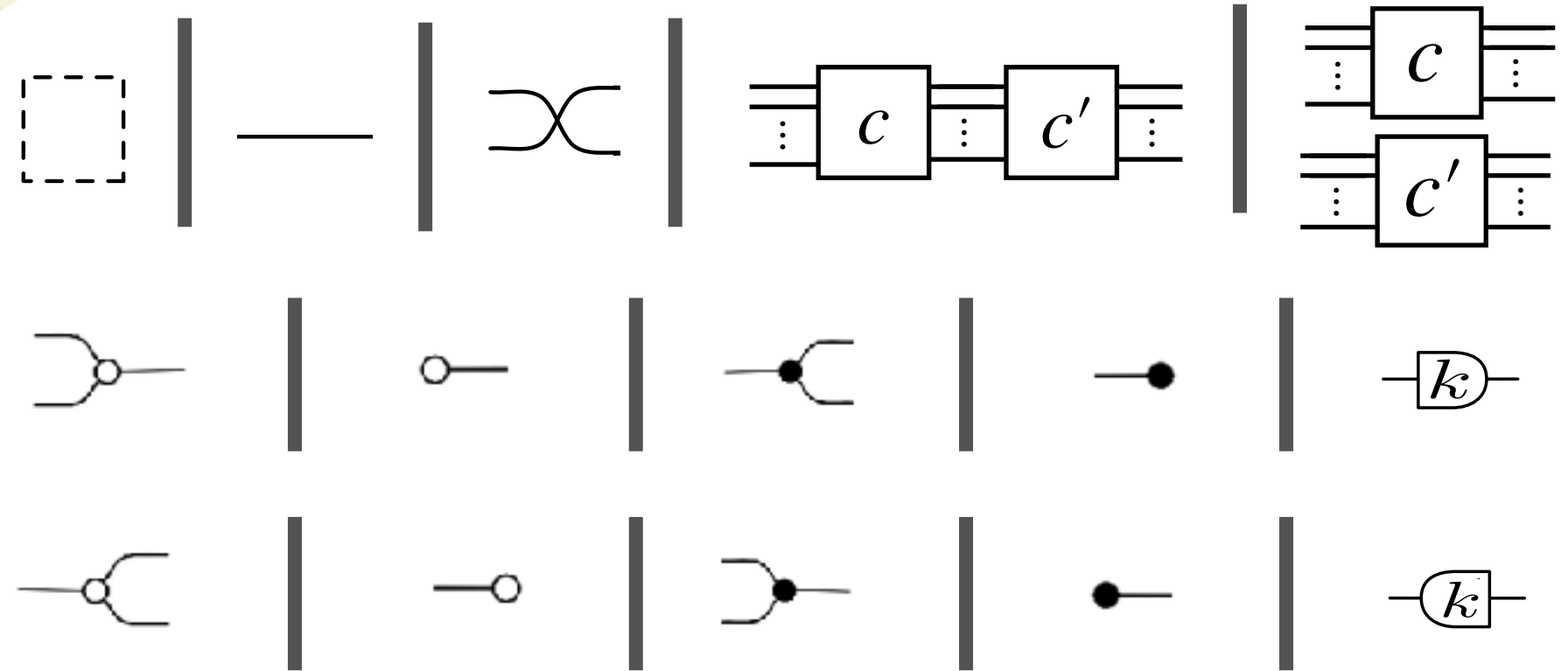
$k \in R$
In this case,
 $R = \mathbb{R}[x]$



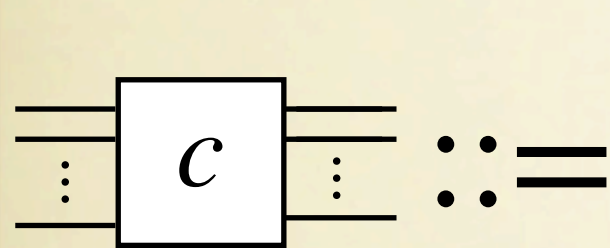
Example: Signal Flow Graphs



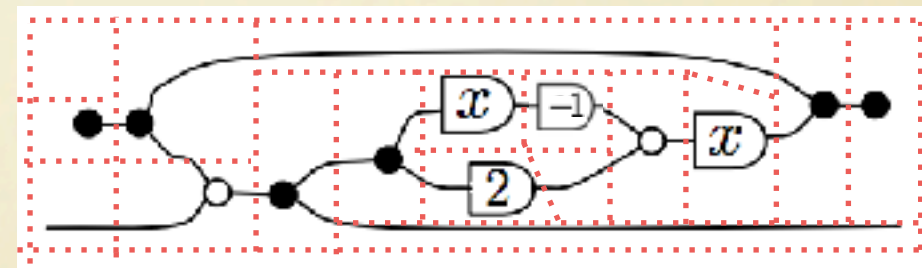
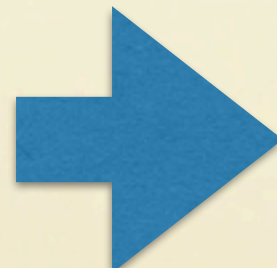
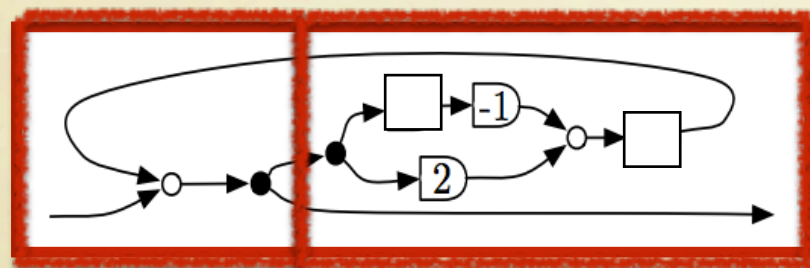
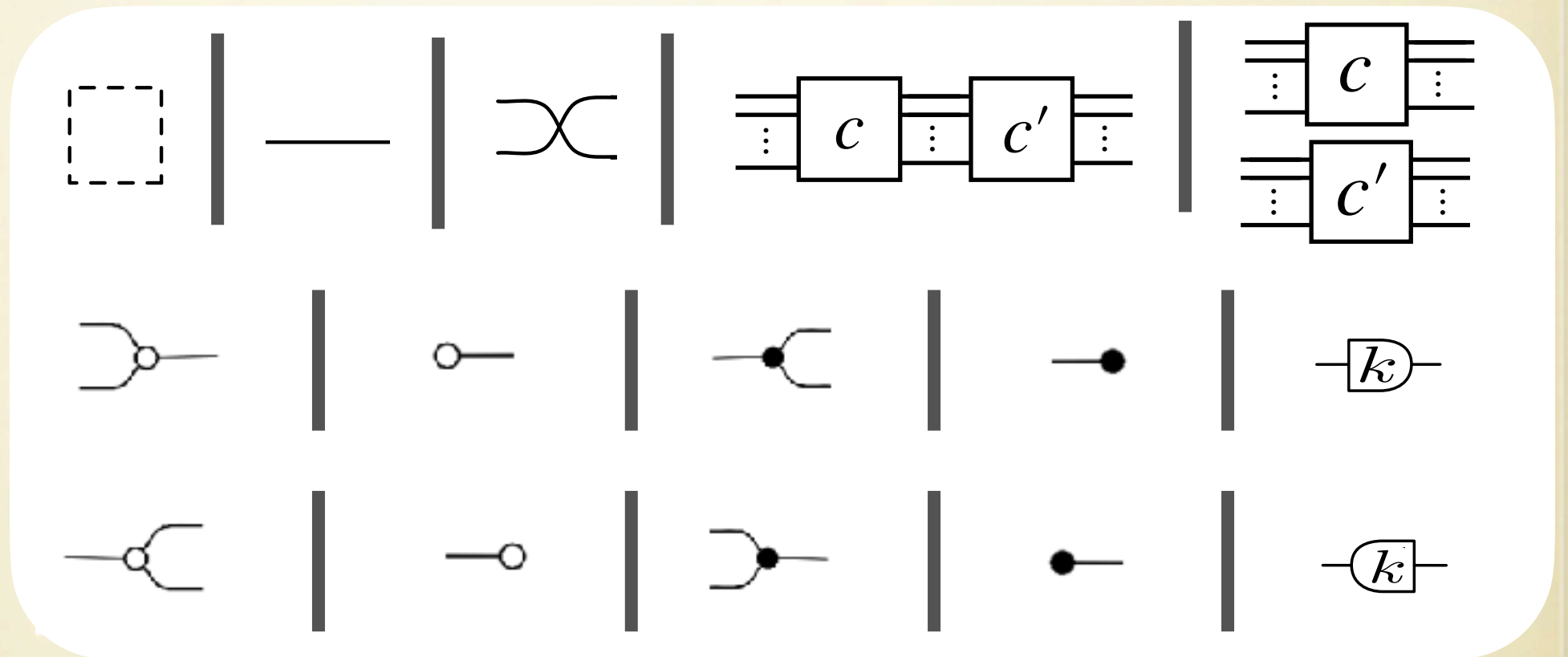
$k \in R$
In this case,
 $R = \mathbb{R}[x]$



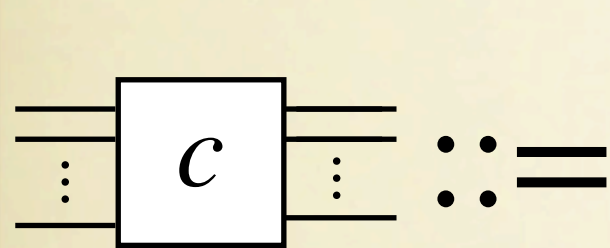
Example: Signal Flow Graphs



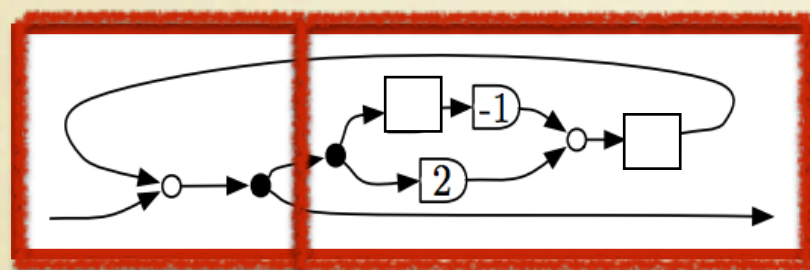
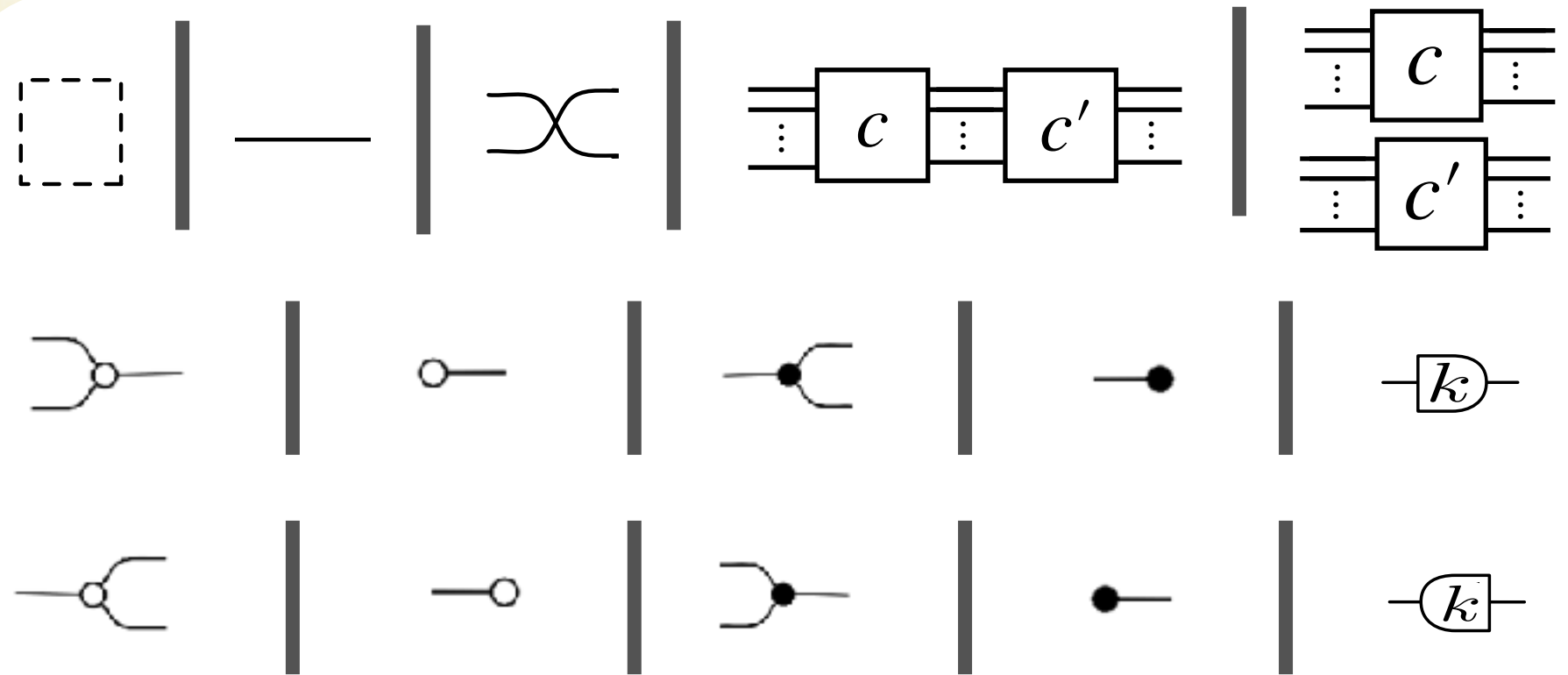
$k \in R$
In this case,
 $R = \mathbb{R}[x]$



Example: Signal Flow Graphs

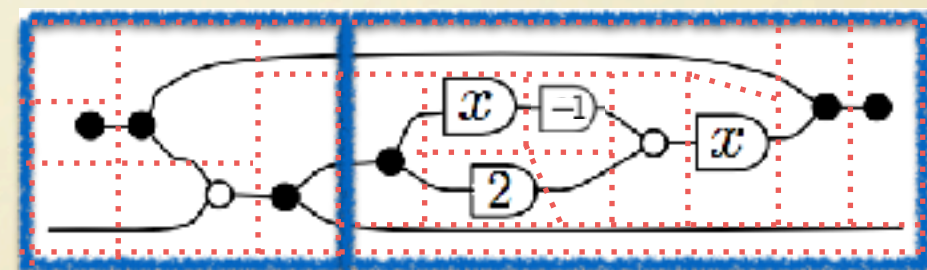


$k \in R$
In this case,
 $R = \mathbb{R}[x]$

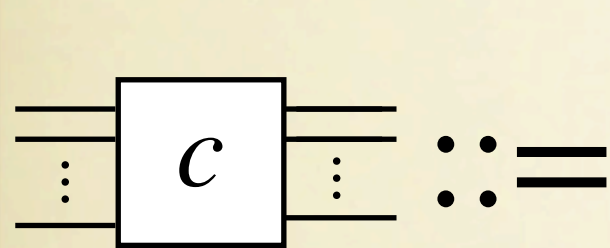


$\updownarrow$
?

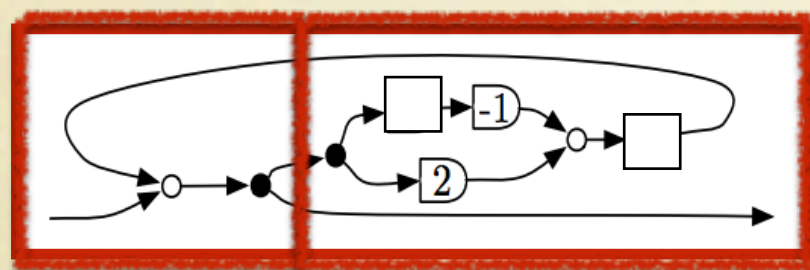
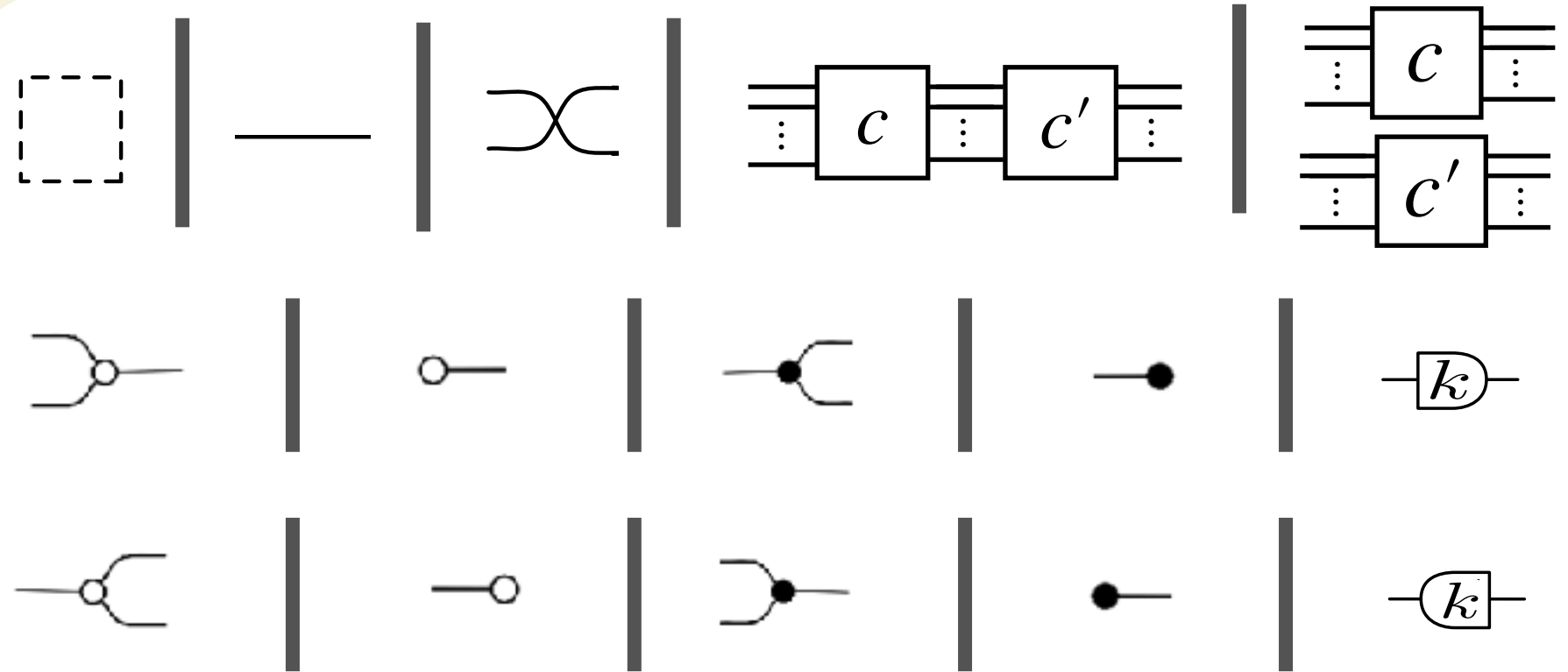
$\updownarrow$
?



Example: Signal Flow Graphs

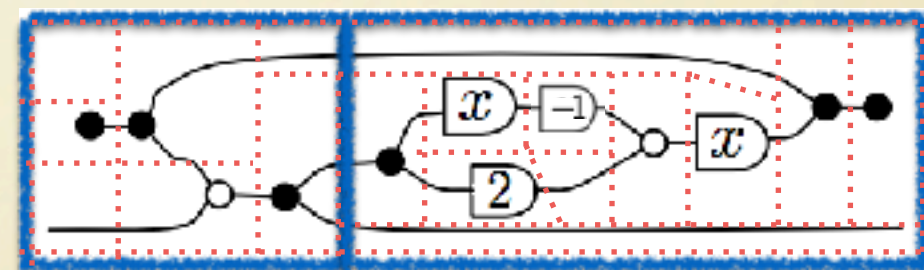


$k \in R$
In this case,
 $R = \mathbb{R}[x]$



$\updownarrow$
?

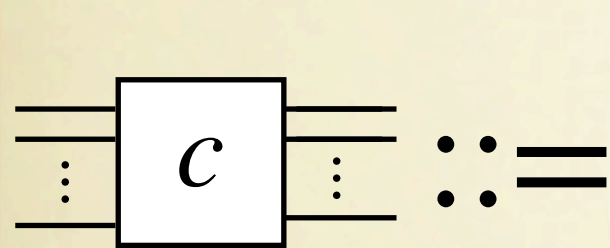
$\updownarrow$
?



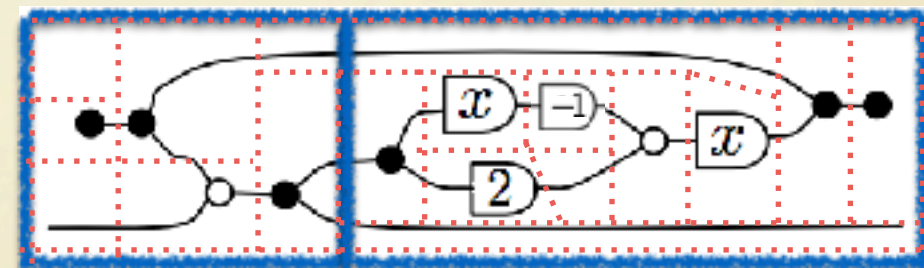
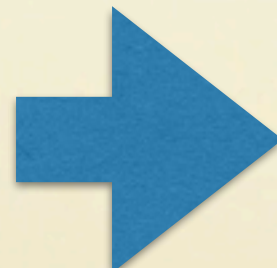
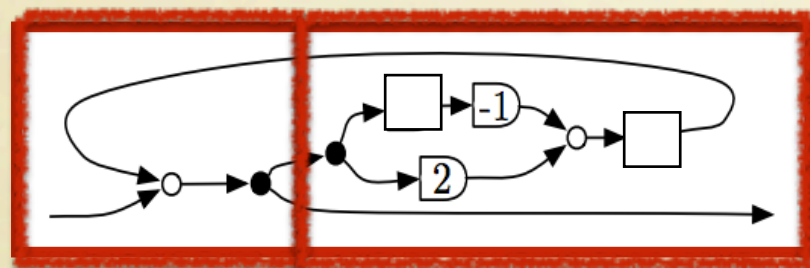
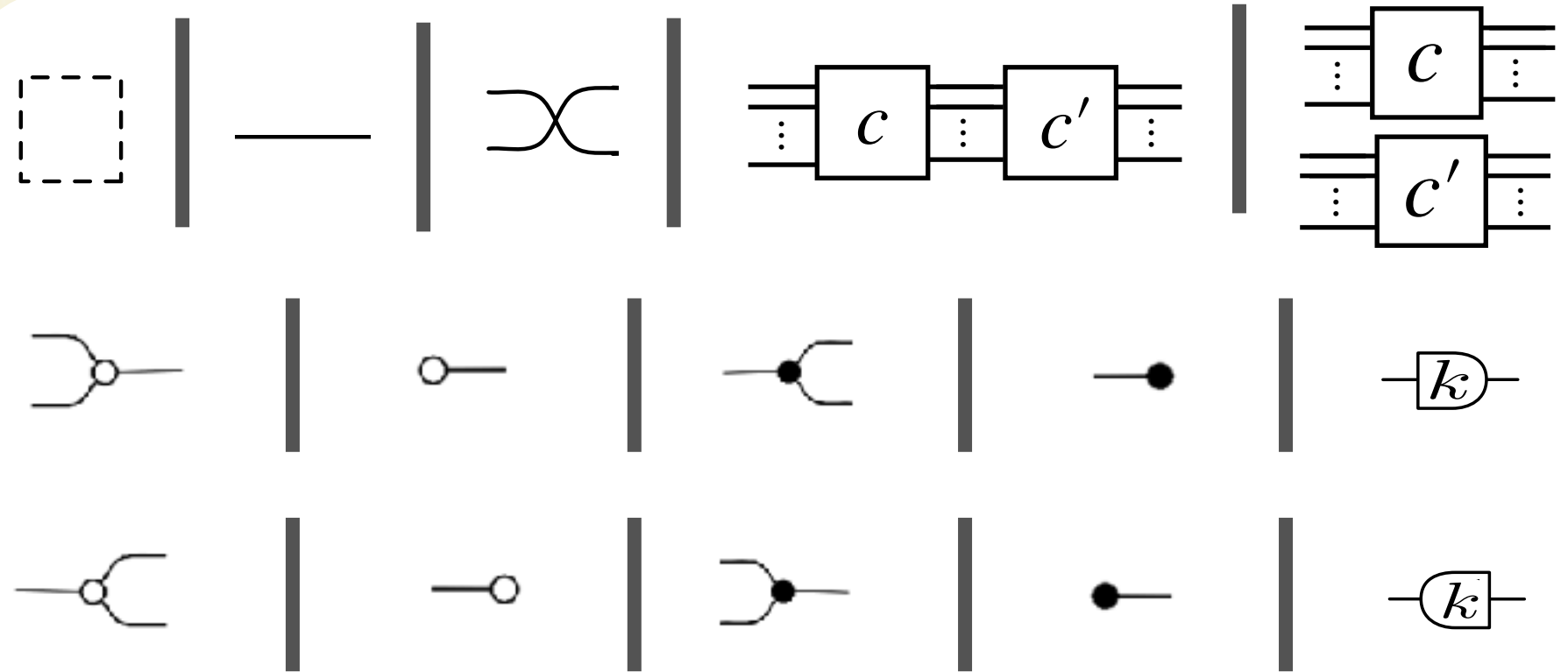
$\updownarrow$
 R_1

$\updownarrow$
 R_2

Example: Signal Flow Graphs



$k \in R$
In this case,
 $R = \mathbb{R}[x]$



R_1 ; R_2 = R

Signal Flow diagrams: semantics

The semantics of a signal flow diagram is a **relation over R-vectors**

$$\langle \text{Addition} \rangle_R := \{((\begin{smallmatrix} x \\ y \end{smallmatrix}), x + y) \mid x, y \in R\}$$

Addition

$$\langle \text{Zero} \rangle_R := \{(\bullet, 0)\}$$

Zero

$$\langle \text{Duplicate} \rangle_R := \{(x, (\begin{smallmatrix} x \\ x \end{smallmatrix})) \mid x \in R\}$$

Duplicate

$$\langle \text{Discard} \rangle_R := \{(x, \bullet) \mid x \in R\}$$

Discard

$$\langle \text{Multiply by } k \rangle_R := \{(x, k \cdot x) \mid x \in R\}$$

Multiply by k

$\vdots$

$$\langle \text{Synch} \rangle_R := \{(a, b) \mid \exists w. (a, w) \in \langle c \rangle_R, (w, b) \in \langle d \rangle_R\}$$

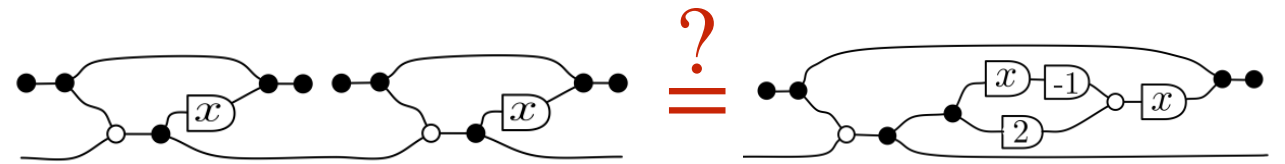
Synch

$$\langle \text{Parallel} \rangle_R := \{((\begin{smallmatrix} a_1 \\ a_2 \end{smallmatrix}), (\begin{smallmatrix} b_1 \\ b_2 \end{smallmatrix})) \mid (a_1, a_2) \in \langle c \rangle_R, (b_1, b_2) \in \langle d \rangle_R\}$$

Parallel

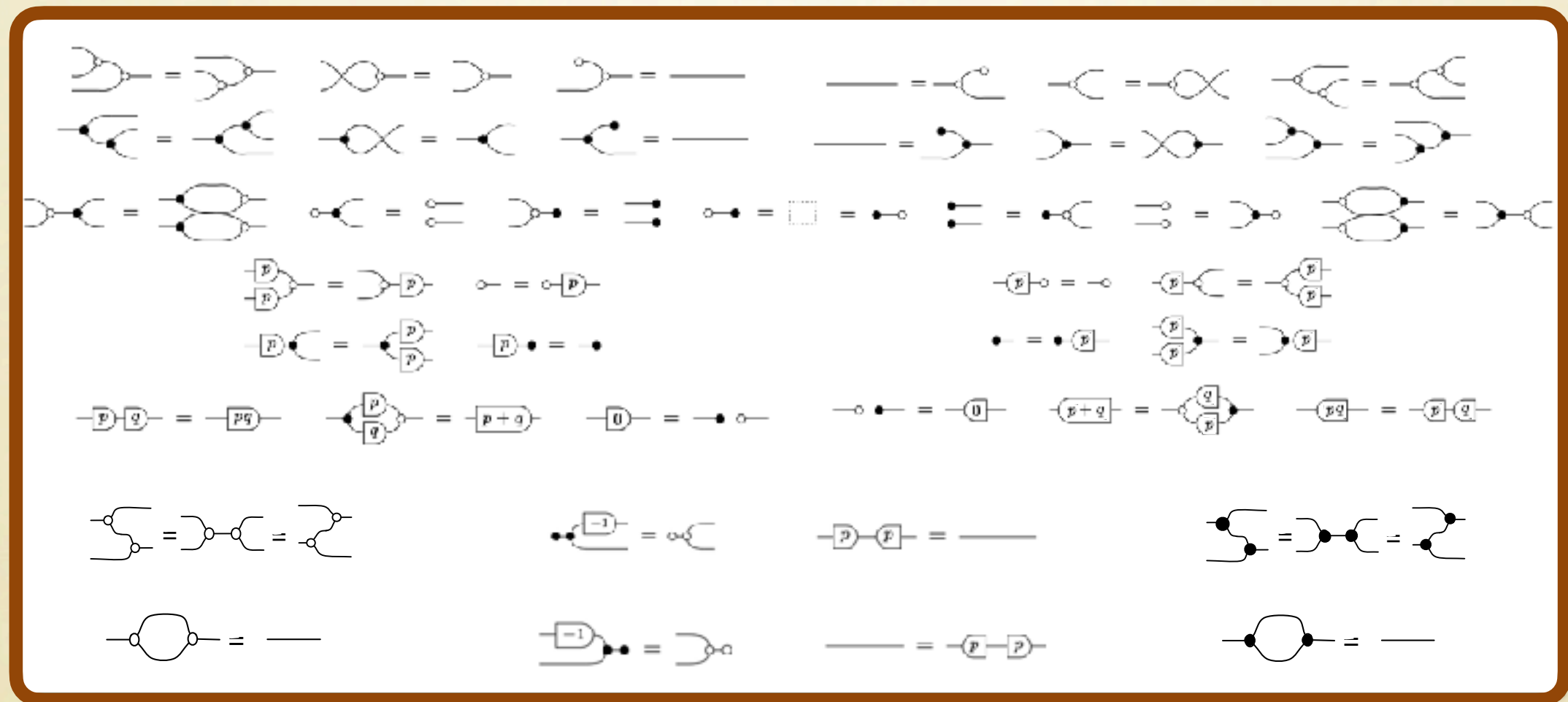
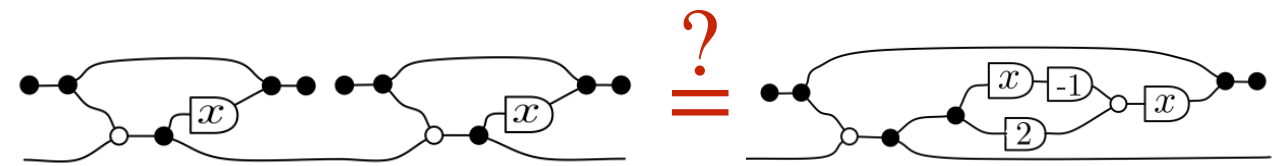
Complete Equational Theory IH

When two string diagrams represent the same system?



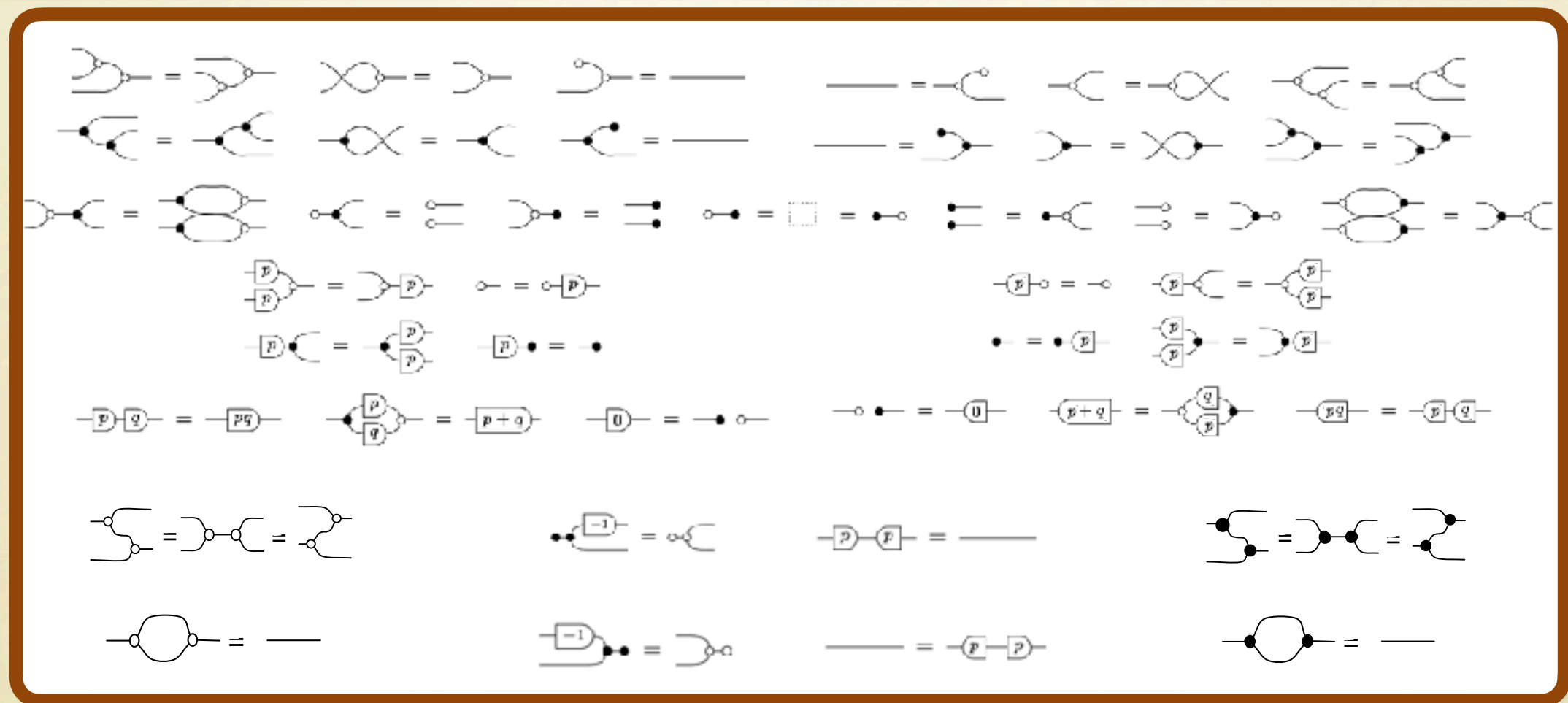
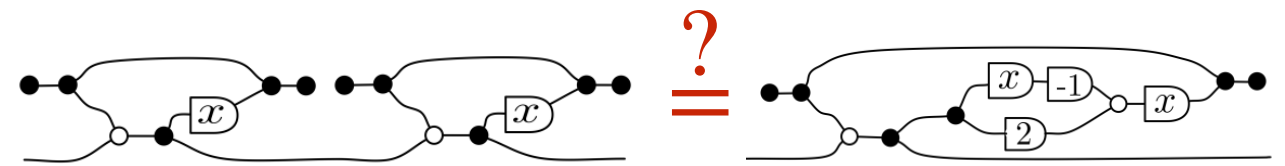
Complete Equational Theory IH

When two string diagrams represent the same system?



Complete Equational Theory IH

When two string diagrams represent the same system?

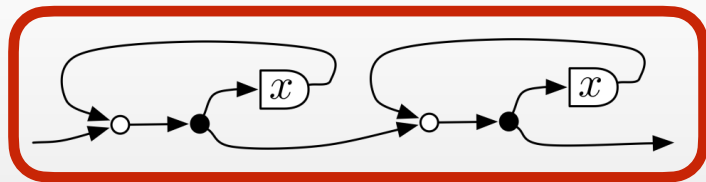


Completeness Theorem

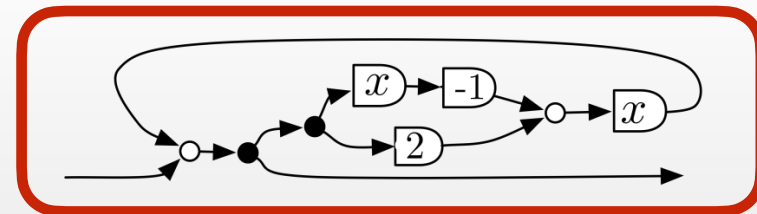
$$\langle \llbracket c \rrbracket \rangle_R = \langle \llbracket d \rrbracket \rangle_R \text{ if and only if } \llbracket c \rrbracket \stackrel{\text{IH}}{=} \llbracket d \rrbracket$$

Equational Reasoning at Work

Proof that two signal flow diagrams represent the same dynamical system:

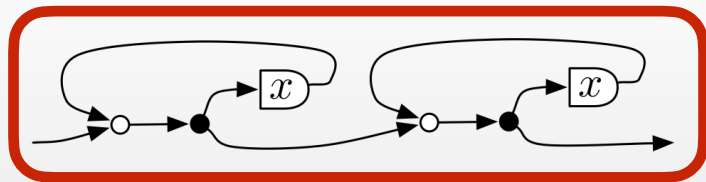


$\approx$?

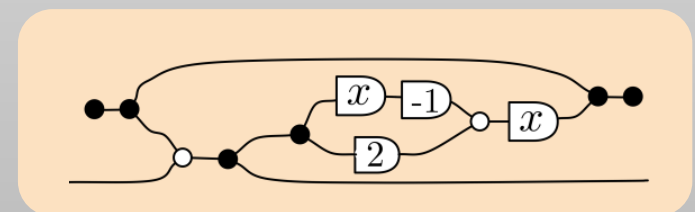
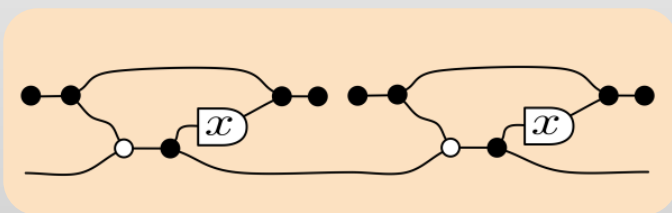
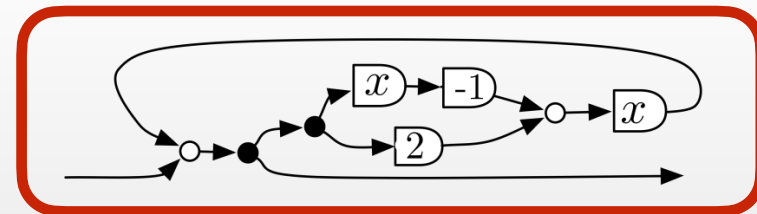


Equational Reasoning at Work

Proof that two signal flow diagrams represent the same dynamical system:

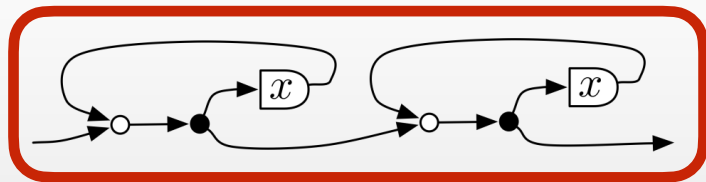


$\approx$?

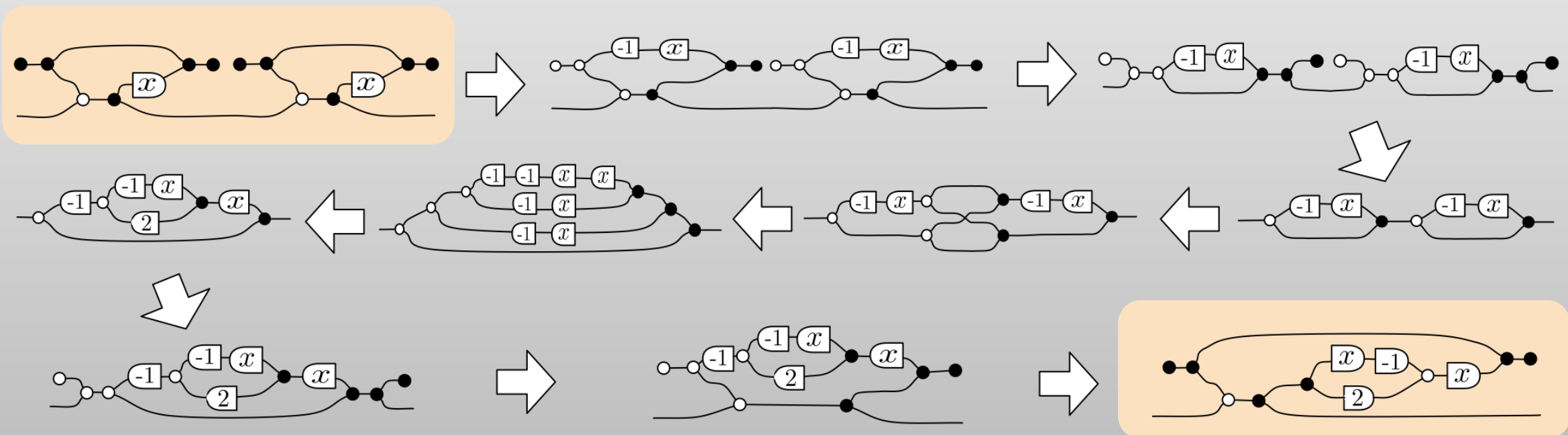
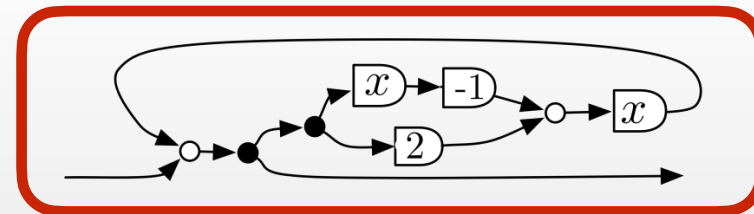


Equational Reasoning at Work

Proof that two signal flow diagrams represent the same dynamical system:



?

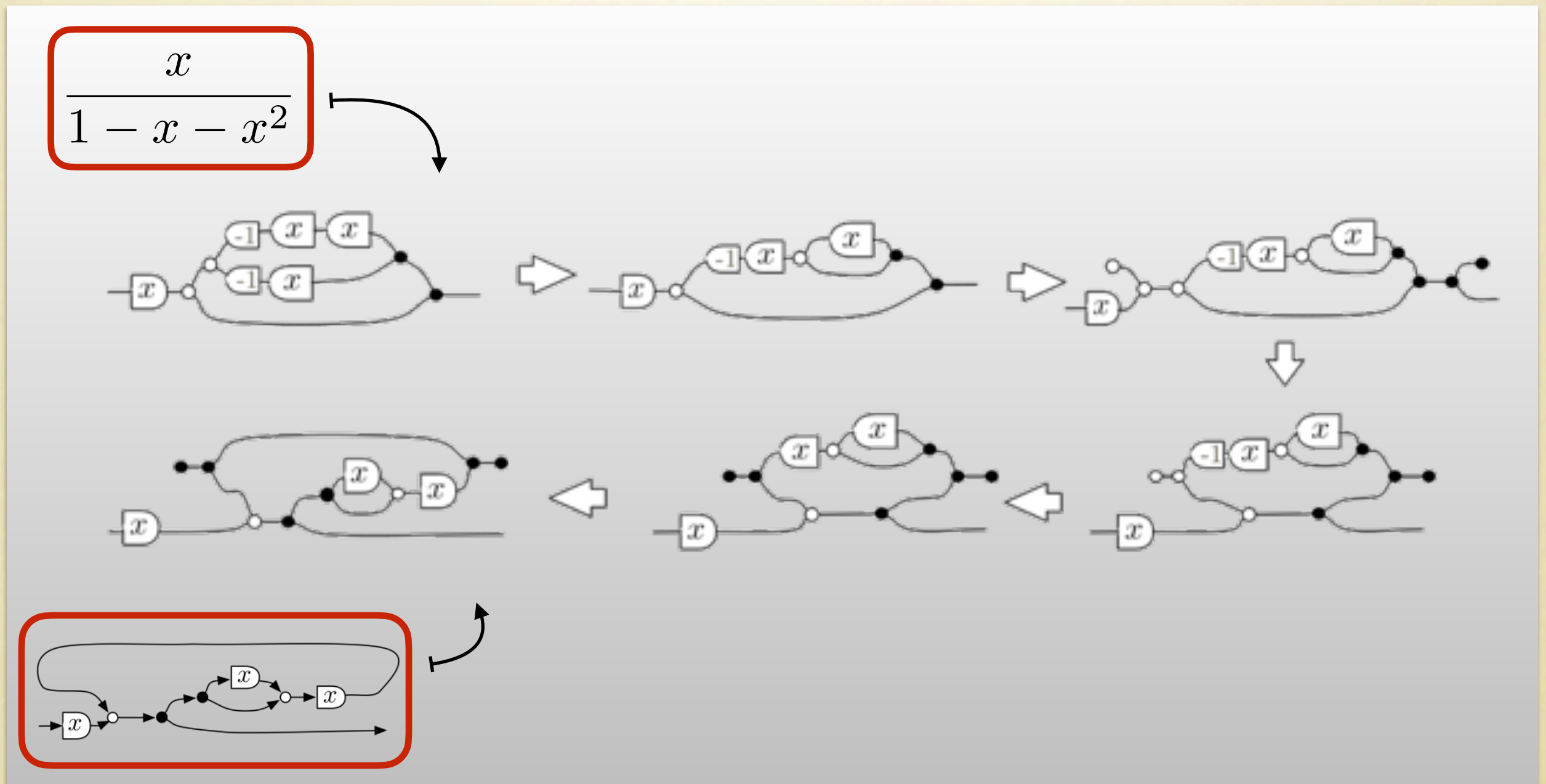


Equational Reasoning at Work

From a specification (a rational fraction) to its implementation as a linear dynamical system.

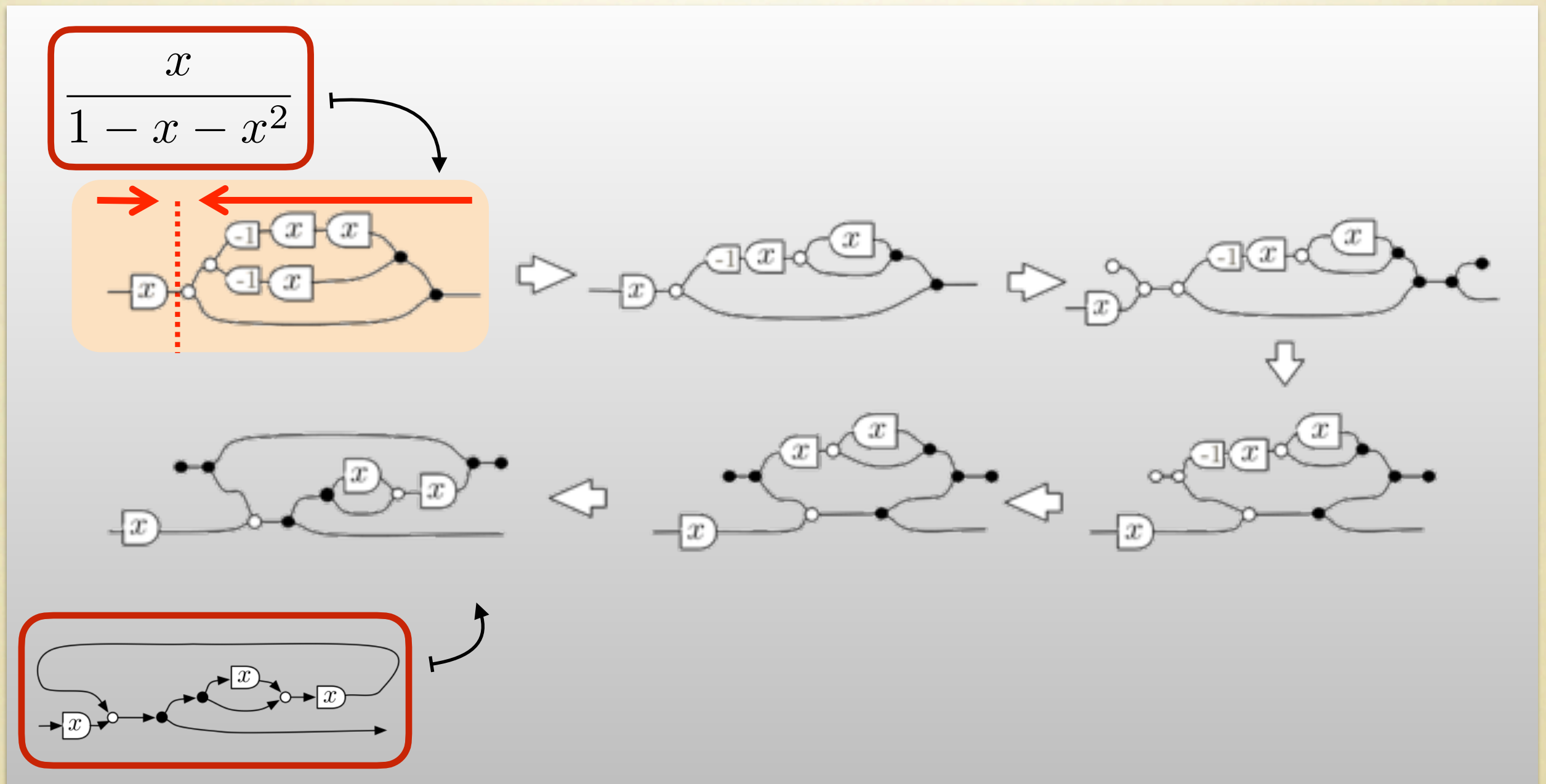
Equational Reasoning at Work

From a specification (a rational fraction) to its implementation as a linear dynamical system.



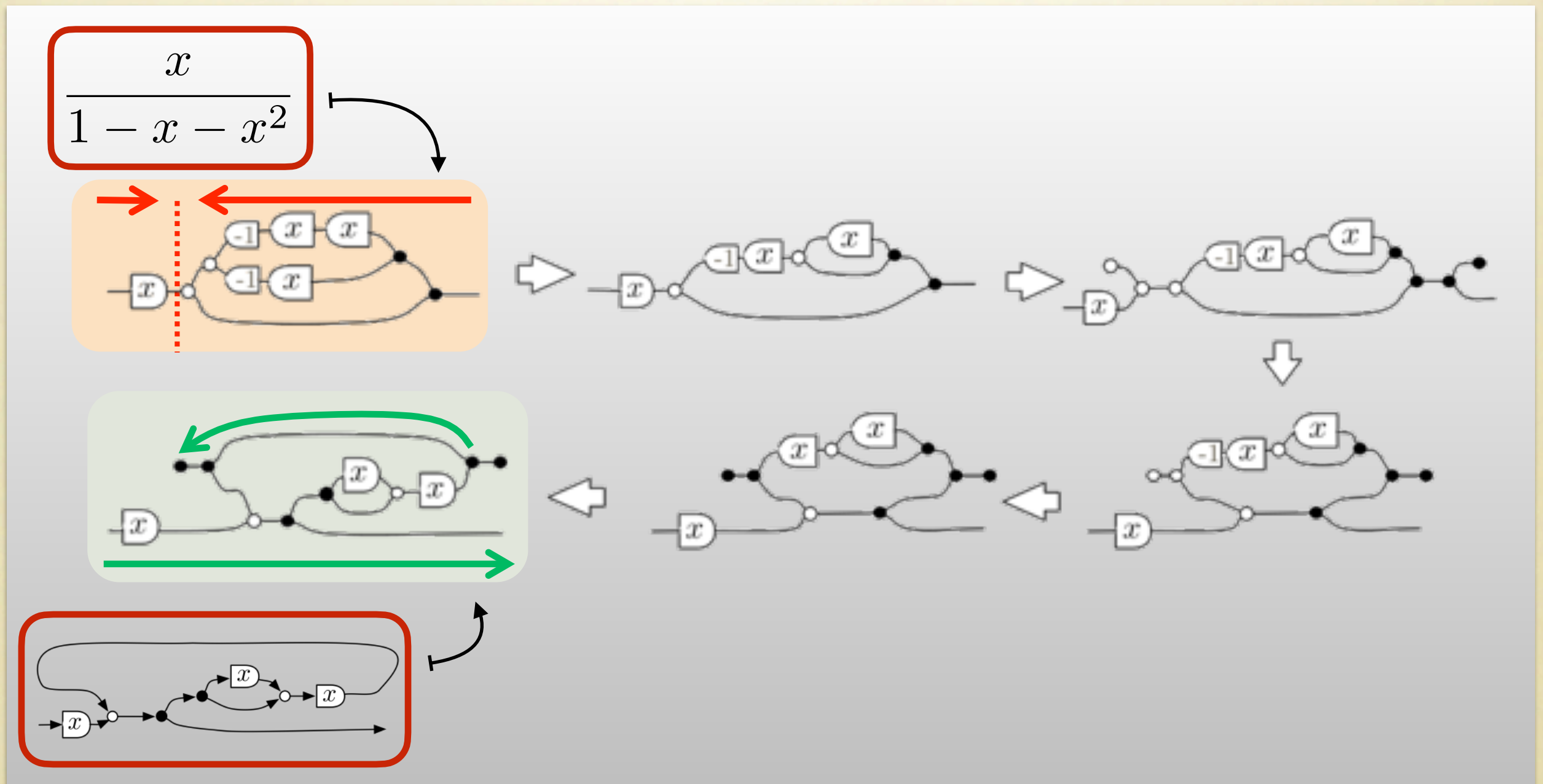
Equational Reasoning at Work

From a specification (a rational fraction) to its implementation as a linear dynamical system.



Equational Reasoning at Work

From a specification (a rational fraction) to its implementation as a linear dynamical system.



...and more

- Structural Operational Semantics
- Contextual Equivalence
- Full Abstraction
- Controllability (Control theory)
- Behaviour Refinement

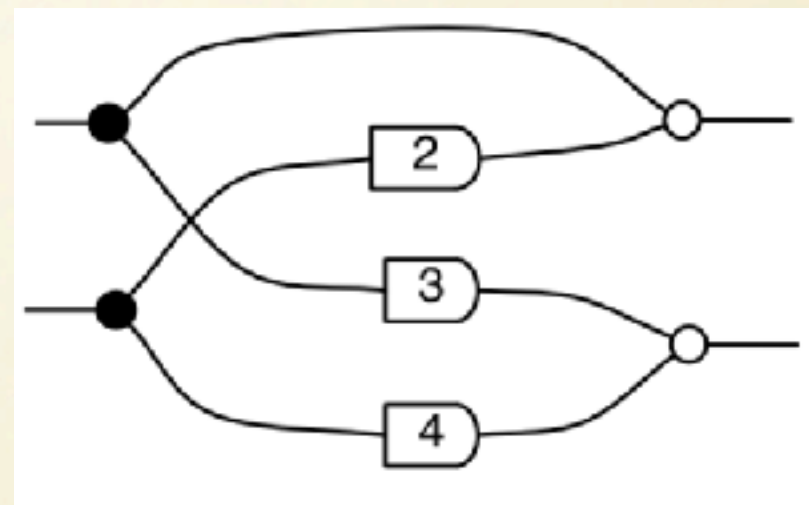
Part II

A Gallery of String Diagram Applications

Example I: Linear Algebra

Picturing matrices with coefficients in R

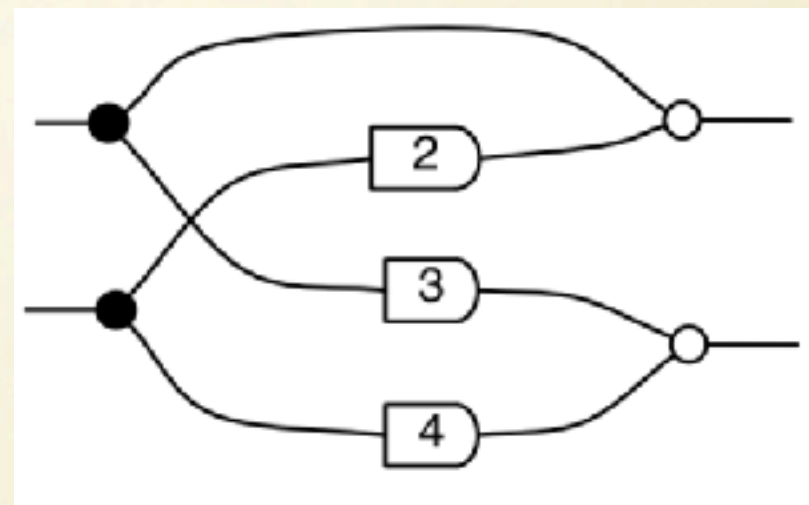
$$\begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix}$$



Example I: Linear Algebra

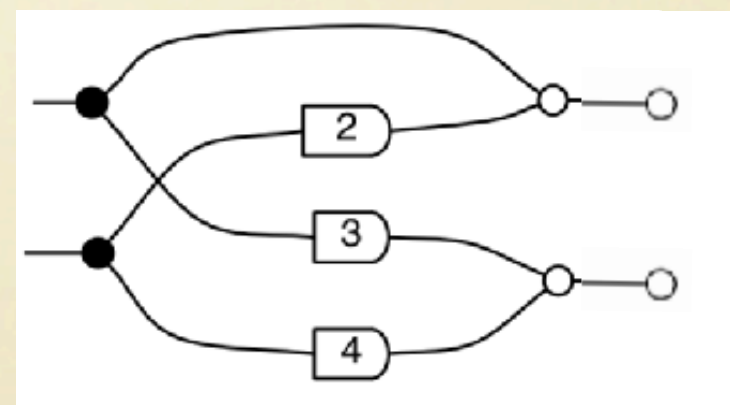
Picturing matrices with coefficients in R

$$\begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix}$$



Picturing R -linear subspaces (if R is a field)

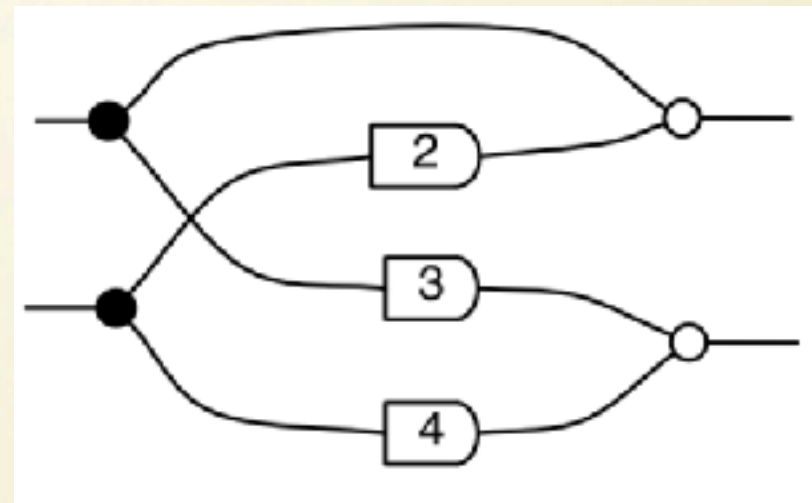
Every subspace is the kernel of some matrix.



Example I: Linear Algebra

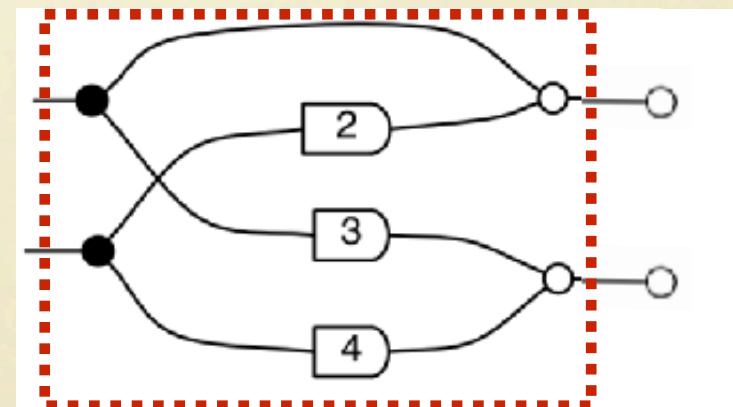
Picturing matrices with coefficients in R

$$\begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix}$$



Picturing R -linear subspaces (if R is a field)

Every subspace is the kernel of some matrix.

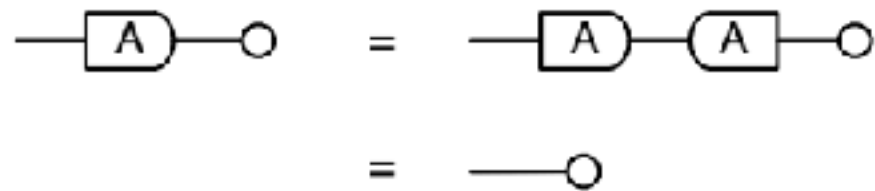


Example I: Linear Algebra

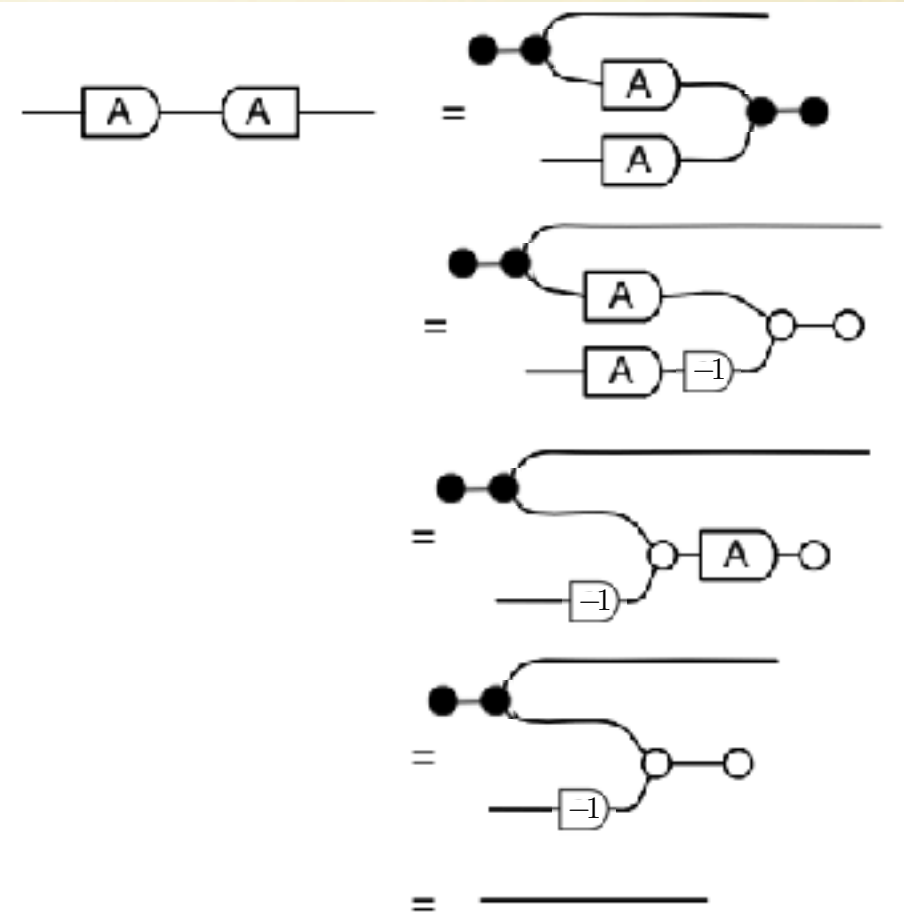
Proposition: a matrix A is injective iff its kernel is 0.

Proof

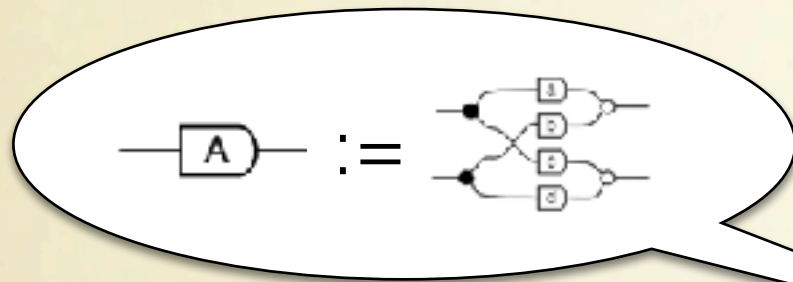
$\Rightarrow$



$\Leftarrow$

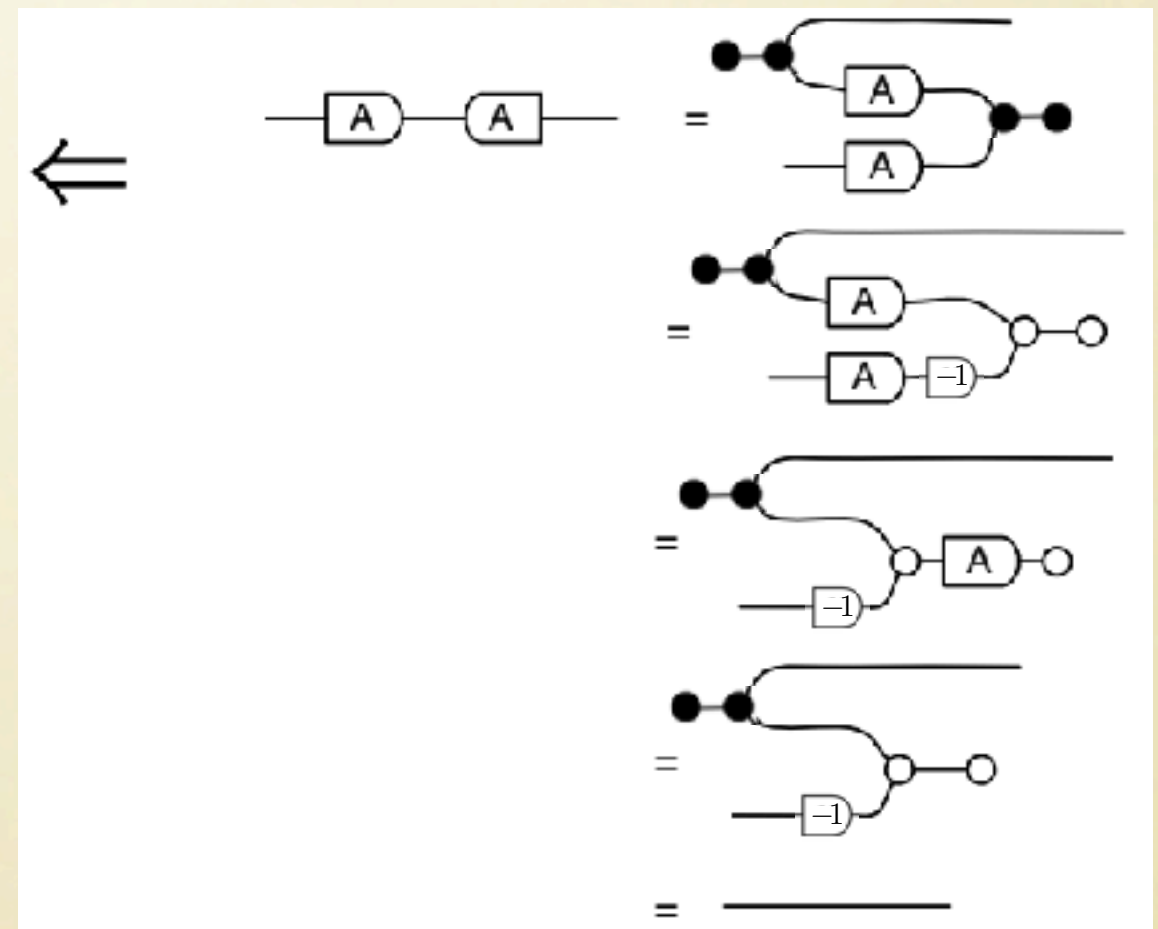
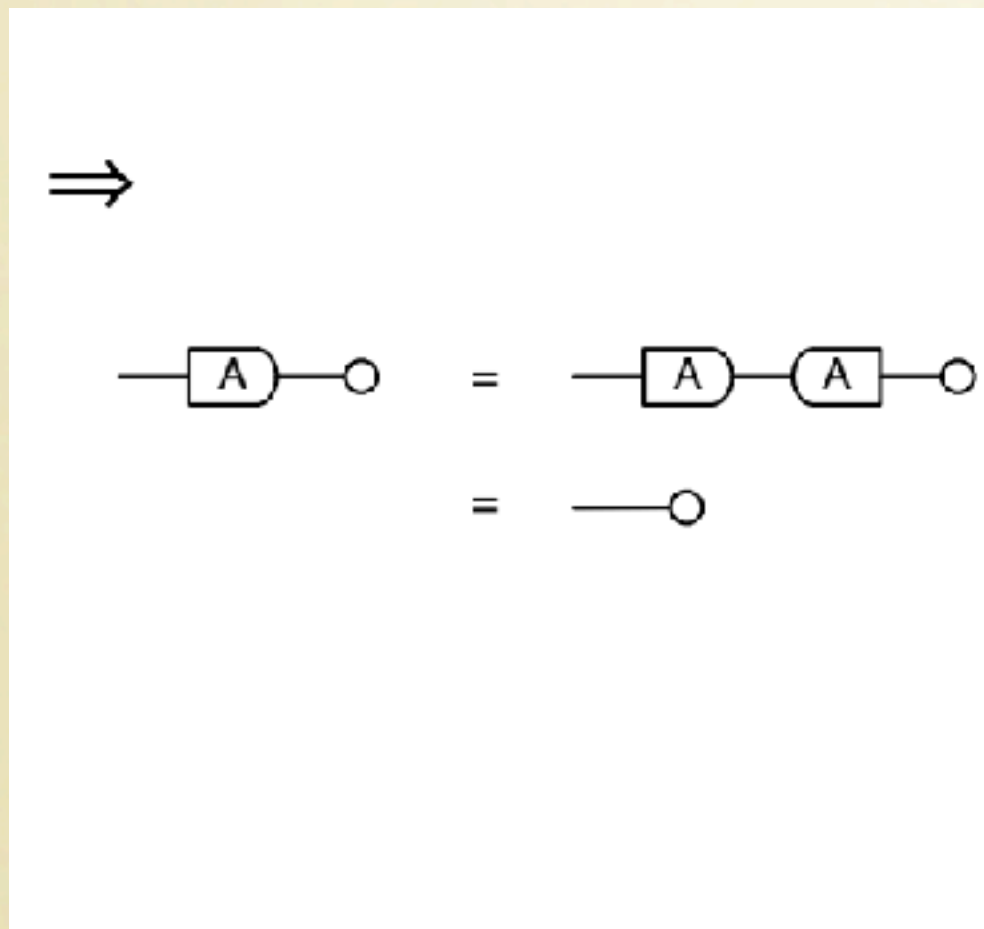


Example I: Linear Algebra

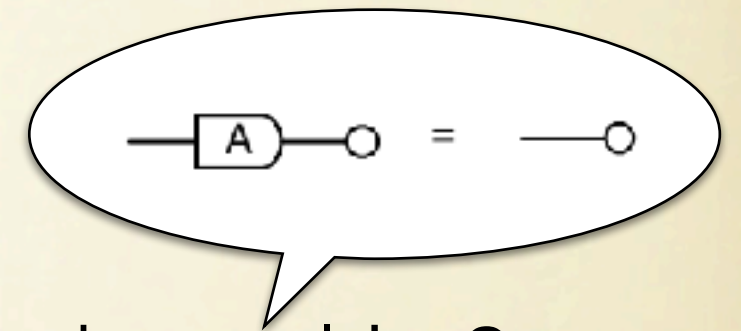
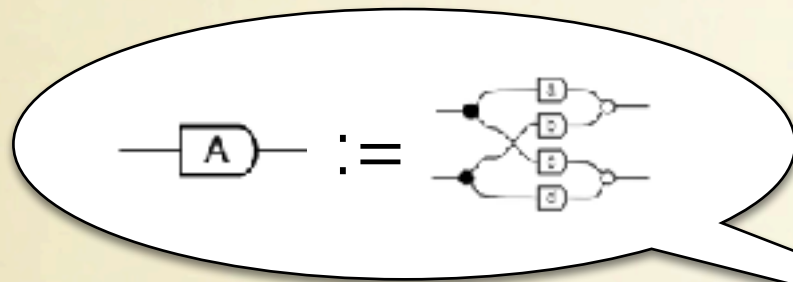


Proposition: a matrix A is injective iff its kernel is 0.

Proof

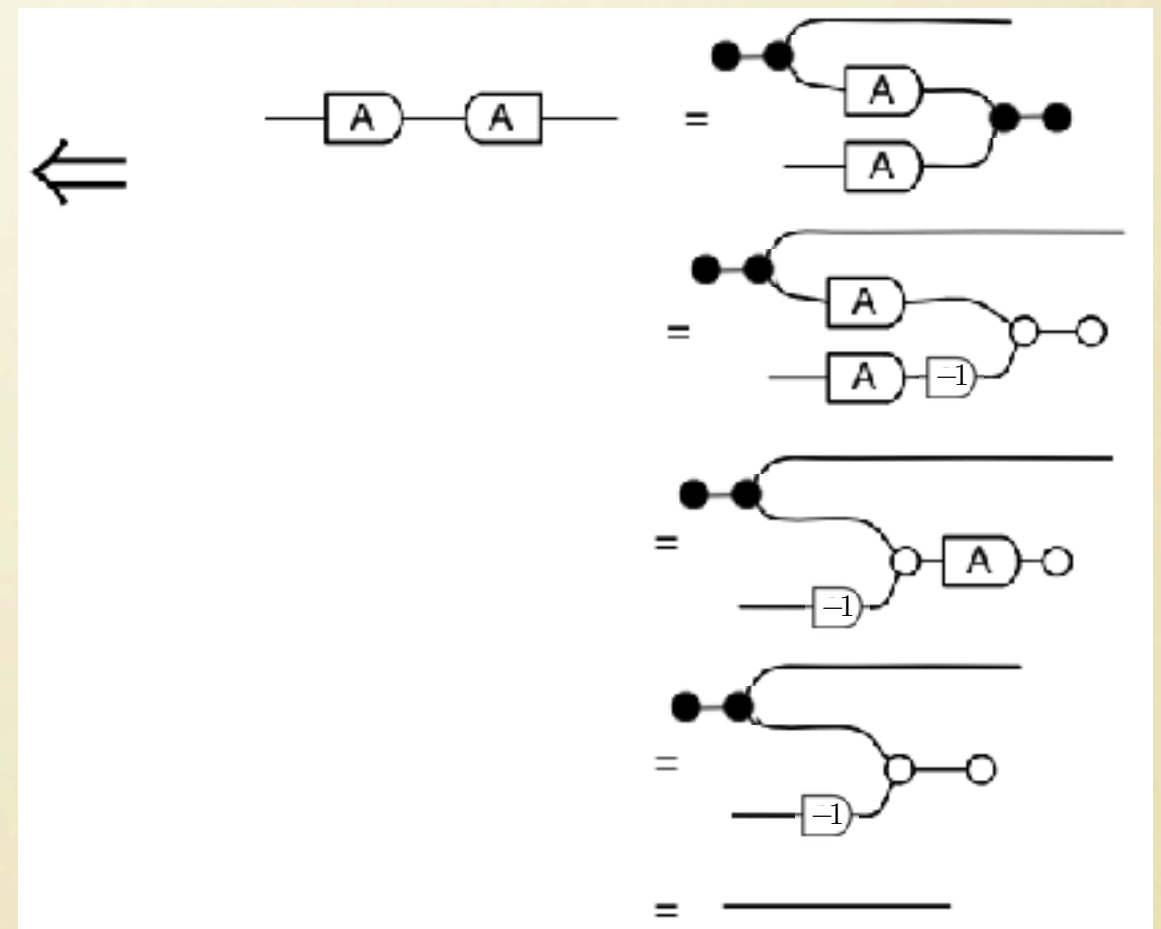
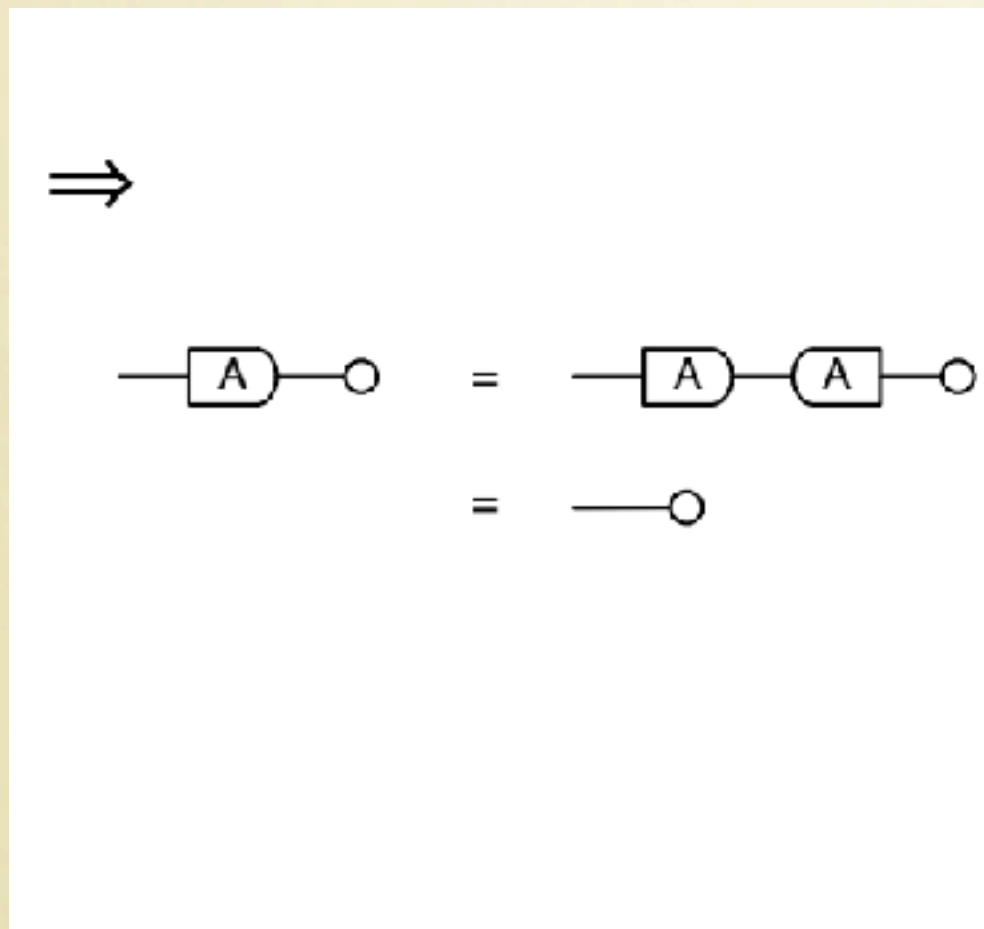


Example I: Linear Algebra

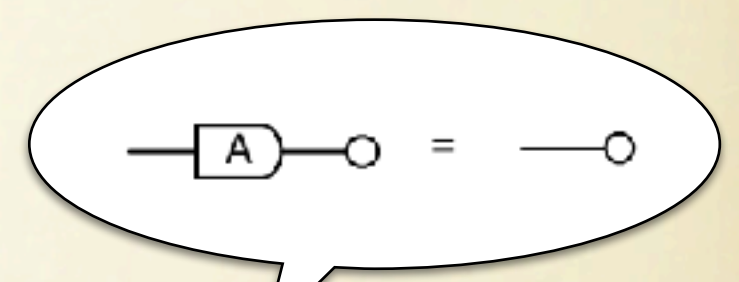
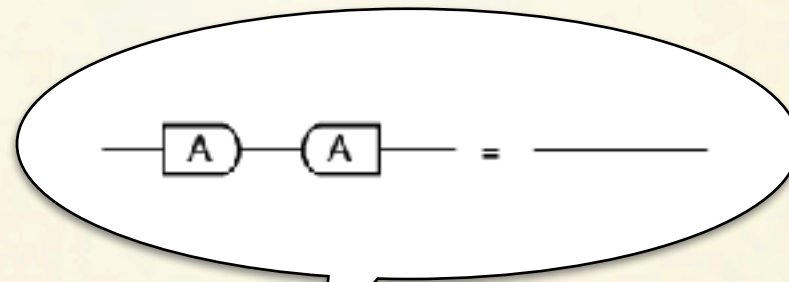
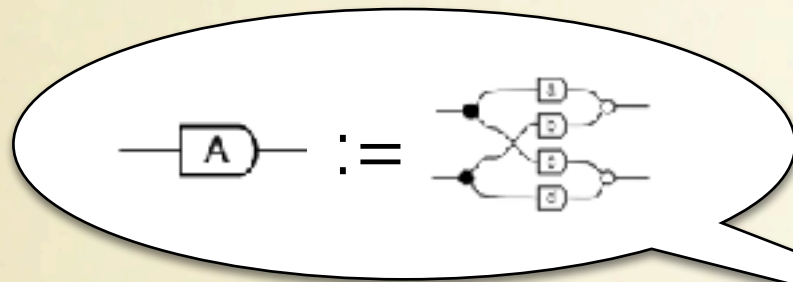


Proposition: a matrix A is injective iff its kernel is 0.

Proof

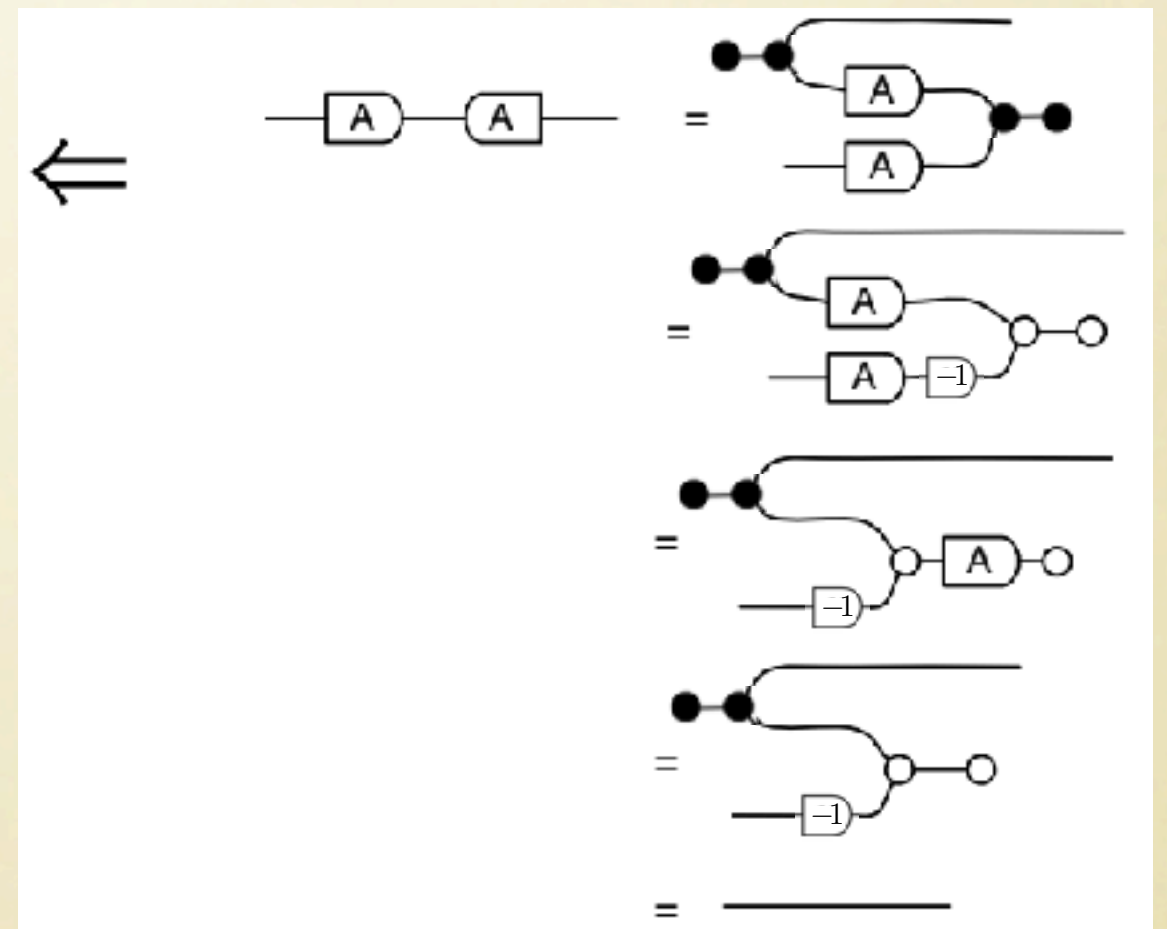
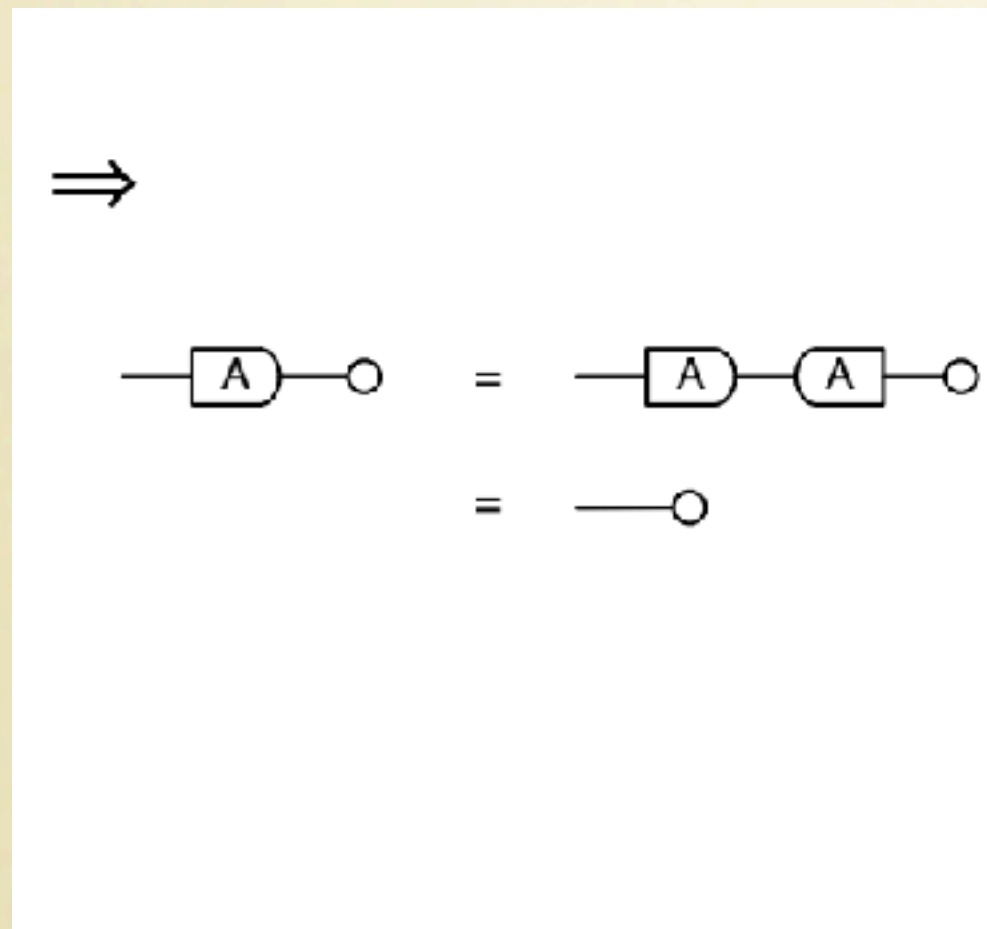


Example I: Linear Algebra

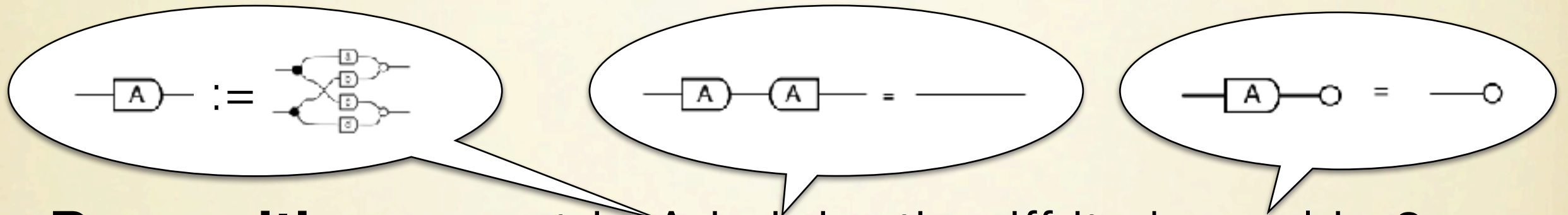


Proposition: a matrix A is injective iff its kernel is 0.

Proof

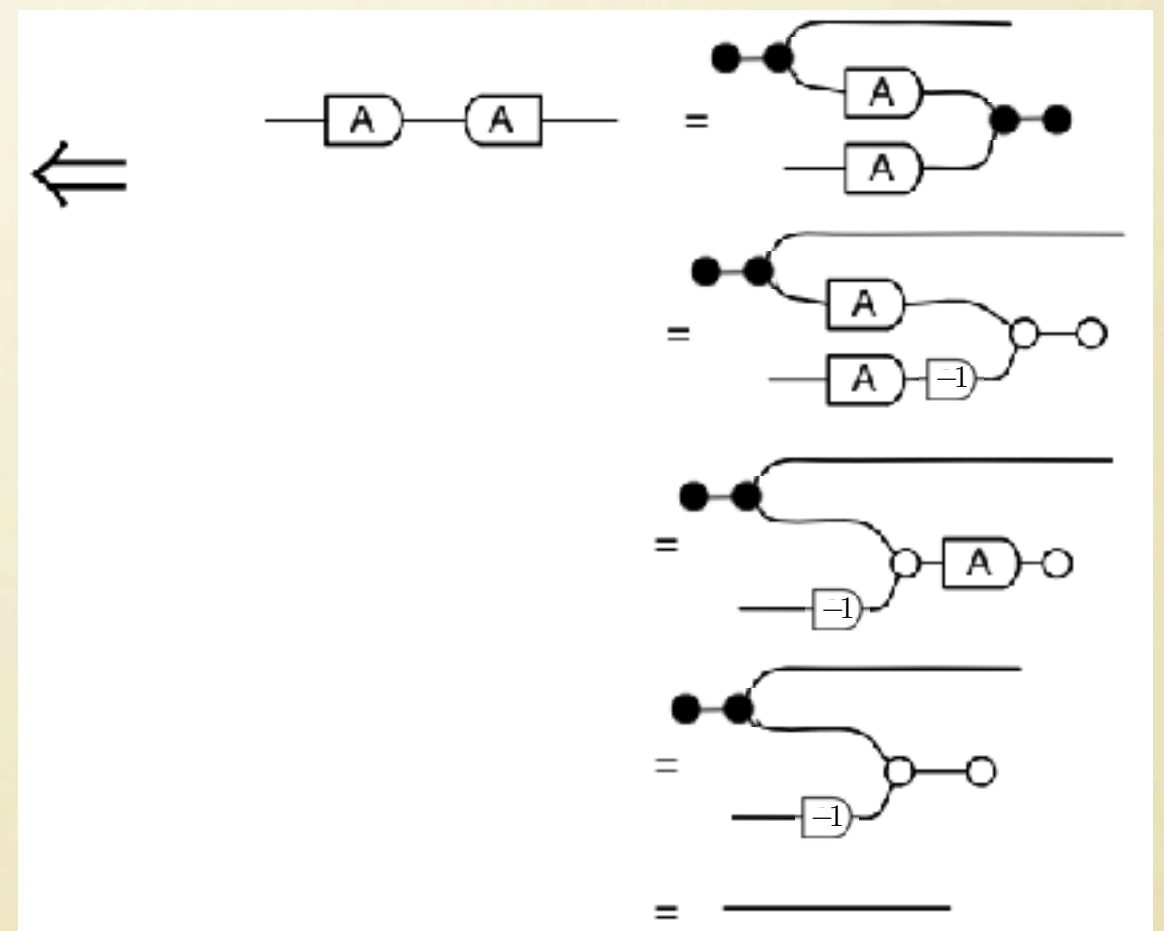
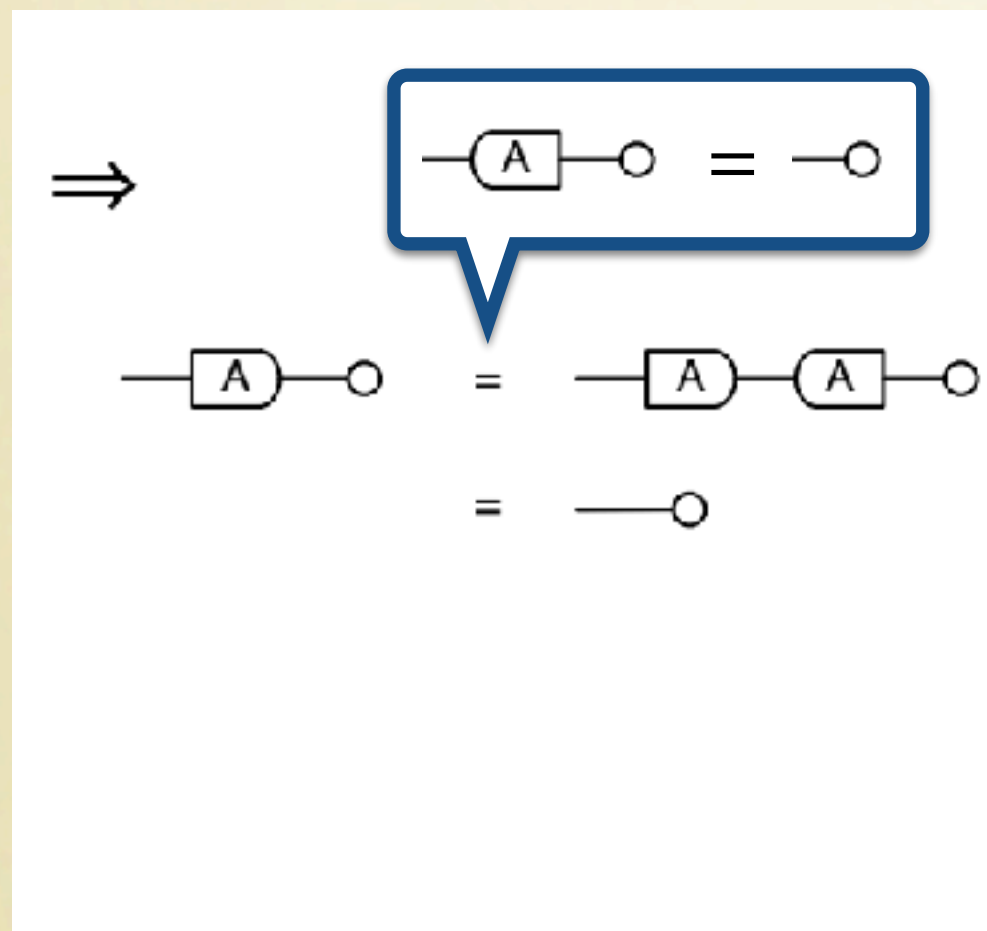


Example I: Linear Algebra

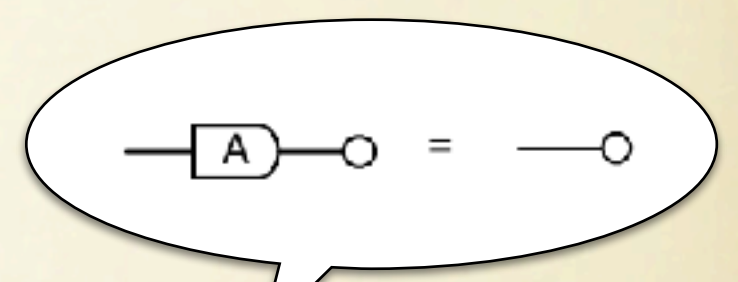
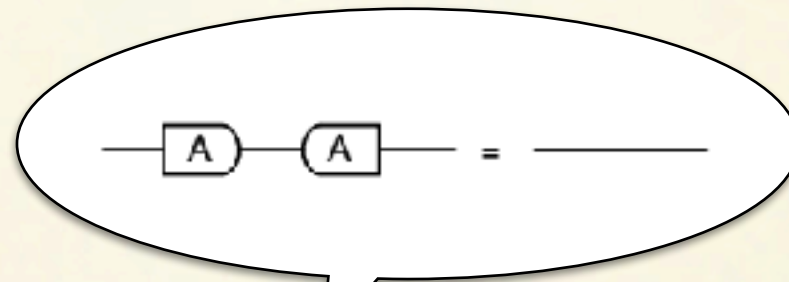
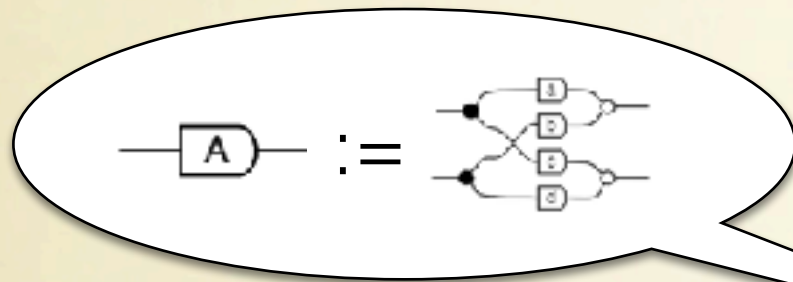


Proposition: a matrix A is injective iff its kernel is 0.

Proof

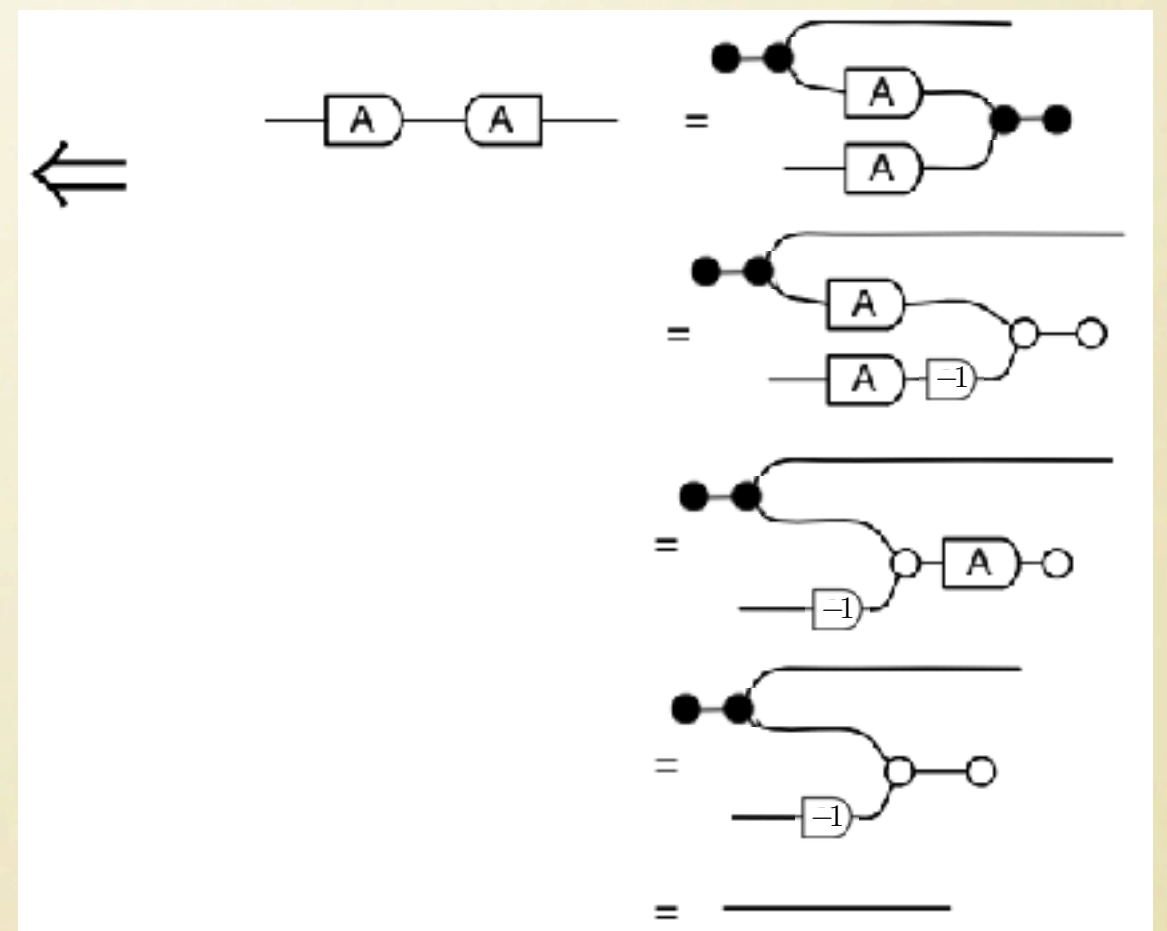
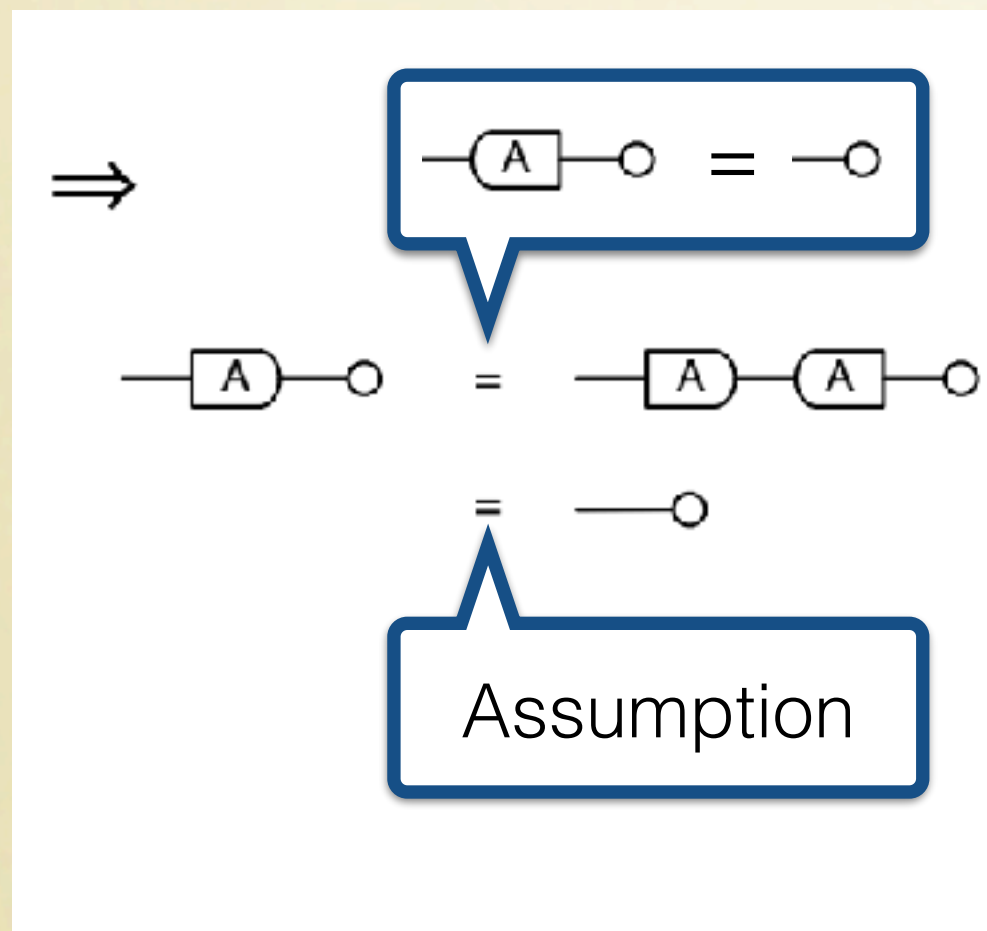


Example I: Linear Algebra

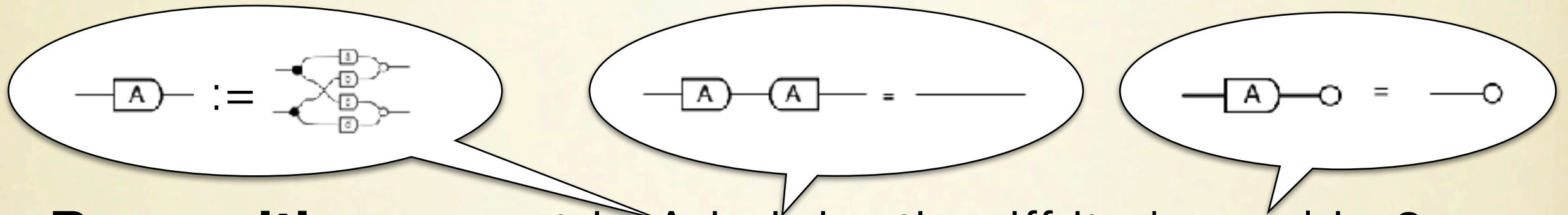


Proposition: a matrix A is injective iff its kernel is 0.

Proof

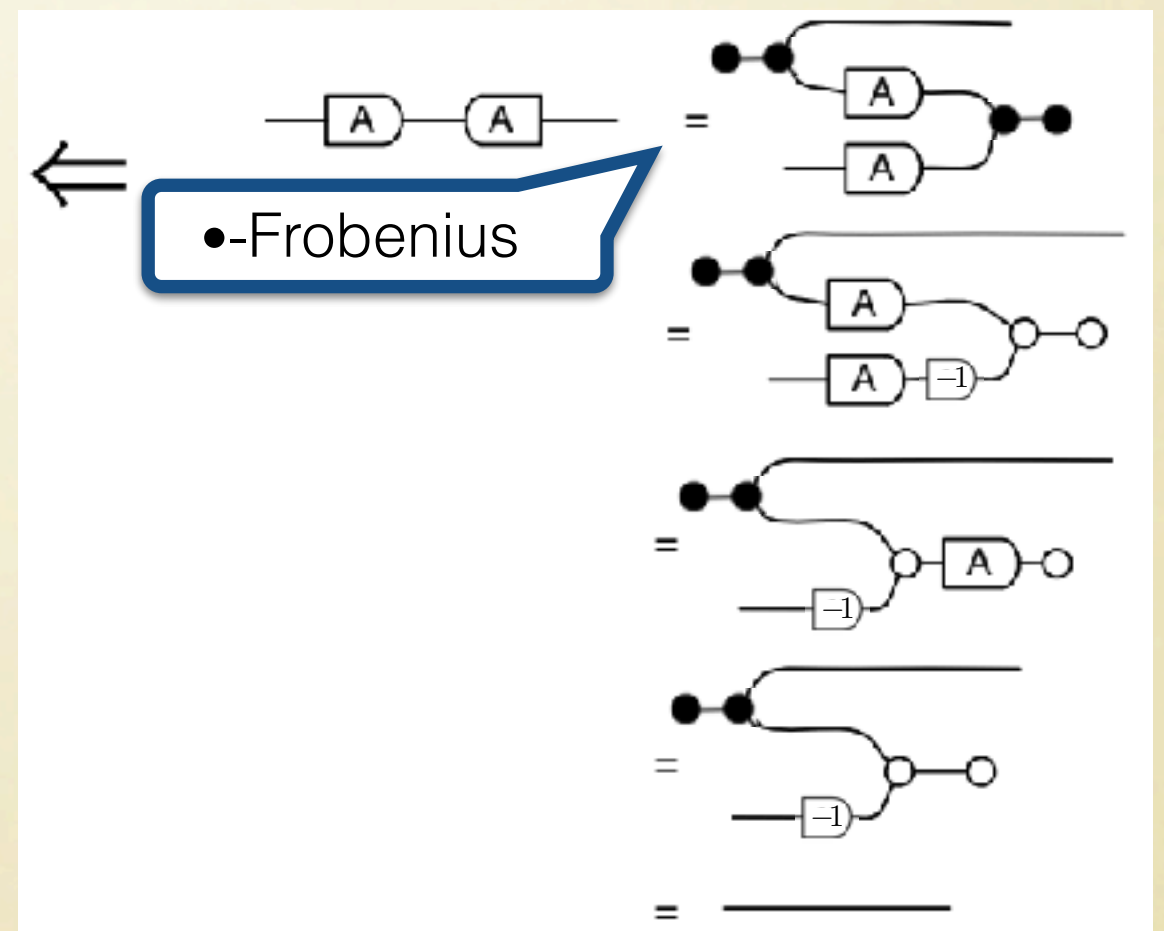
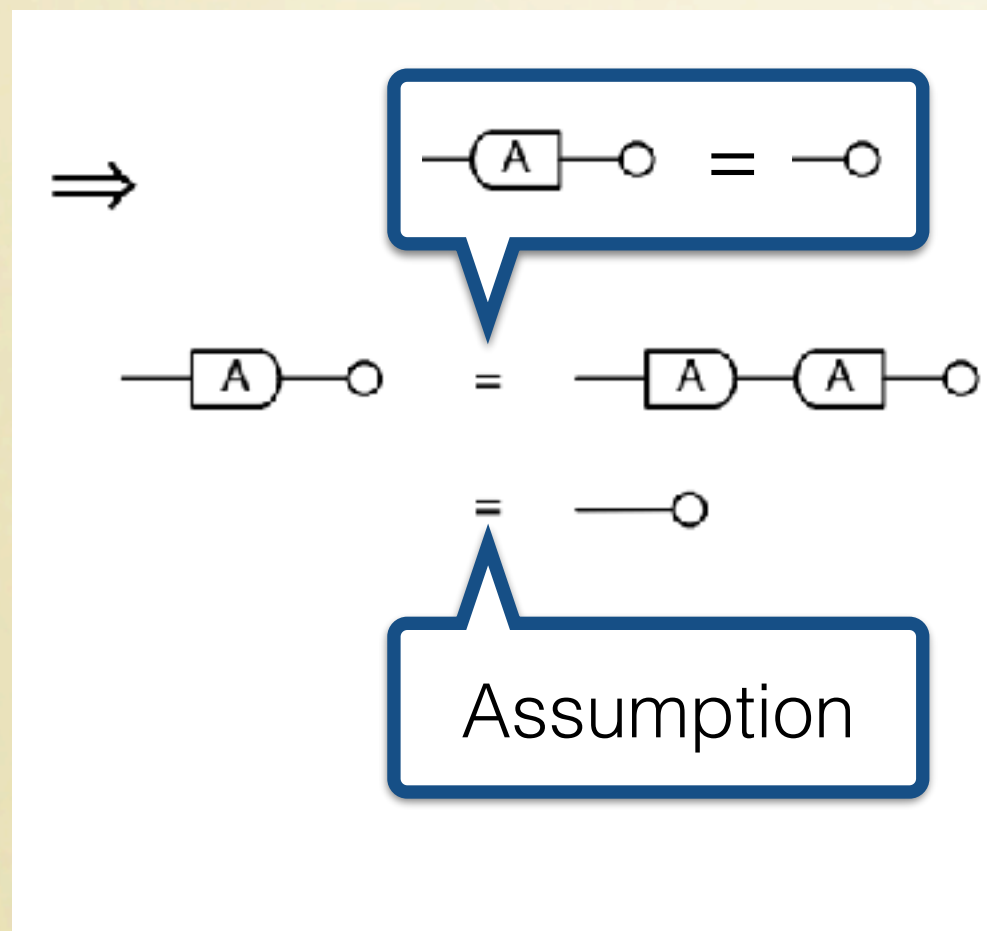


Example I: Linear Algebra

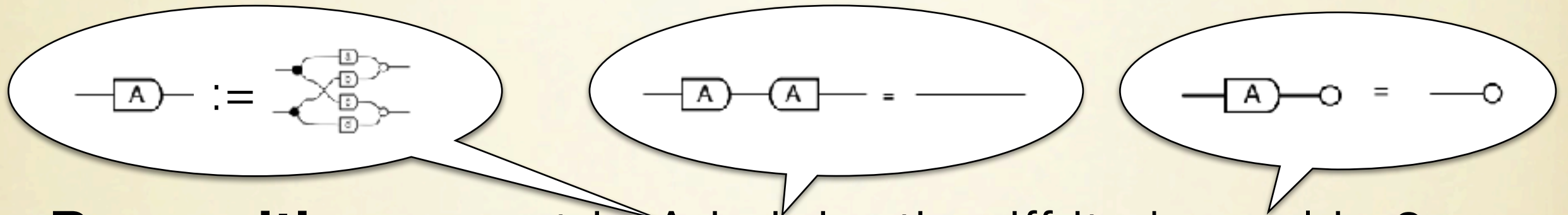


Proposition: a matrix A is injective iff its kernel is 0.

Proof

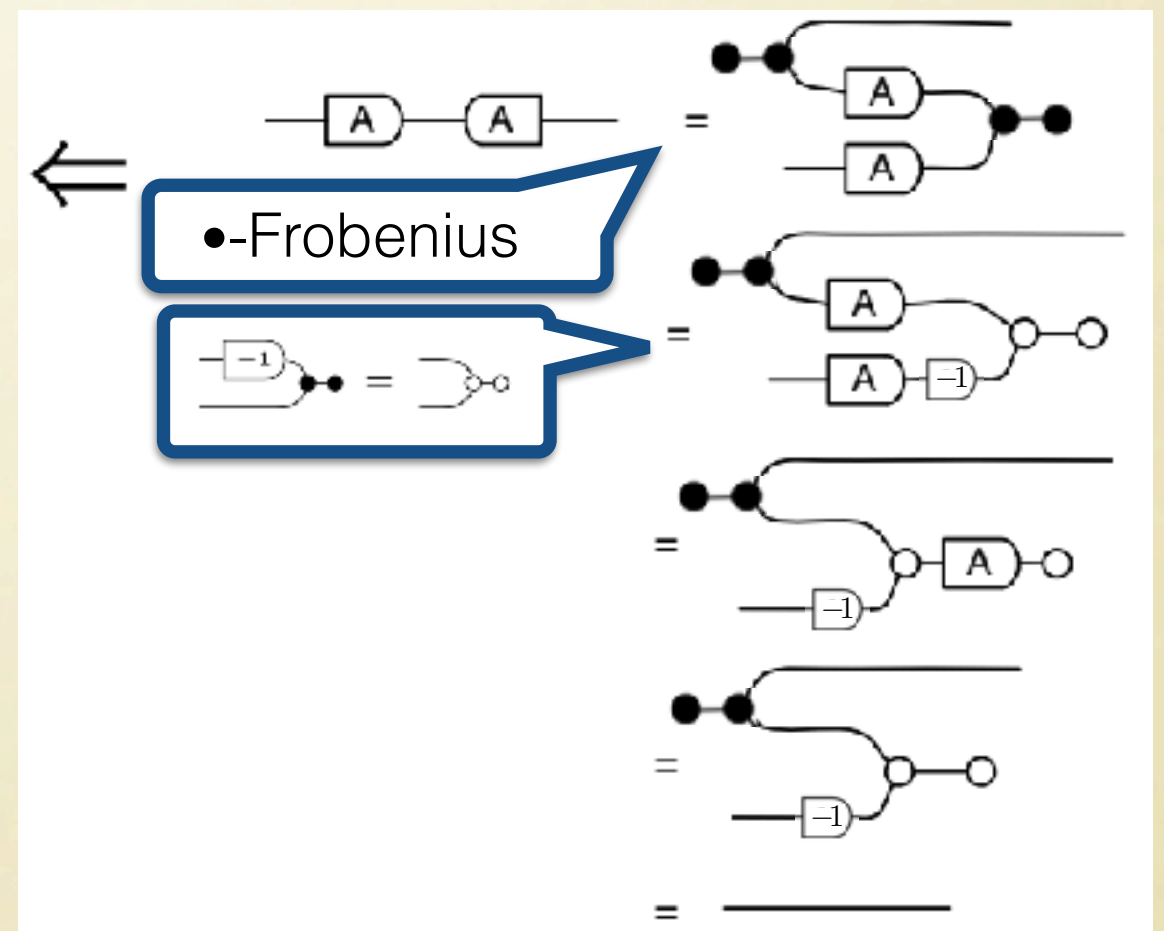
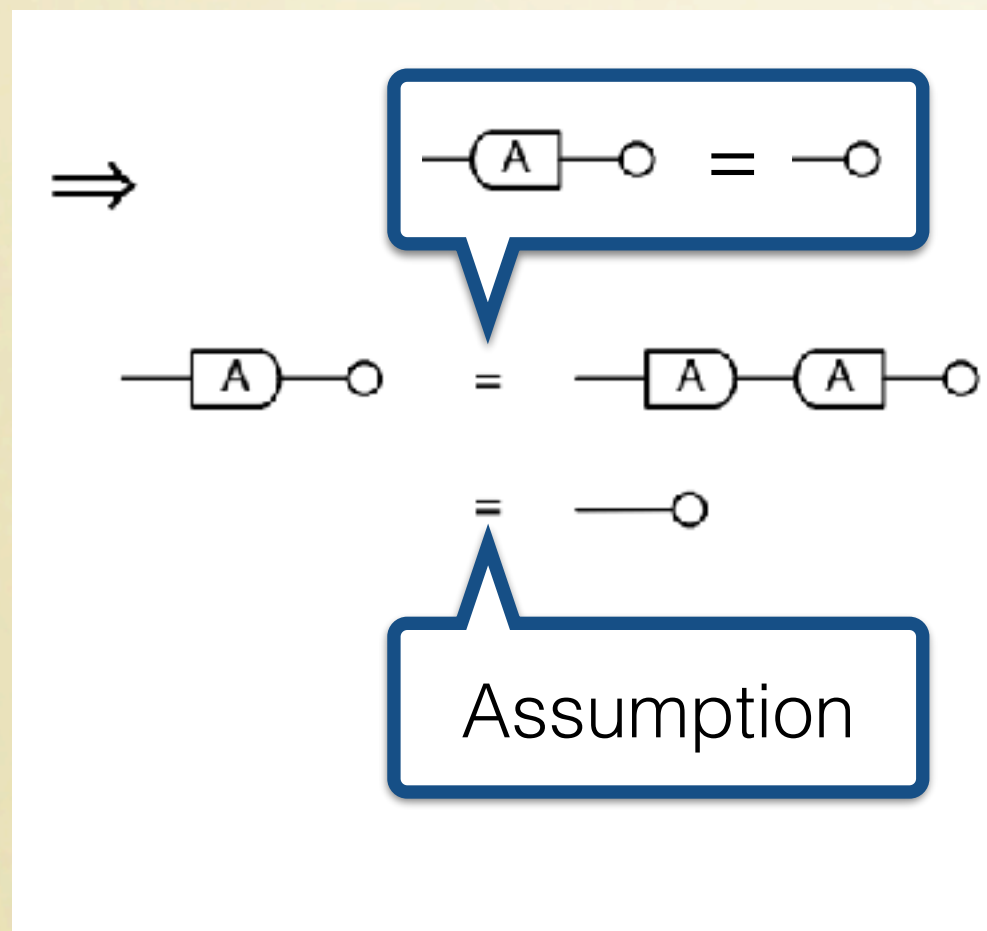


Example I: Linear Algebra

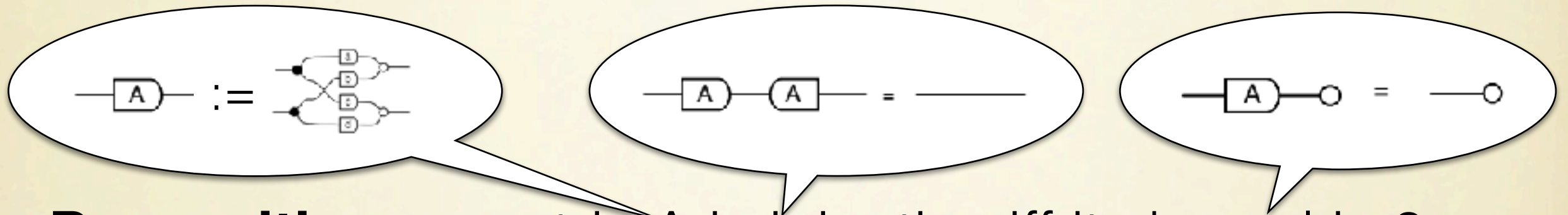


Proposition: a matrix A is injective iff its kernel is 0.

Proof

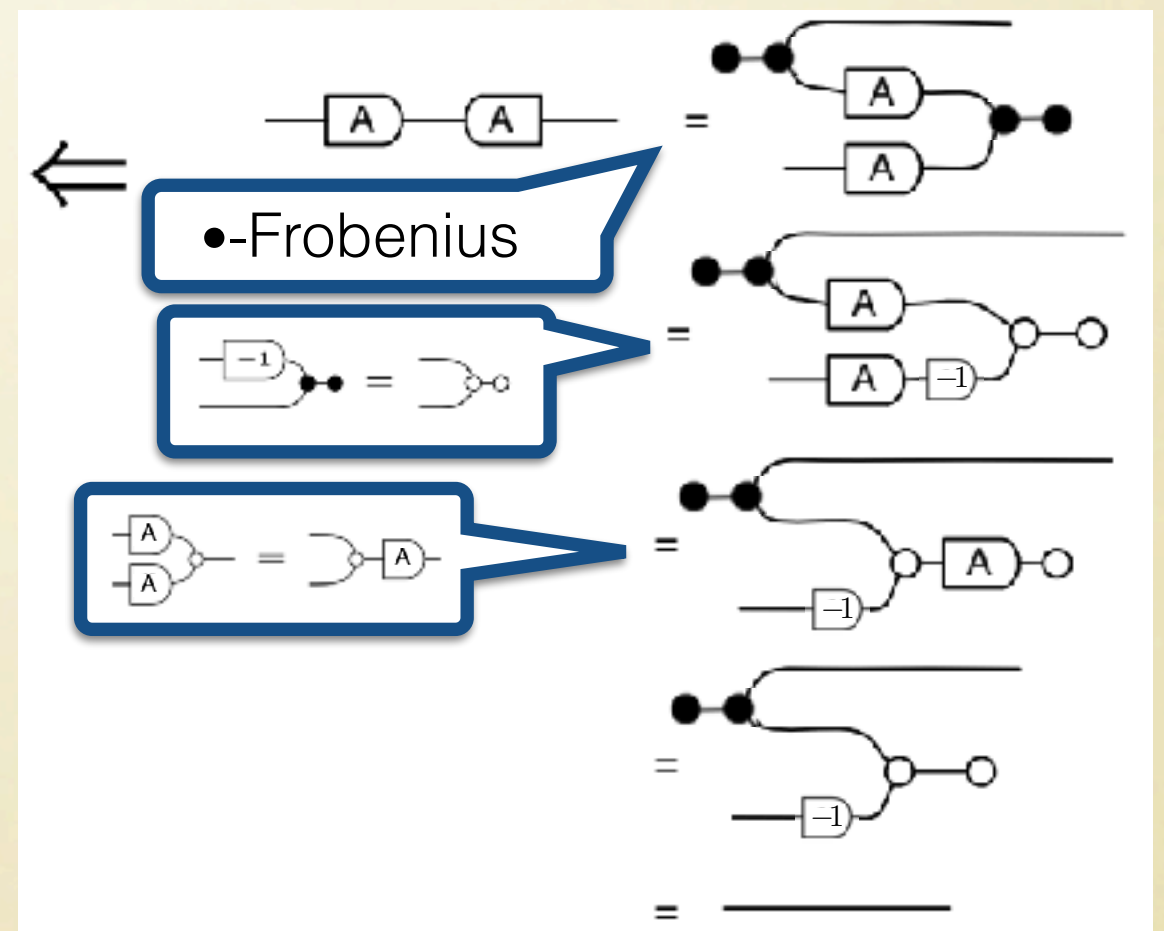
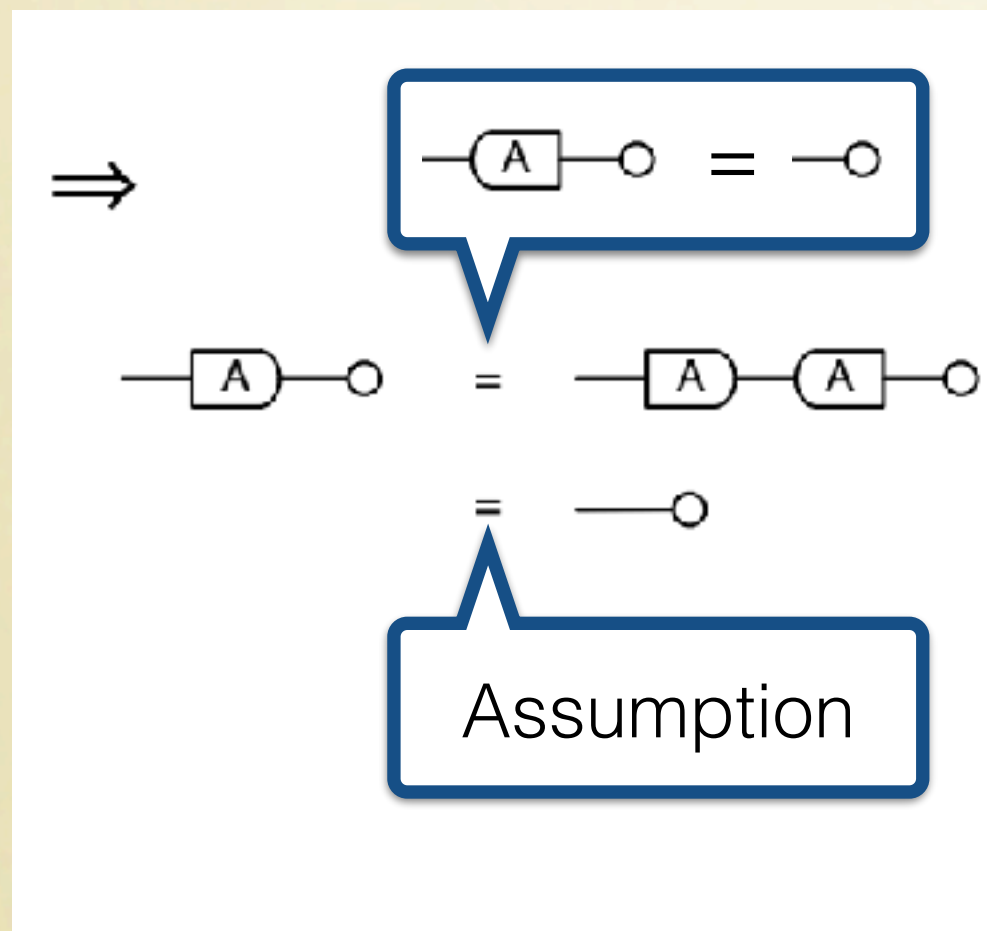


Example I: Linear Algebra

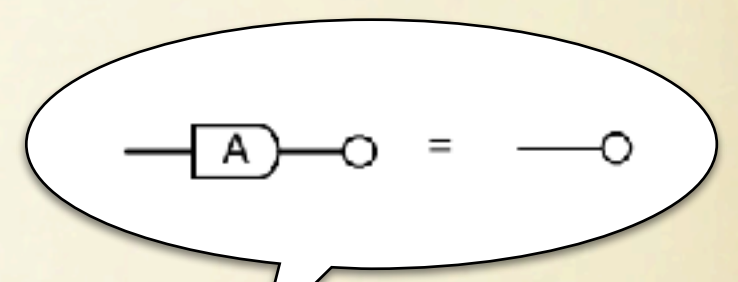
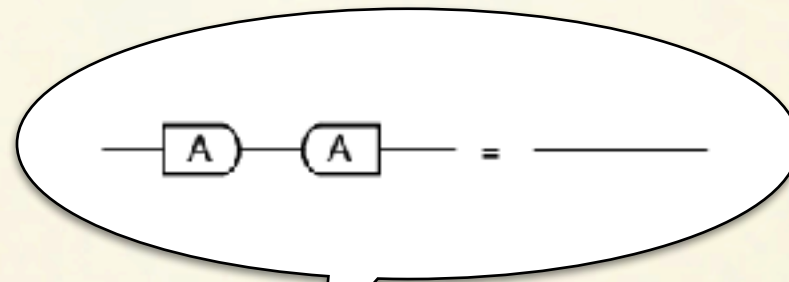
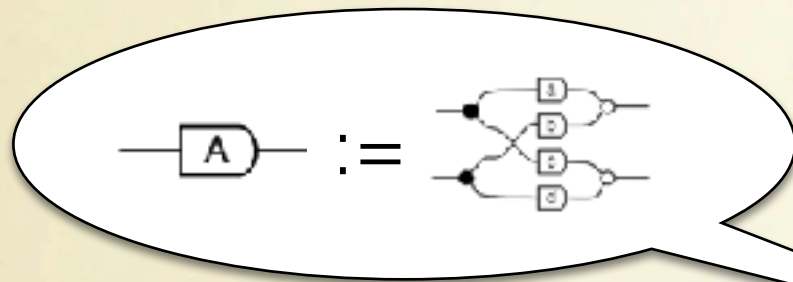


Proposition: a matrix A is injective iff its kernel is 0.

Proof

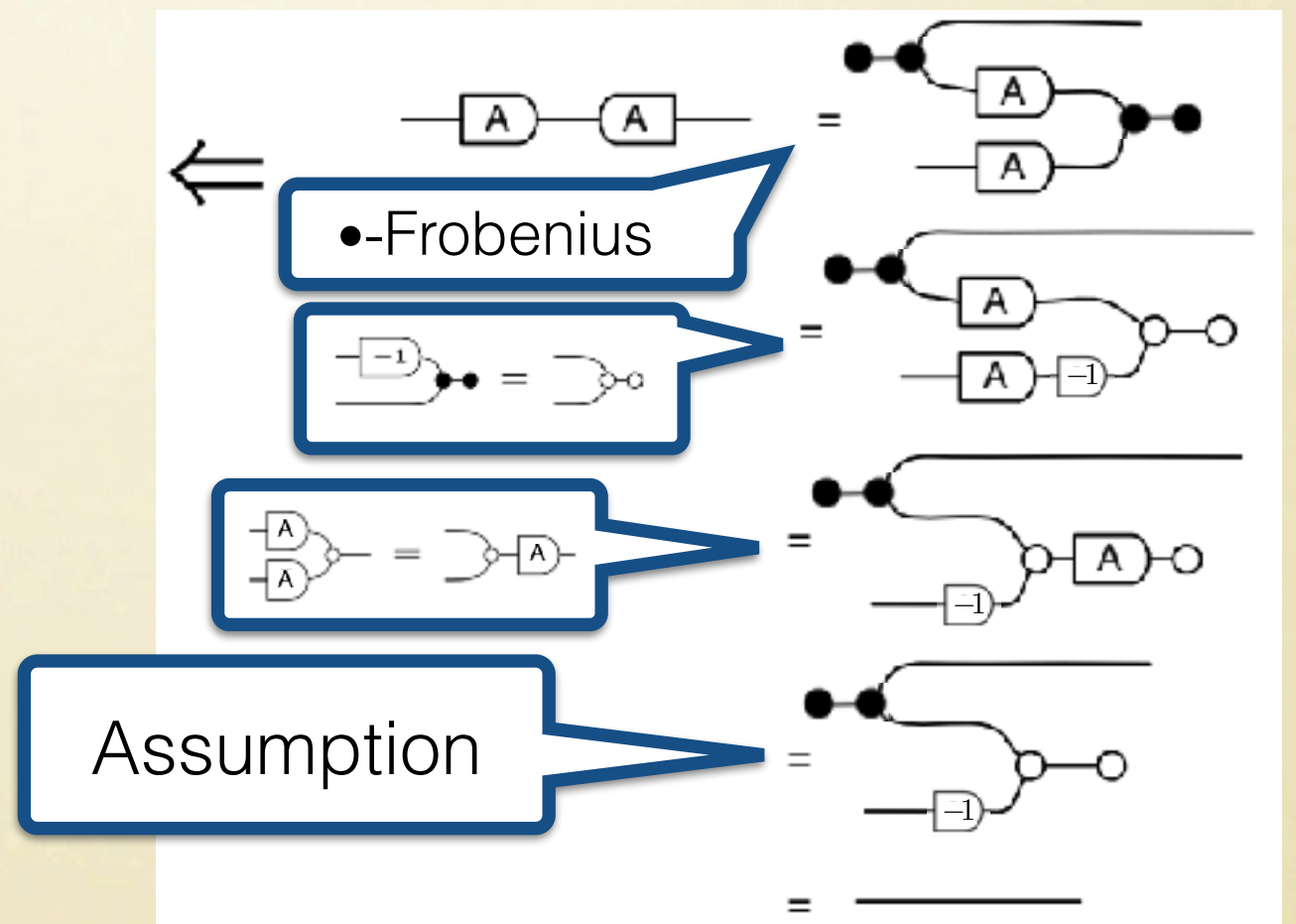
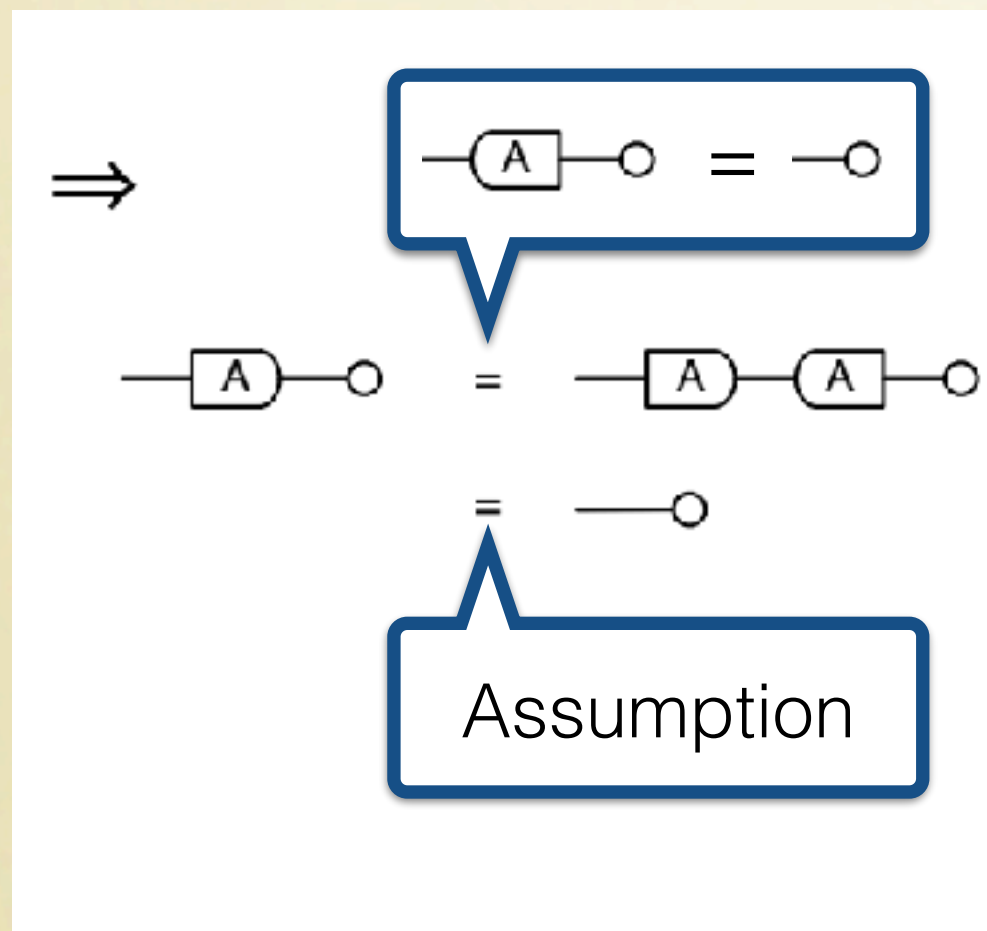


Example I: Linear Algebra

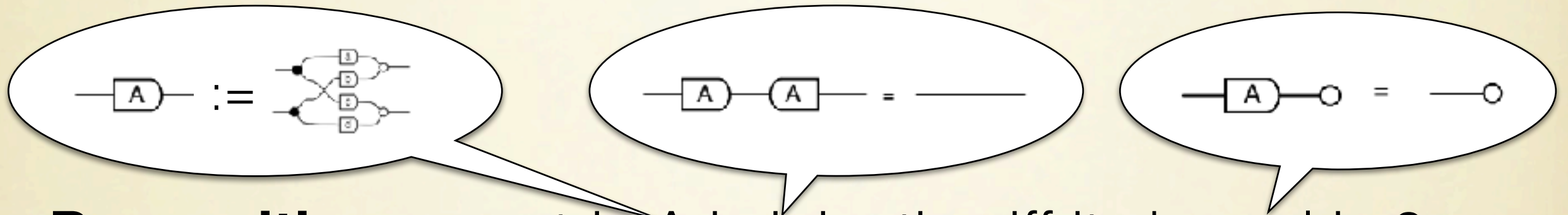


Proposition: a matrix A is injective iff its kernel is 0.

Proof

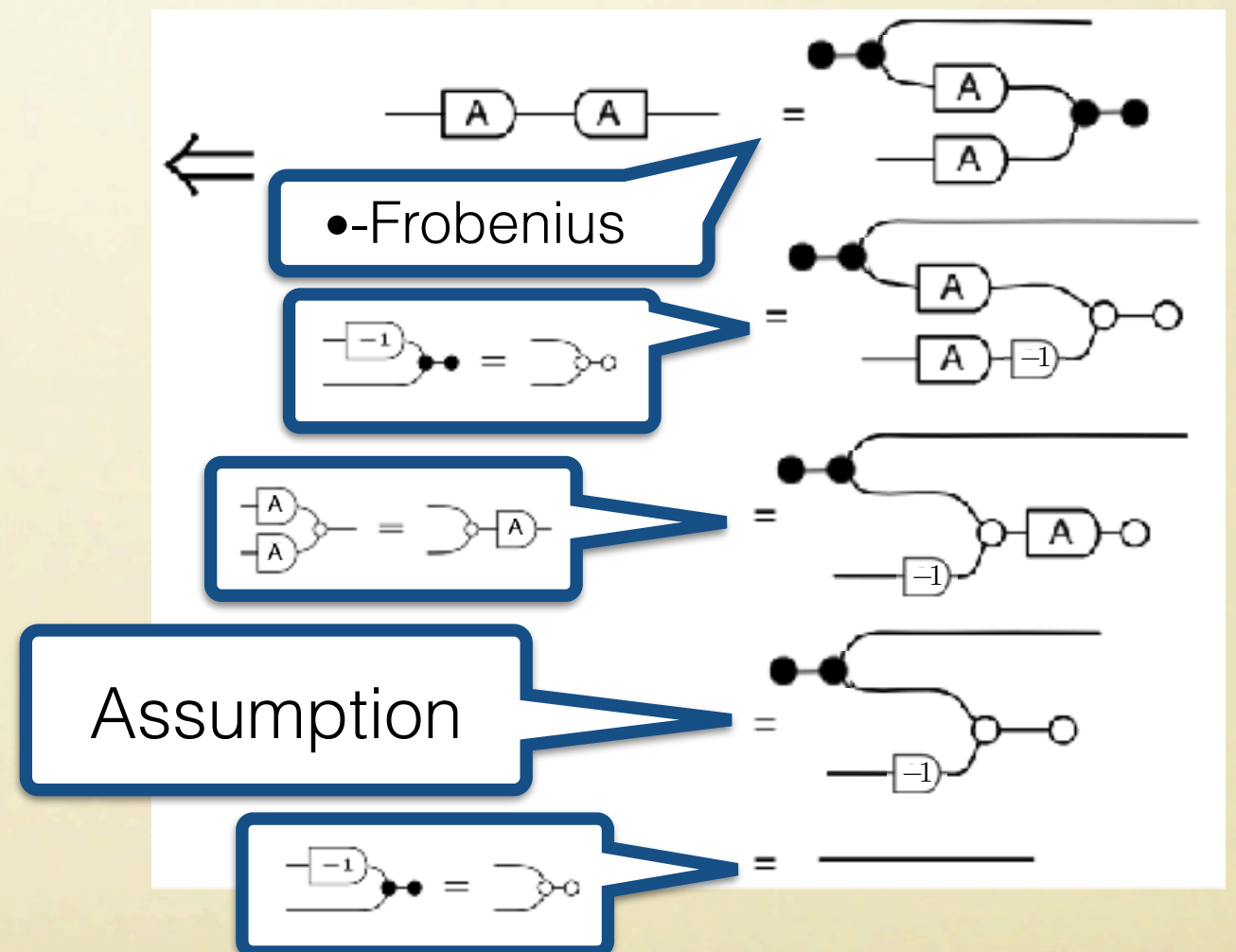
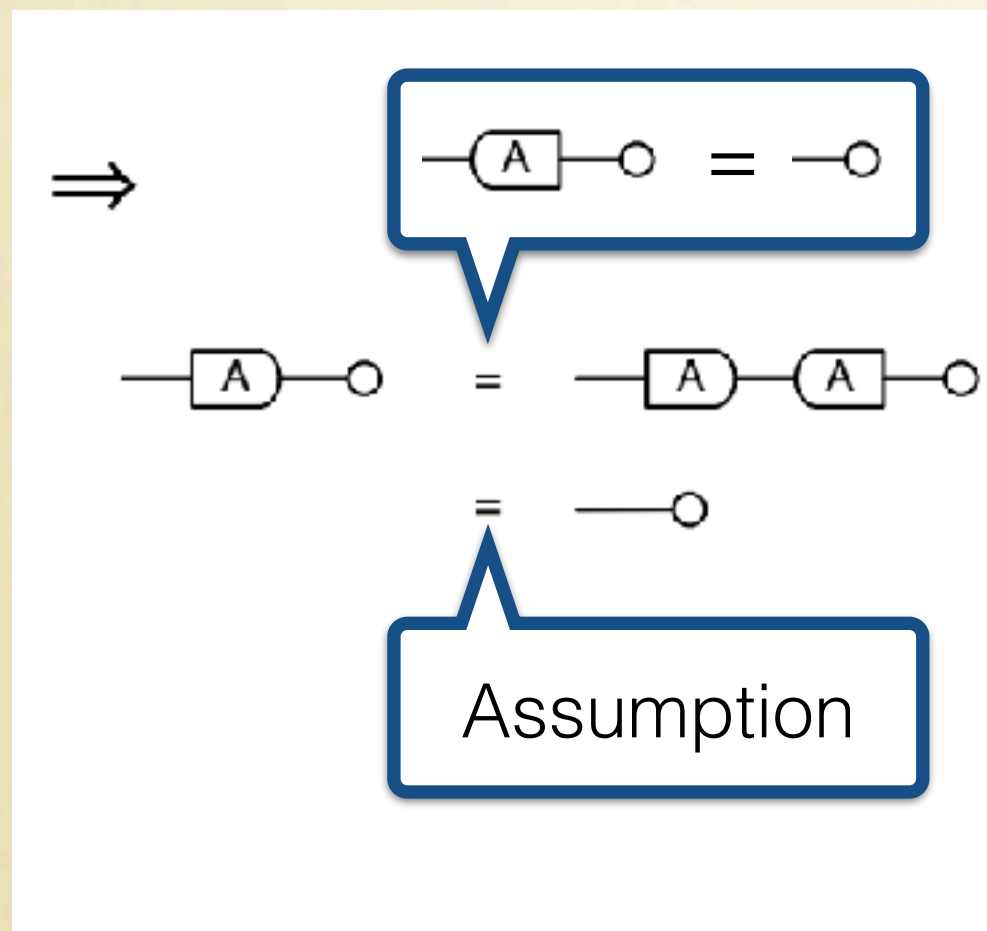


Example I: Linear Algebra



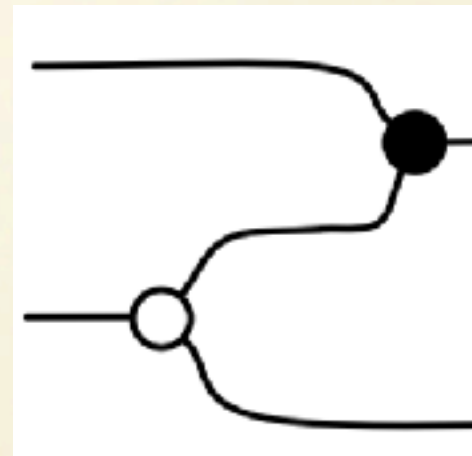
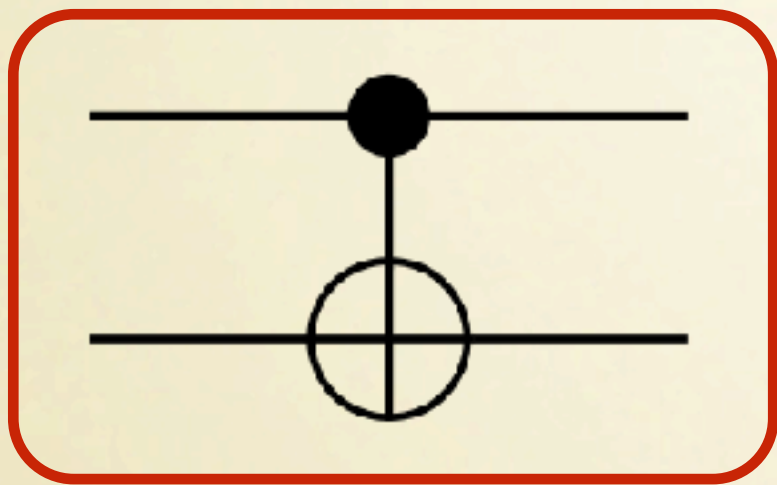
Proposition: a matrix A is injective iff its kernel is 0.

Proof



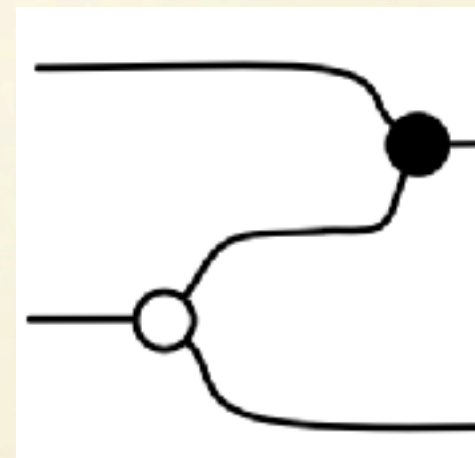
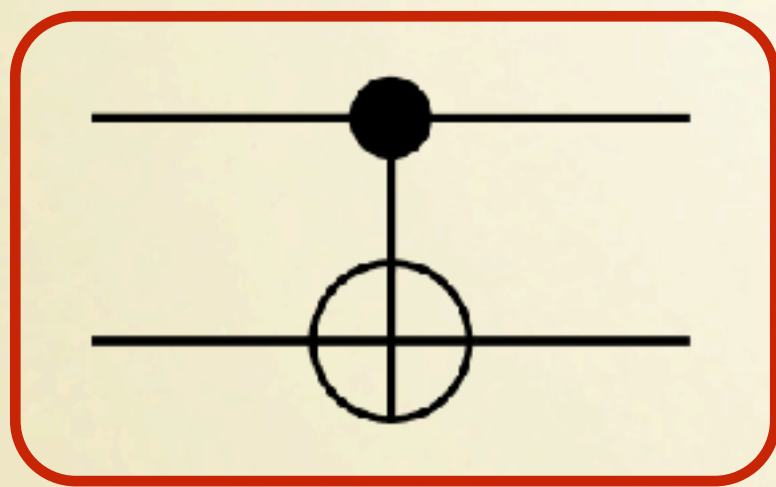
Example II: Quantum Circuits

CNOT gate interpreted as a signal flow diagram on $\mathbb{Z}/2$

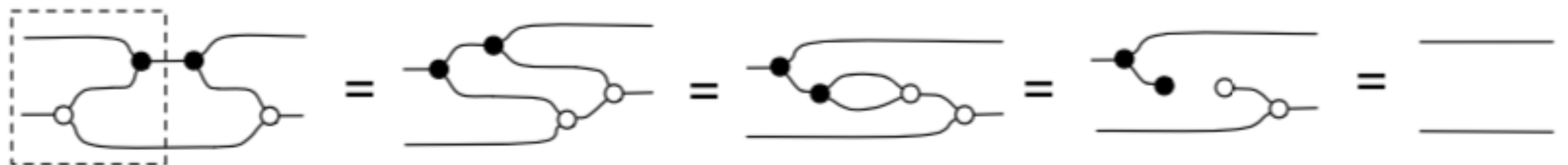


Example II: Quantum Circuits

CNOT gate interpreted as a signal flow diagram on $\mathbb{Z}/2$

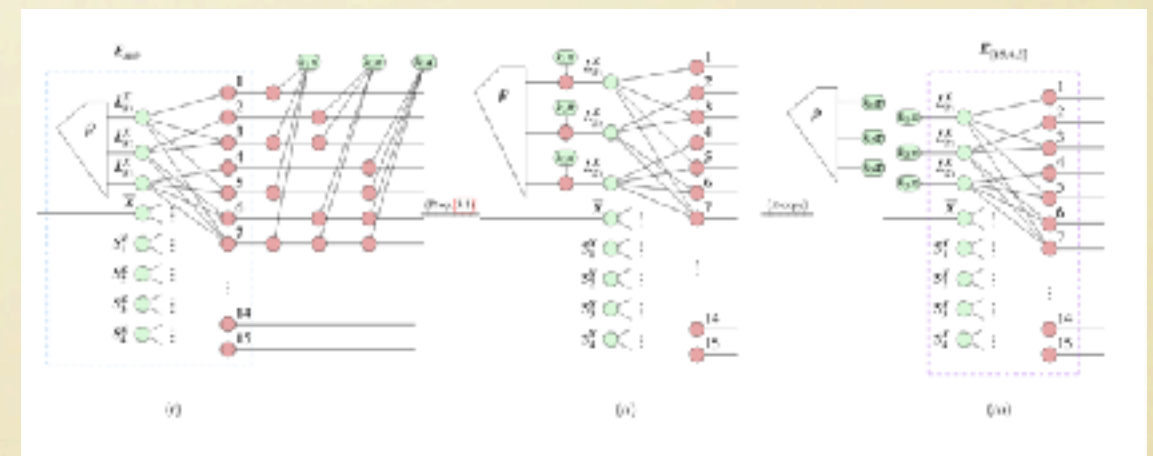
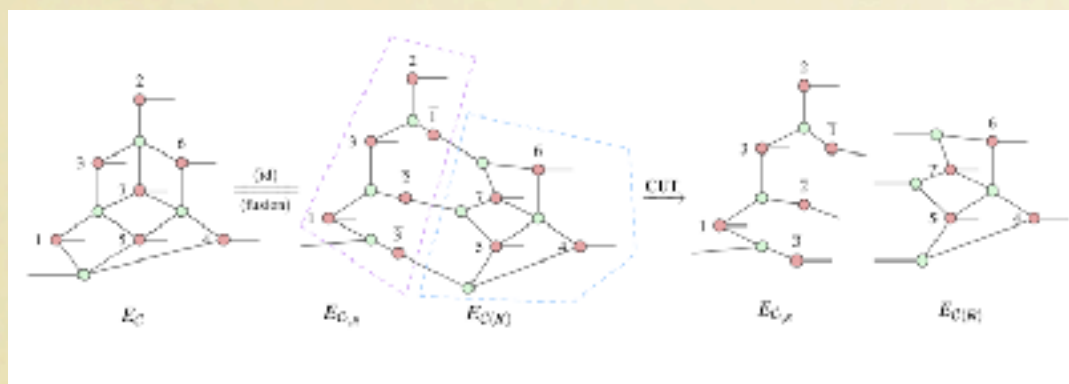
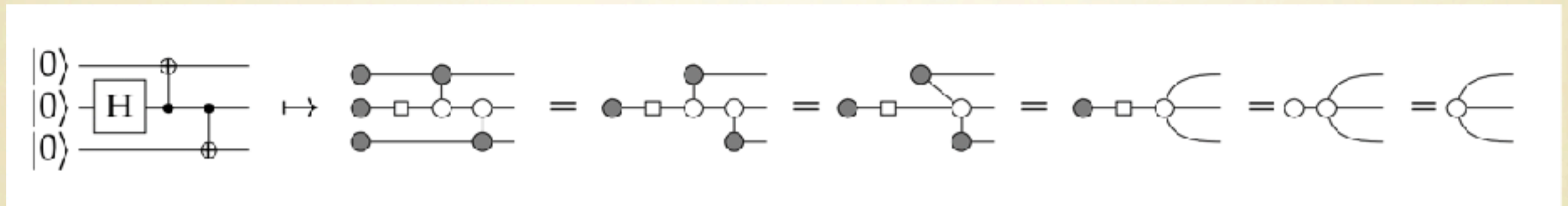


Proof by equational reasoning: CNOT gate is unitary



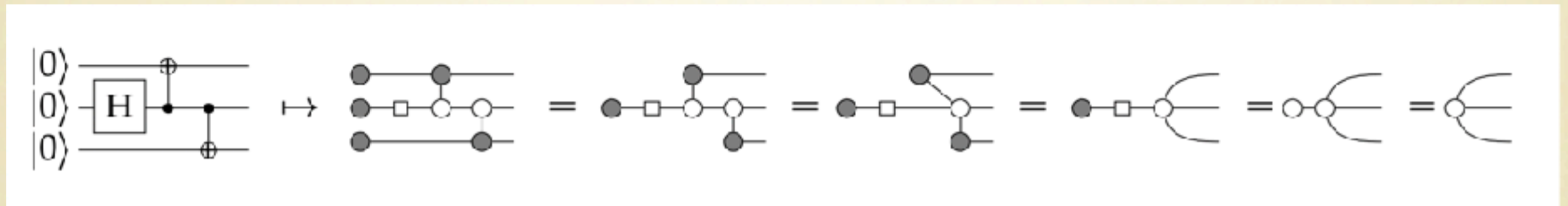
Example II: Quantum Circuits

A quantum circuit (left) preparing the GHZ state $|000\rangle + |111\rangle$



Example II: Quantum Circuits

A quantum circuit (left) preparing the GHZ state $|000\rangle + |111\rangle$



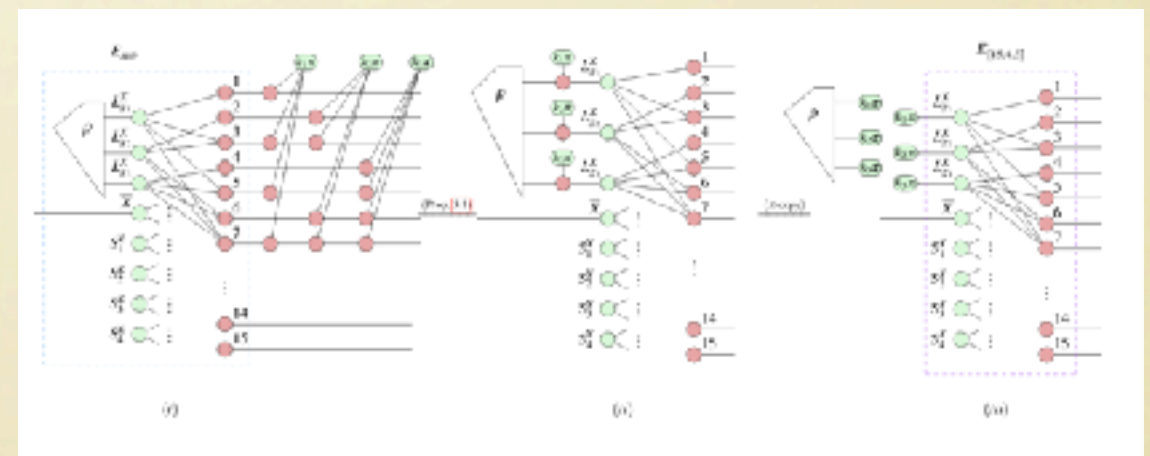
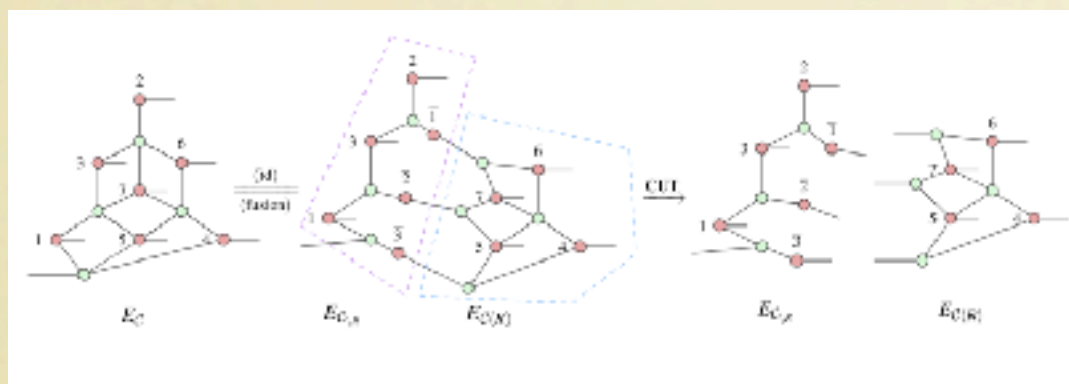
State-of-the-art algorithms for quantum circuits:

Graph-theoretic Simplification of Quantum Circuits with the ZX-calculus

Ross Duncan, Aleks Kissinger, Simon Perdrix, John van de Wetering

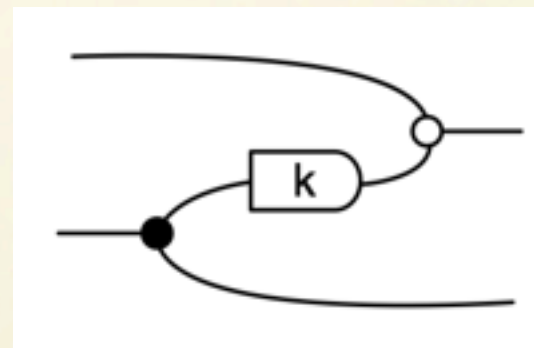
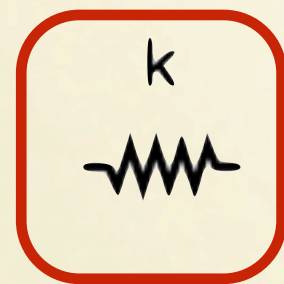
Graphical quantum Clifford-encoder compilers from the ZX calculus

Andrey Boris Khesin, Jonathan Z. Lu, Peter W. Shor

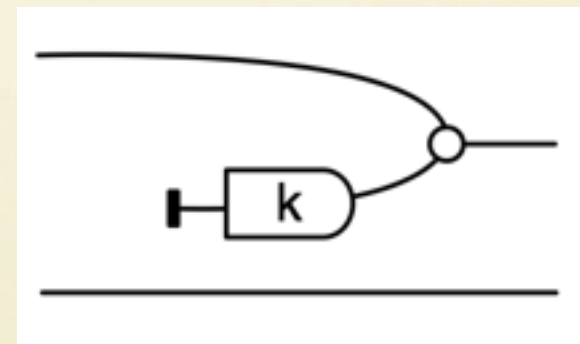
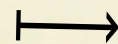
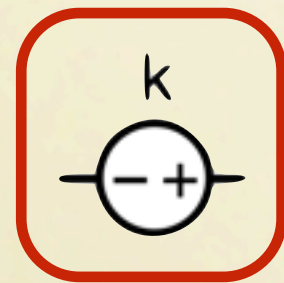


Example III: Electrical Circuits

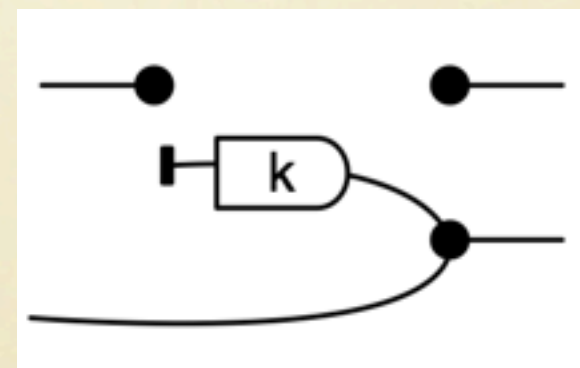
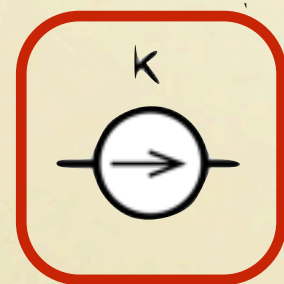
Encode electrical components as circuit diagrams on $\mathbb{R}$



Resistor



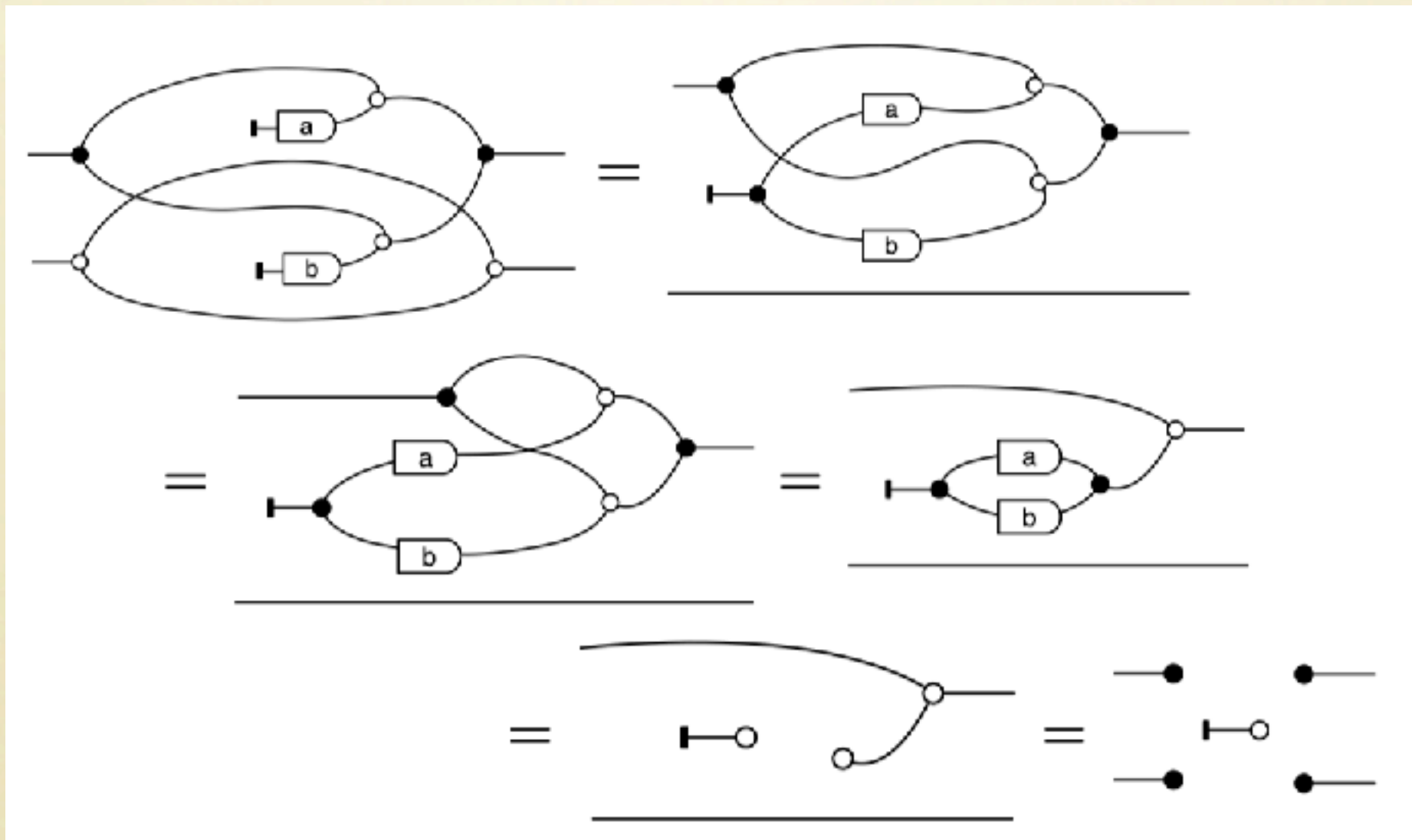
Voltage



Current

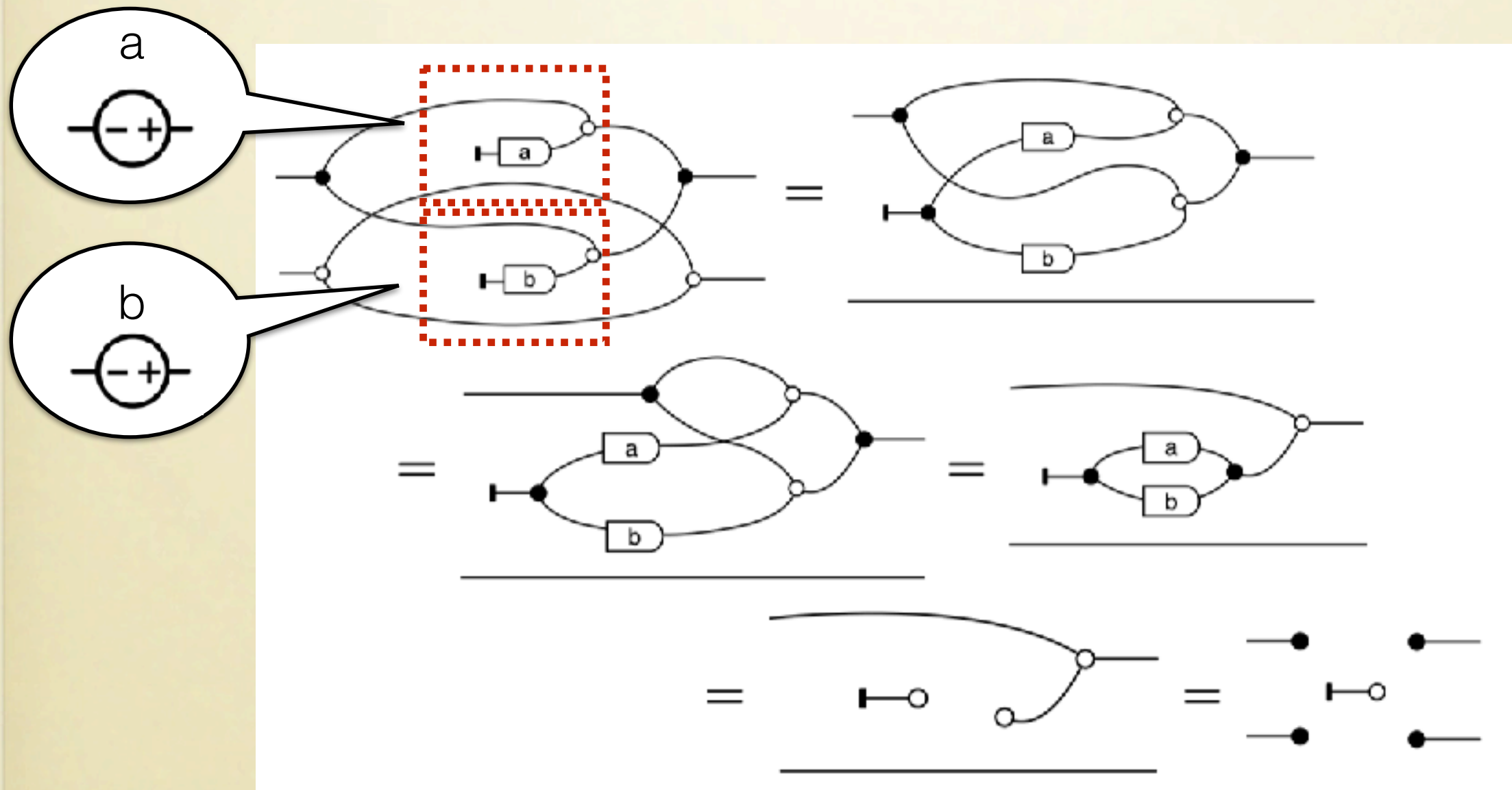
Example III: Electrical Circuits

Proof that parallel voltage sources of different voltages are disallowed.



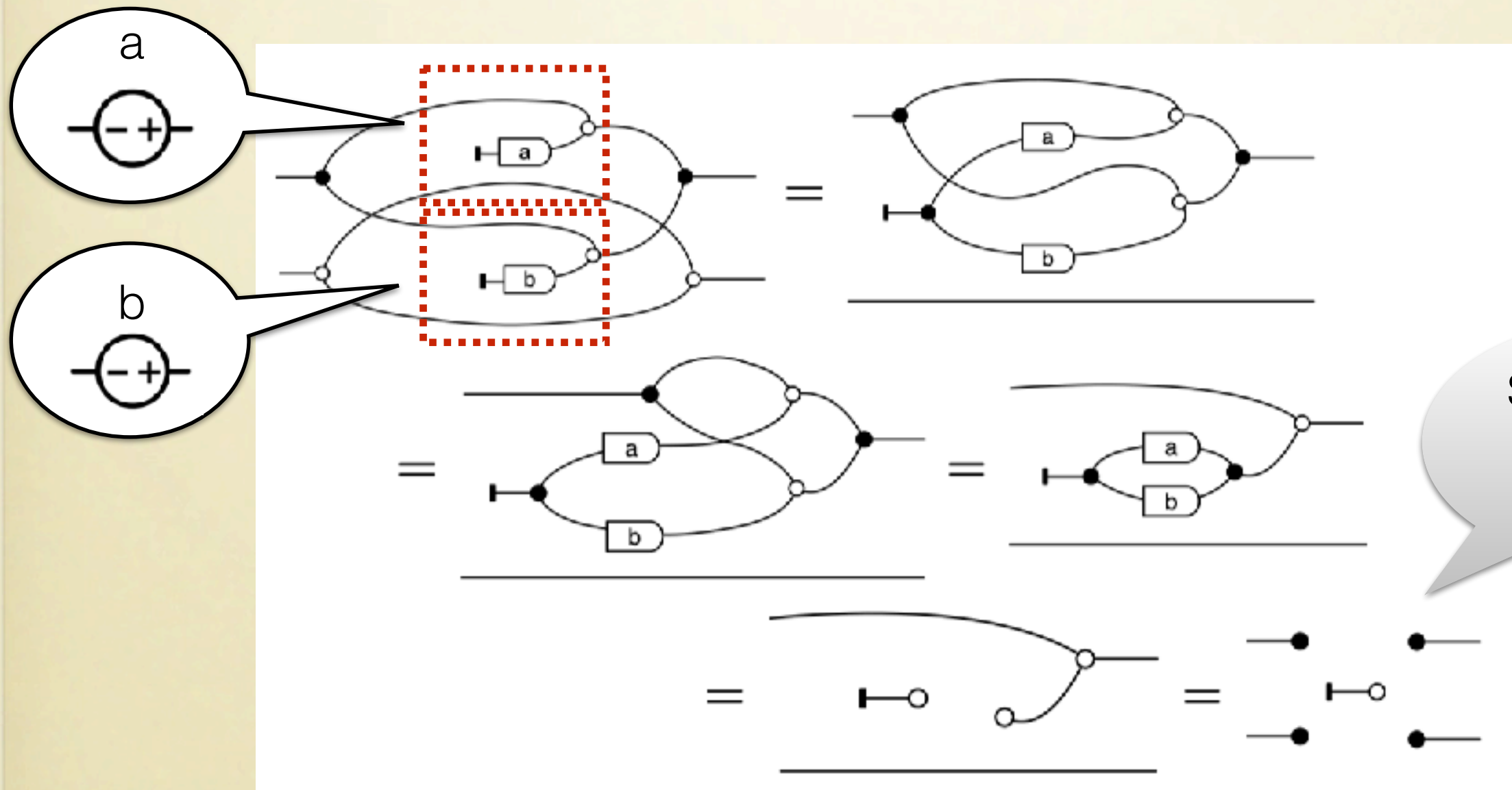
Example III: Electrical Circuits

Proof that parallel voltage sources of different voltages are disallowed.



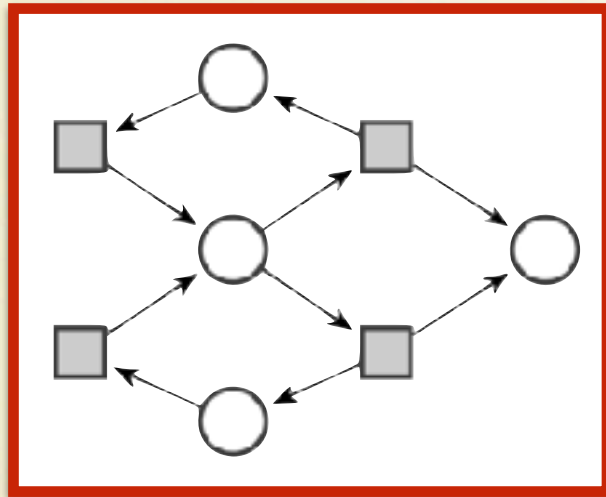
Example III: Electrical Circuits

Proof that parallel voltage sources of different voltages are disallowed.

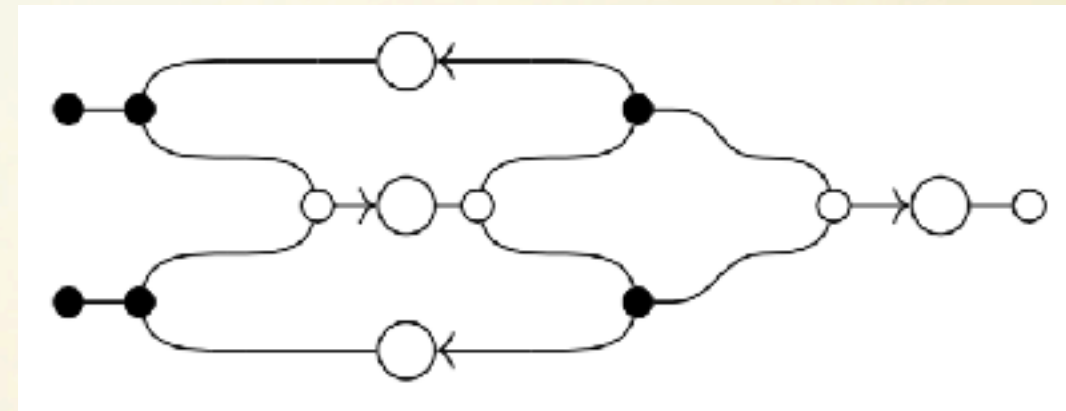


Semantics:
the empty
relation.

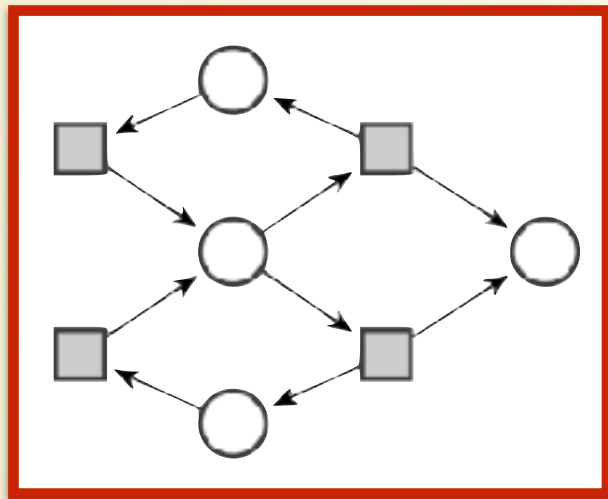
Example V: Petri Nets



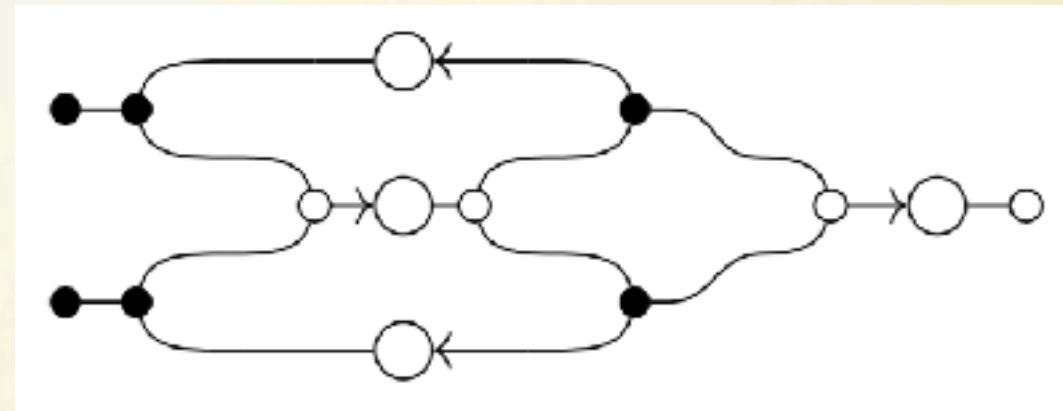
$\mapsto$



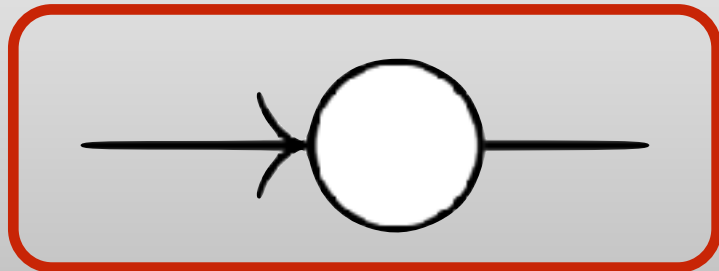
Example V: Petri Nets



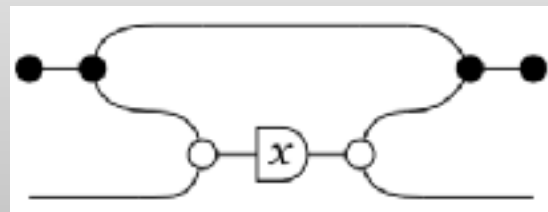
$\mapsto$



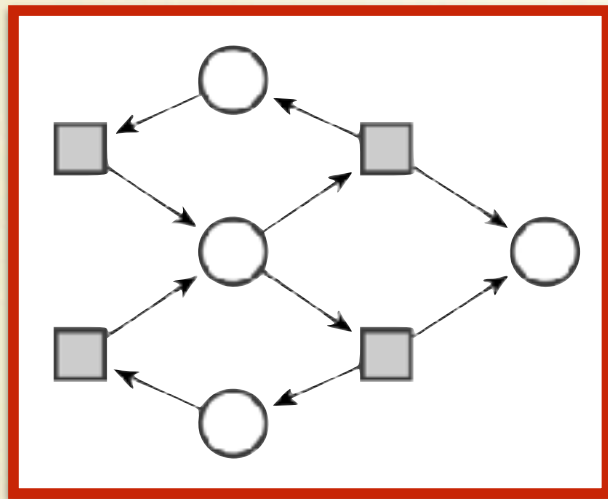
We interpret the **place of a Petri net** as a signal flow diagram over the semiring $\mathbb{N}$.



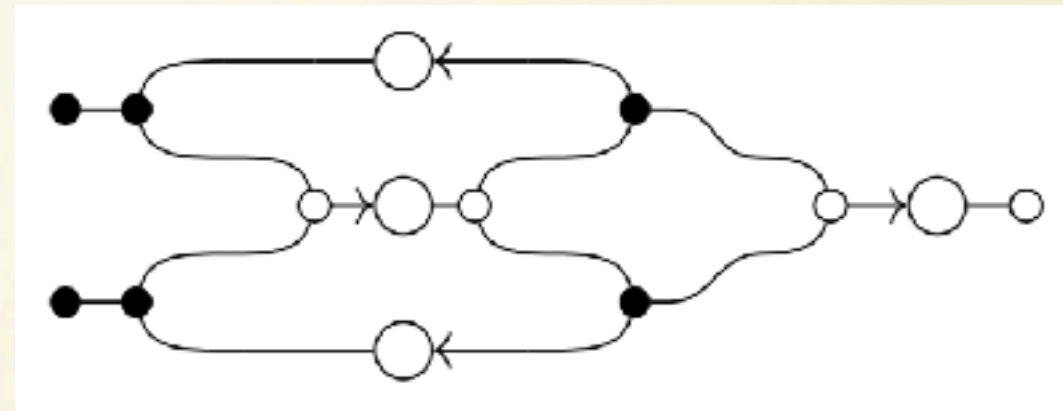
$\mapsto$



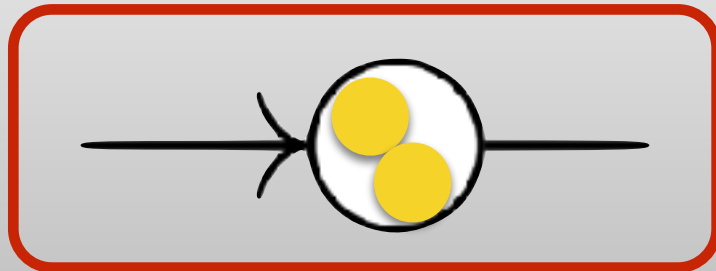
Example V: Petri Nets



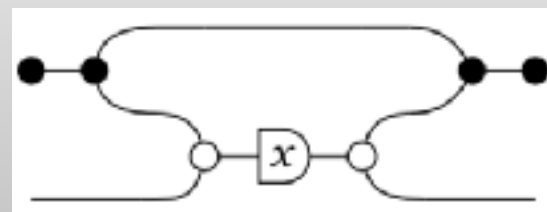
$\mapsto$



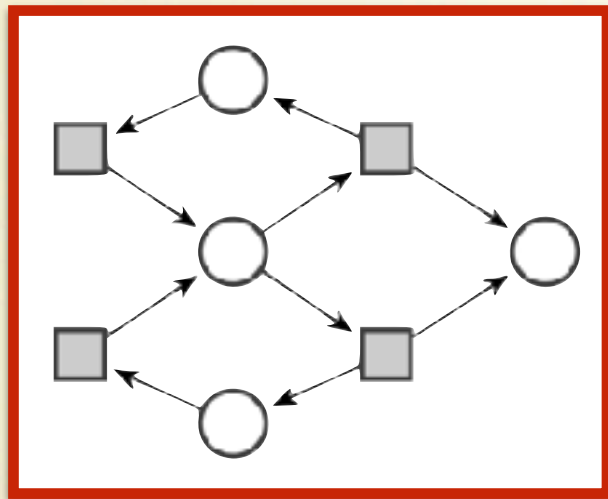
We interpret the **place of a Petri net** as a signal flow diagram over the semiring $\mathbb{N}$.



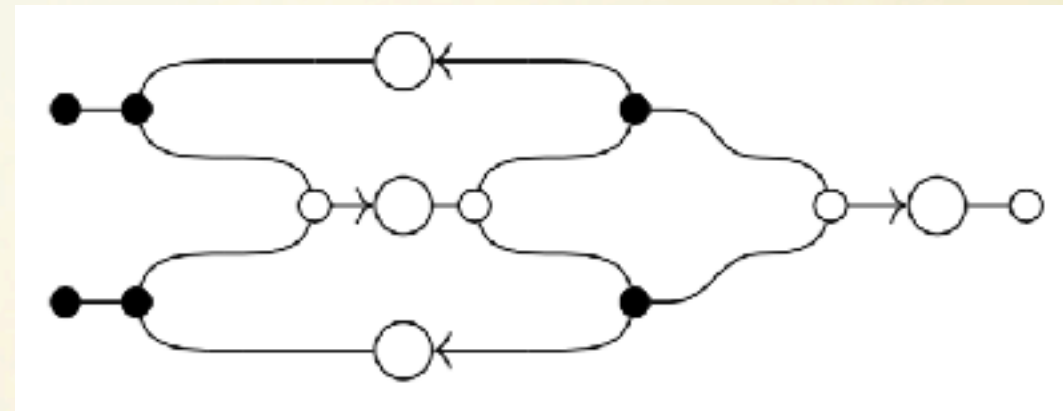
$\mapsto$



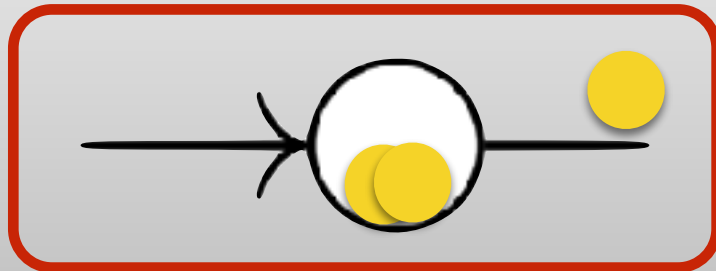
Example V: Petri Nets



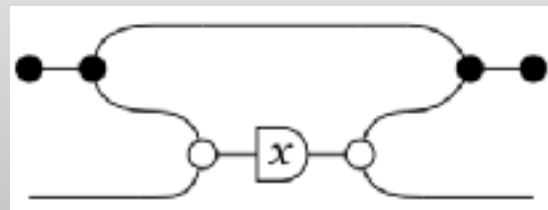
$\mapsto$



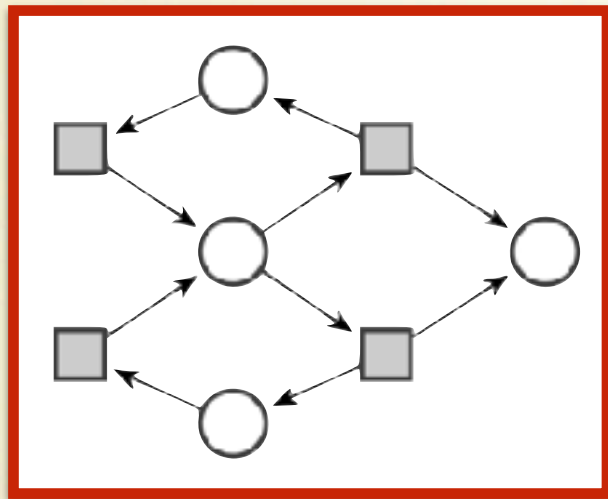
We interpret the **place of a Petri net** as a signal flow diagram over the semiring $\mathbb{N}$.



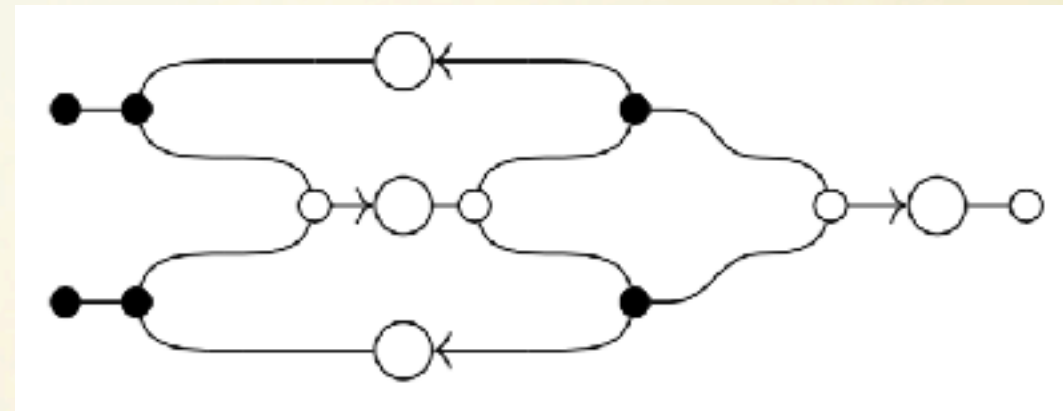
$\mapsto$



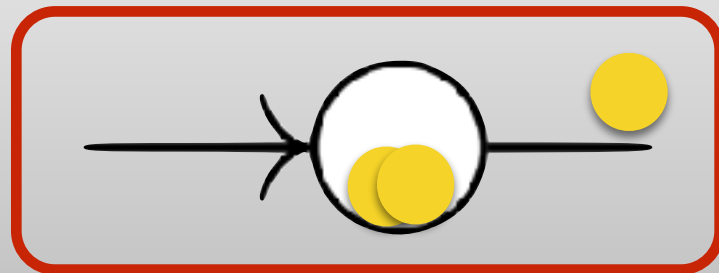
Example V: Petri Nets



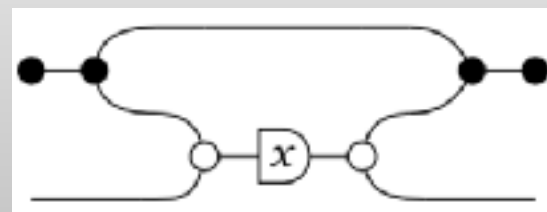
$\mapsto$



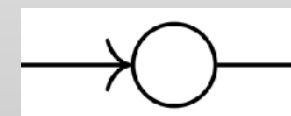
We interpret the **place of a Petri net** as a signal flow diagram over the semiring $\mathbb{N}$.



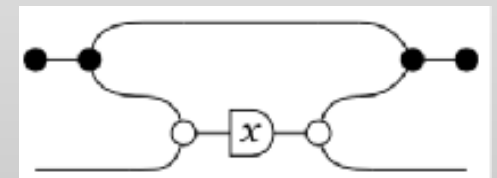
$\mapsto$



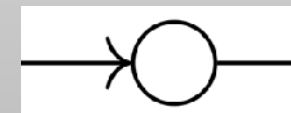
Firing
semantics



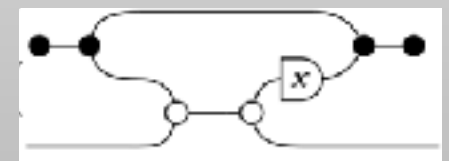
$::=$



Banking
semantics



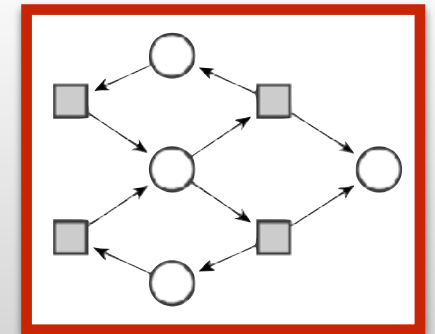
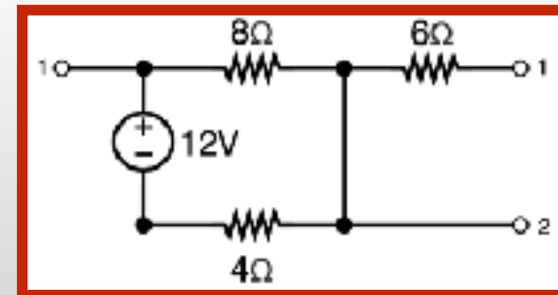
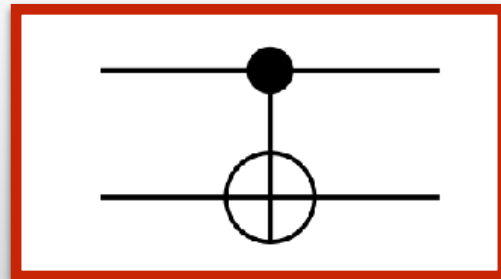
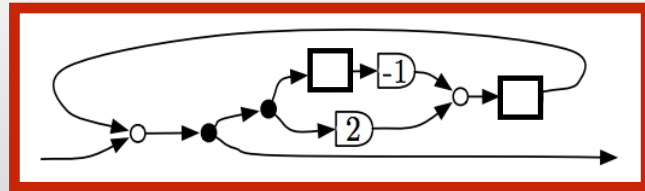
$::=$



A uniform picture

A uniform picture

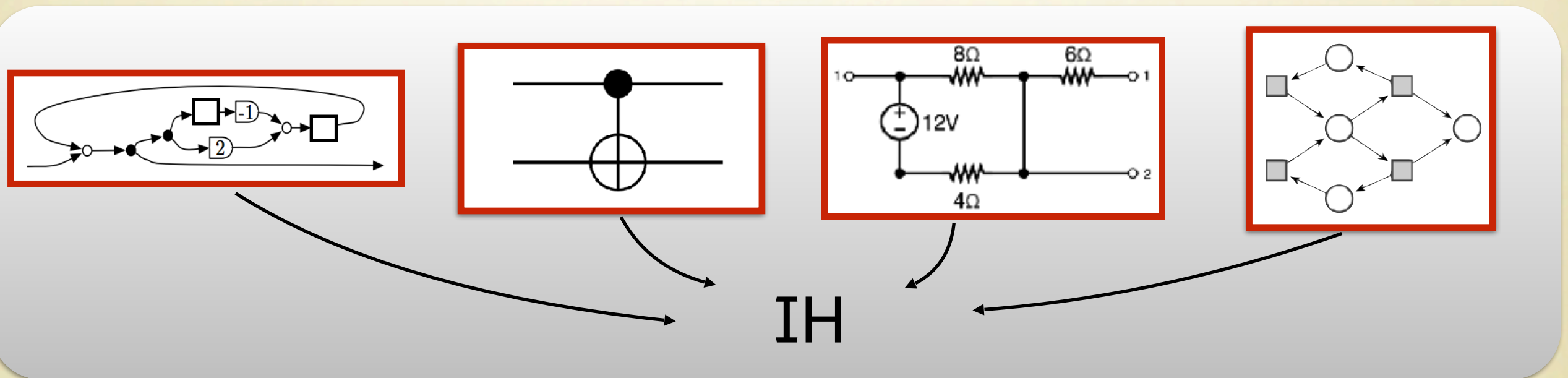
Signal flow diagrams (with their equational theory IH) acts as an **assembly language** for diverse families of networks.



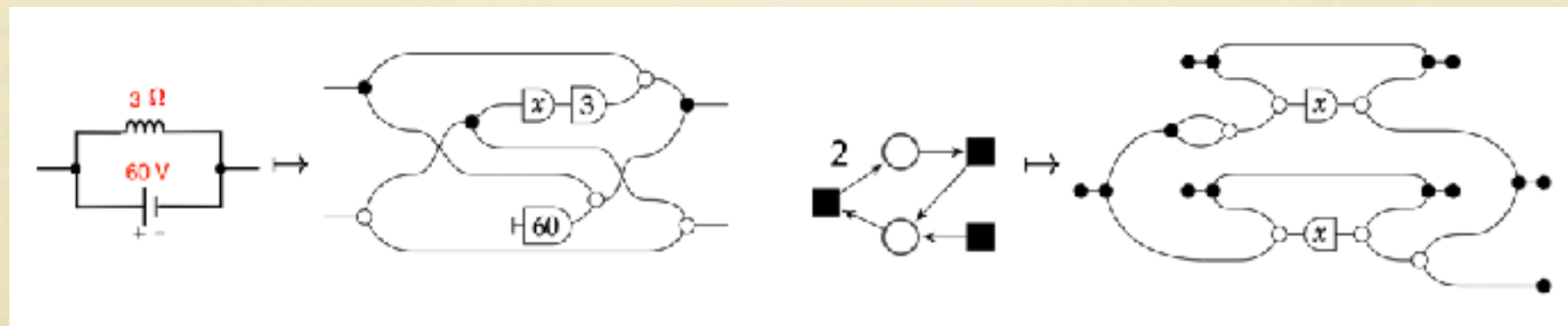
IH

A uniform picture

Signal flow diagrams (with their equational theory IH) acts as an **assembly language** for diverse families of networks.

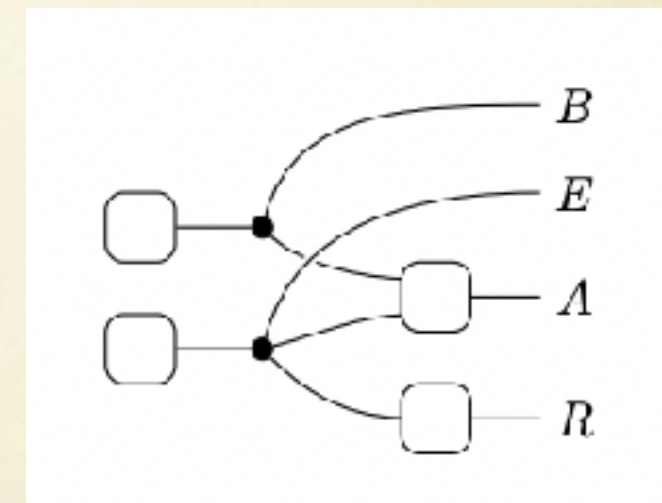
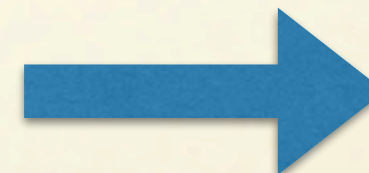
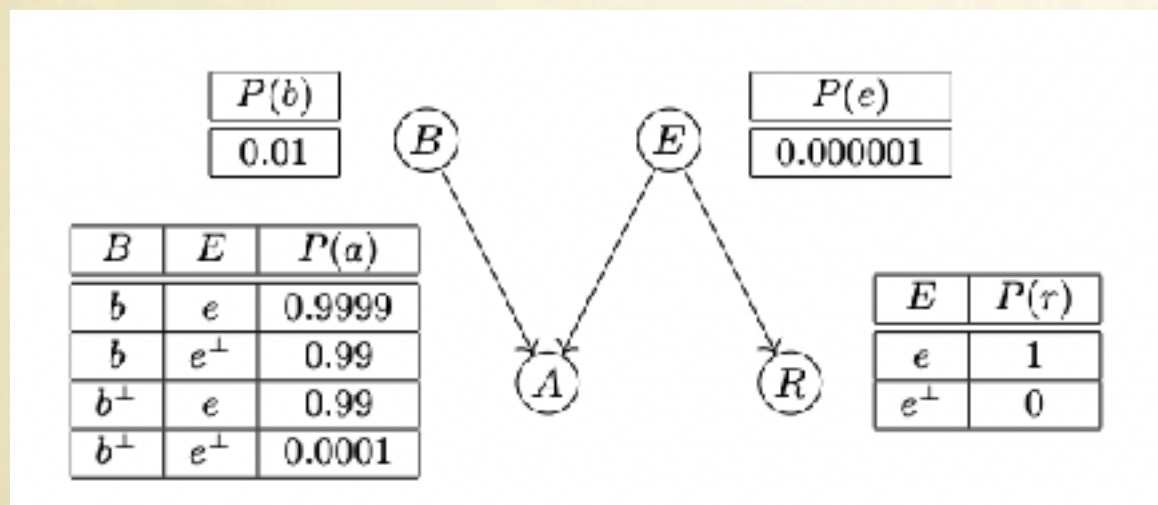


This approach reveals different models are reducible to the same elementary interactions.

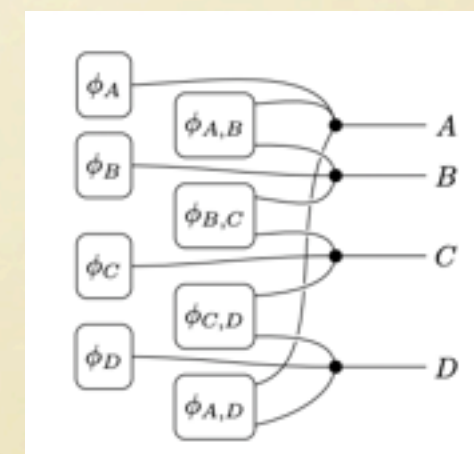
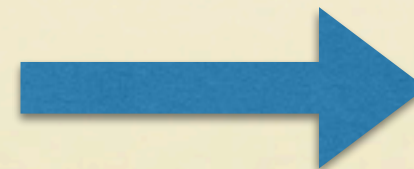
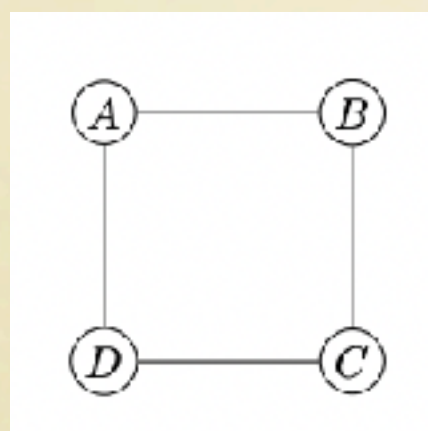


Example VI: Probabilistic Graphical Models

Bayesian Networks

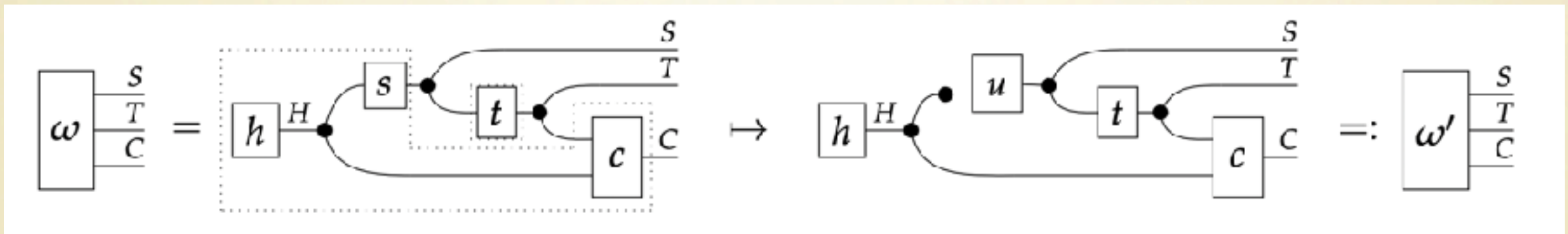


Markov Networks



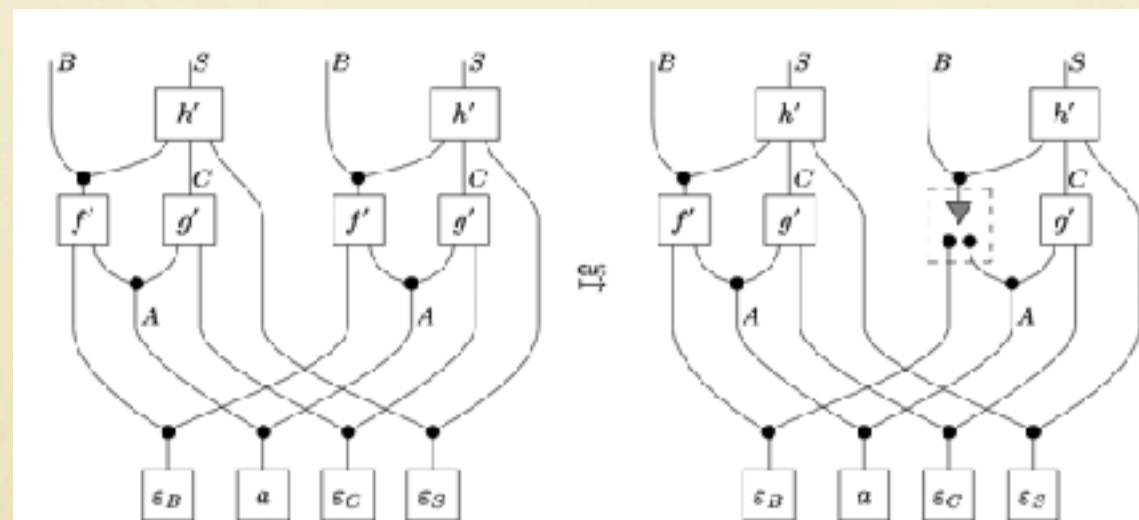
Example VI: Bayesian Probability

Causal Inference



Establish whether statistical correlation between smoking and cancer actually identifies a causal relationship.

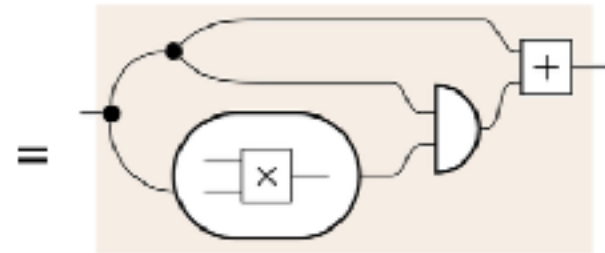
Counterfactuals via twin networks



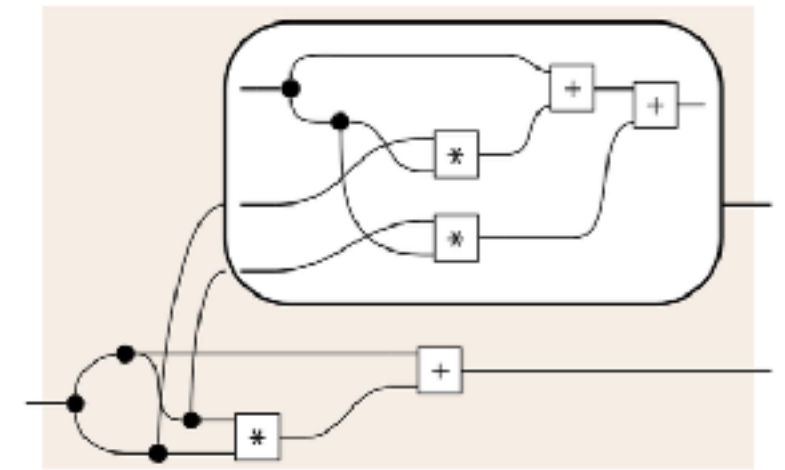
Example VII: Automatic Differentiation

Evaluating the derivative of a program function

```
def poly(x):  
  def mul(y):  
    return x * y  
  return mul(x) + x
```



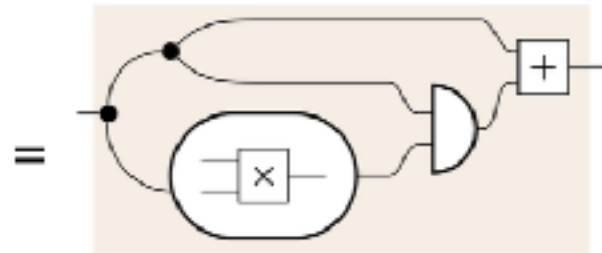
$\Rightarrow^*$



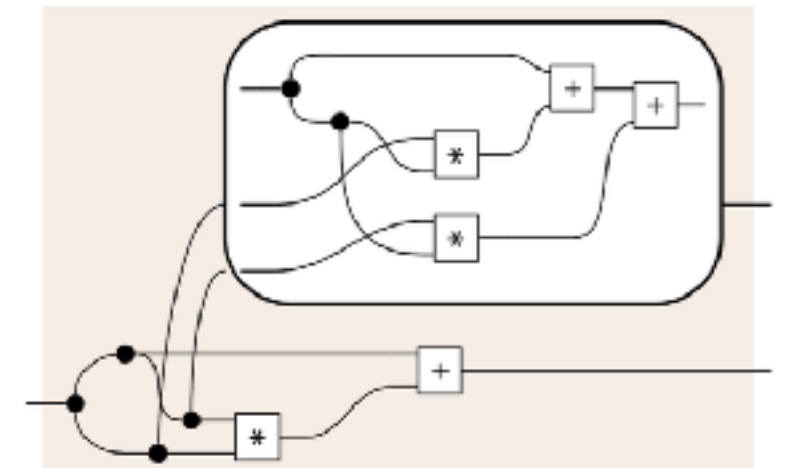
Example VII: Automatic Differentiation

Evaluating the derivative of a program function

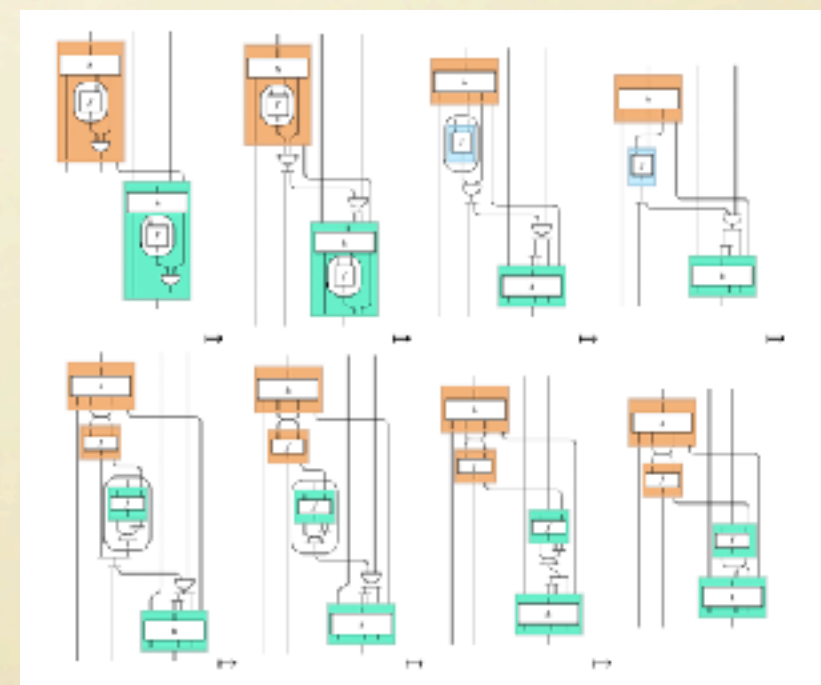
```
def poly(x):  
  def mul(y):  
    return x * y  
  return mul(x) + x
```



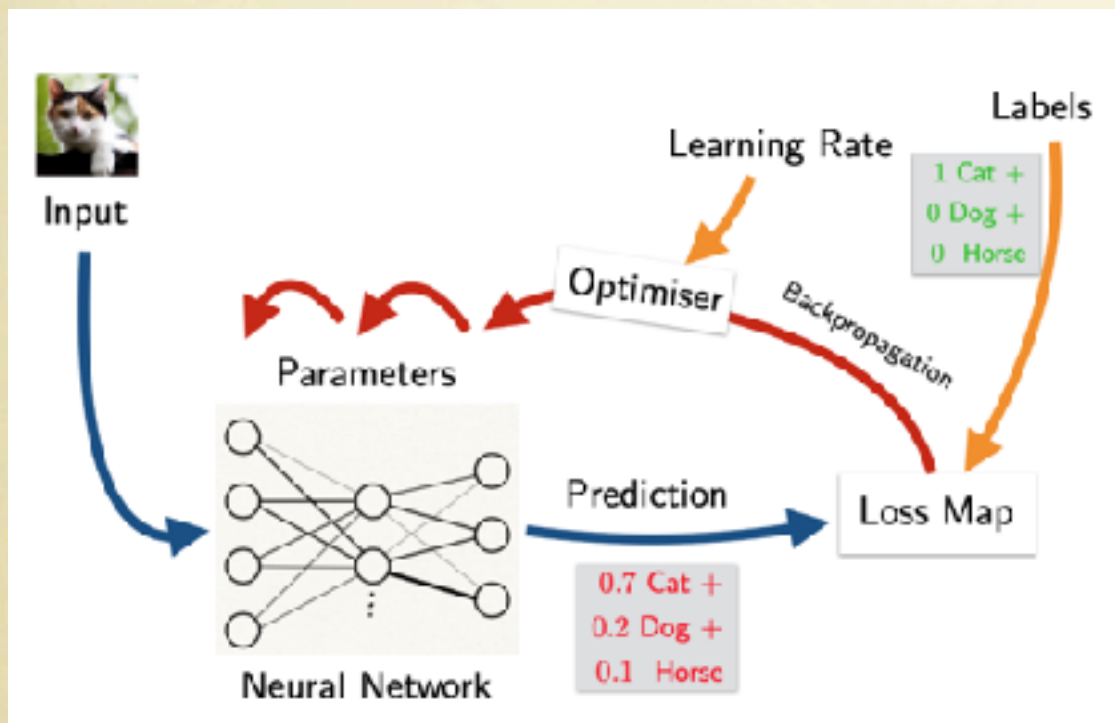
$\Rightarrow^*$



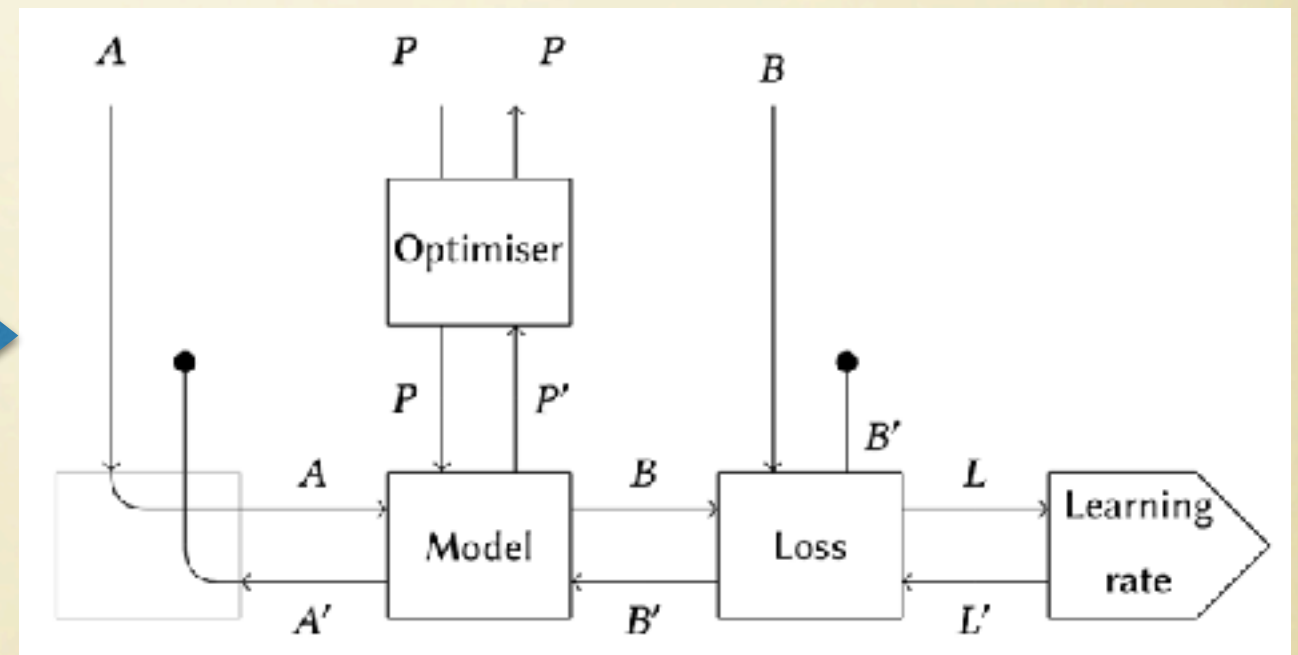
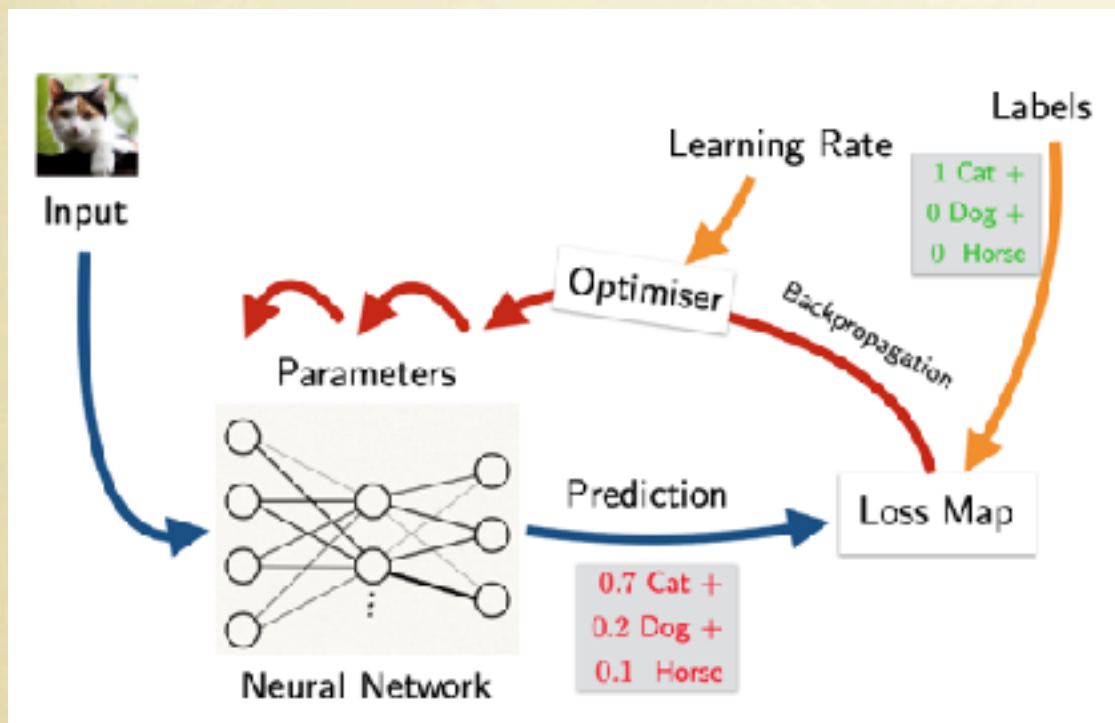
Correctness Theorem
via String Diagram Rewriting



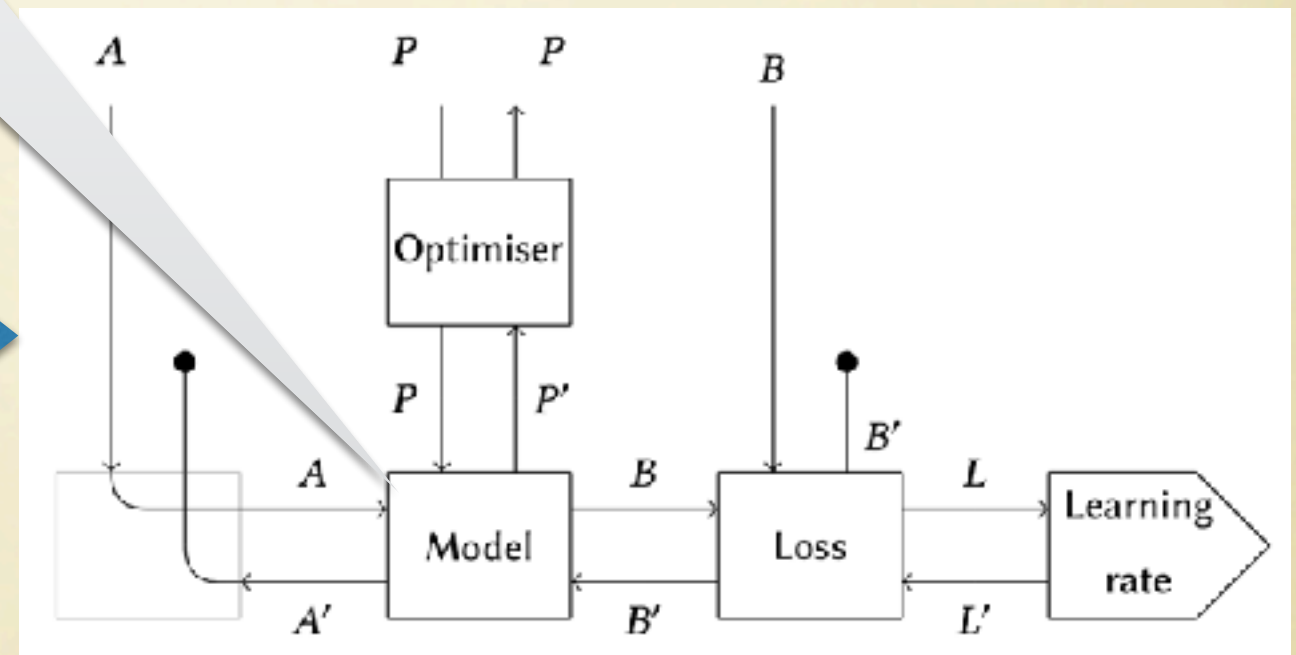
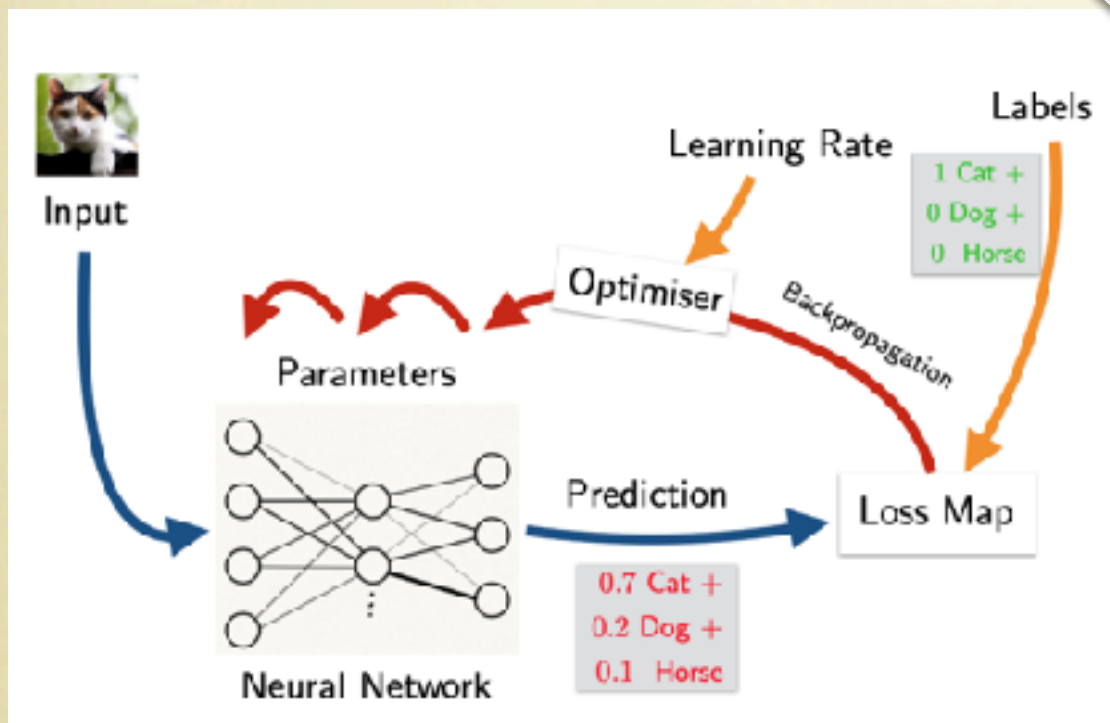
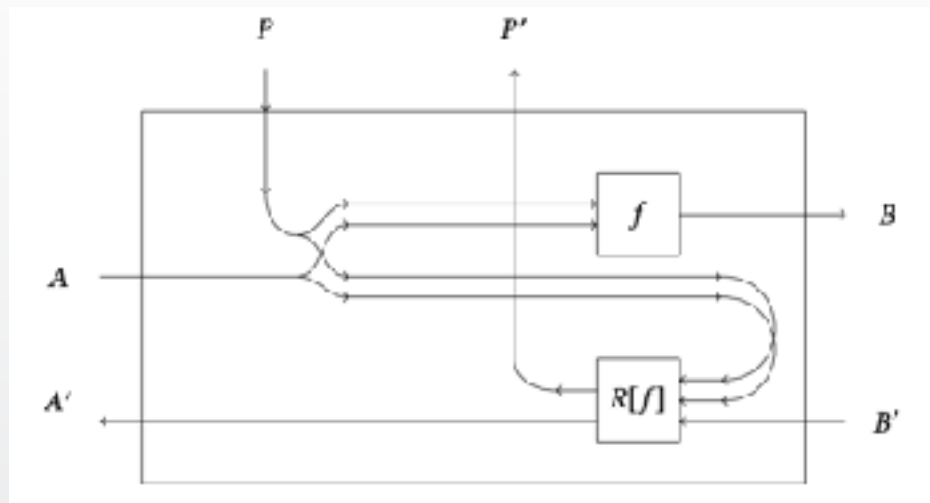
Example VIII: Deep Learning



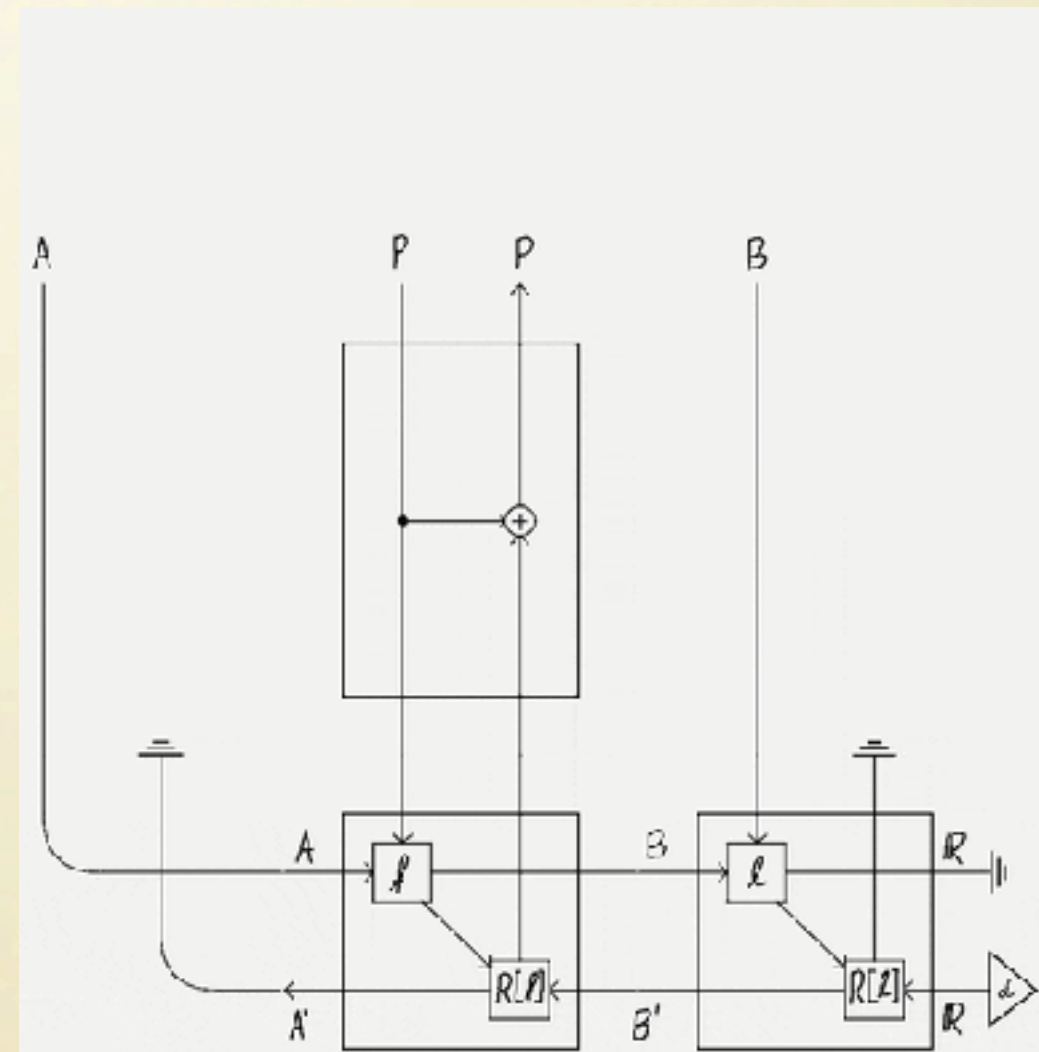
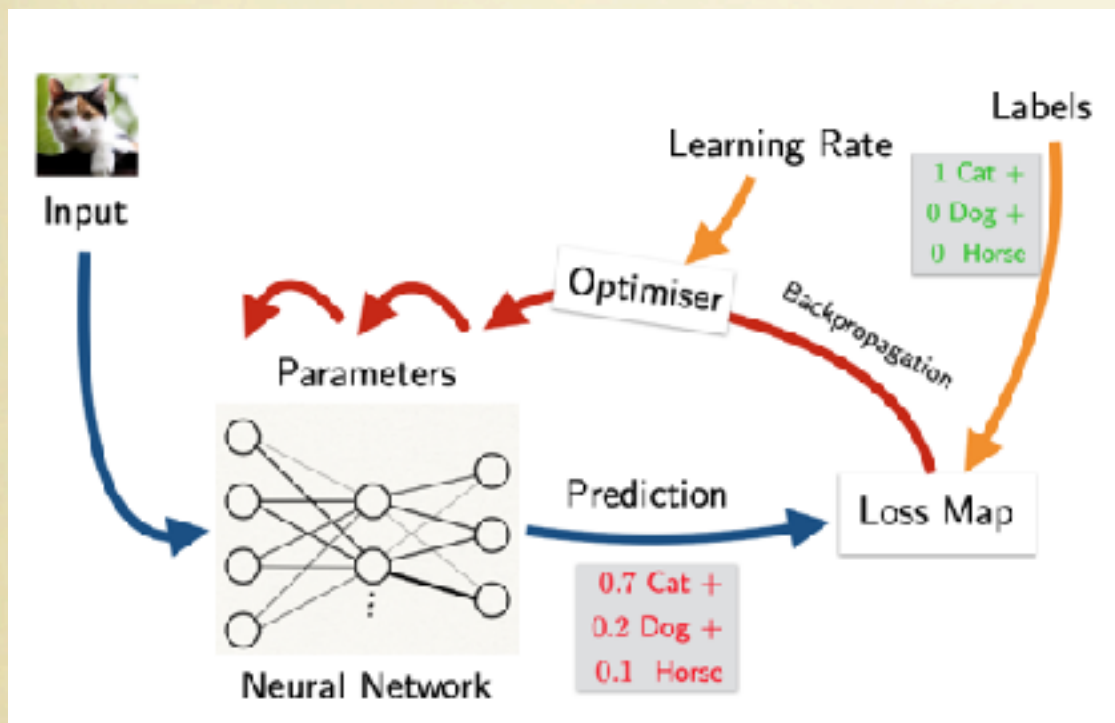
Example VIII: Deep Learning



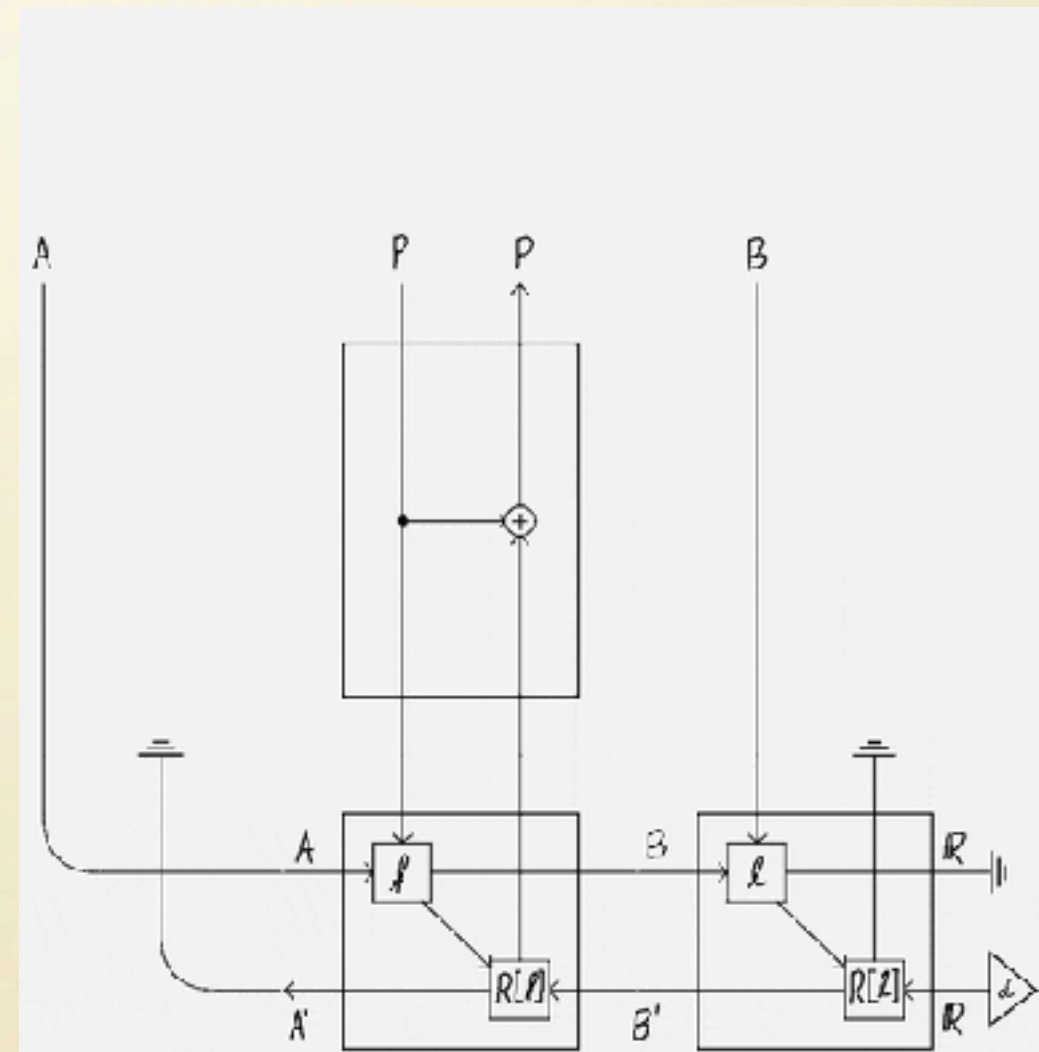
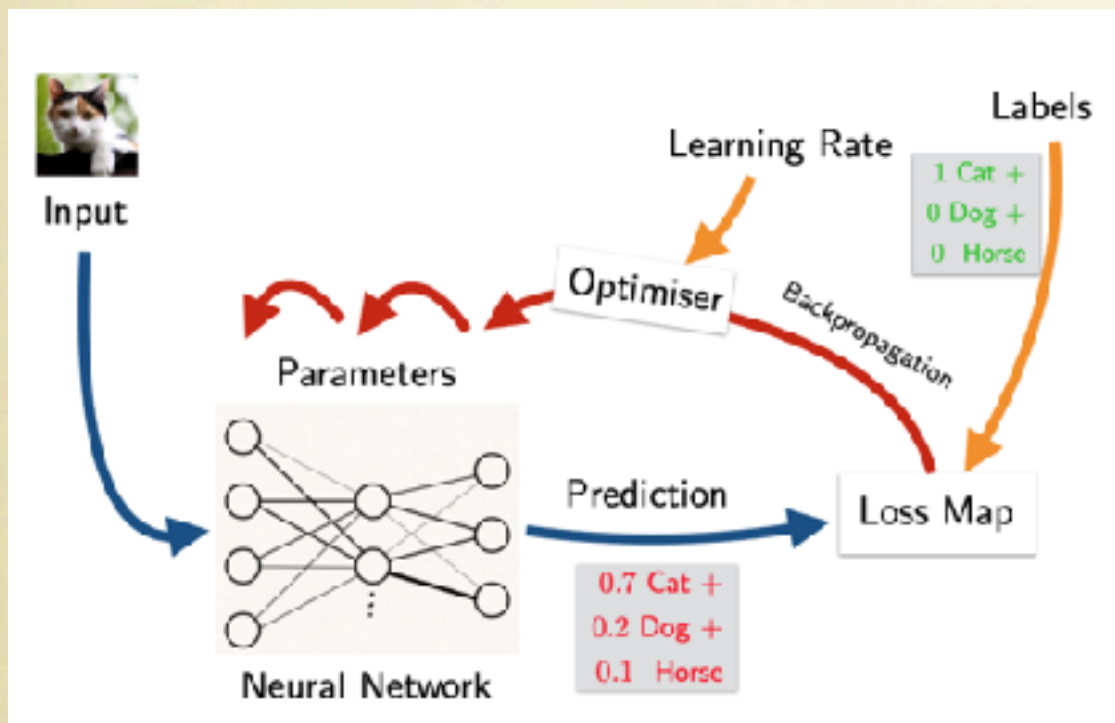
Example VIII: Deep Learning



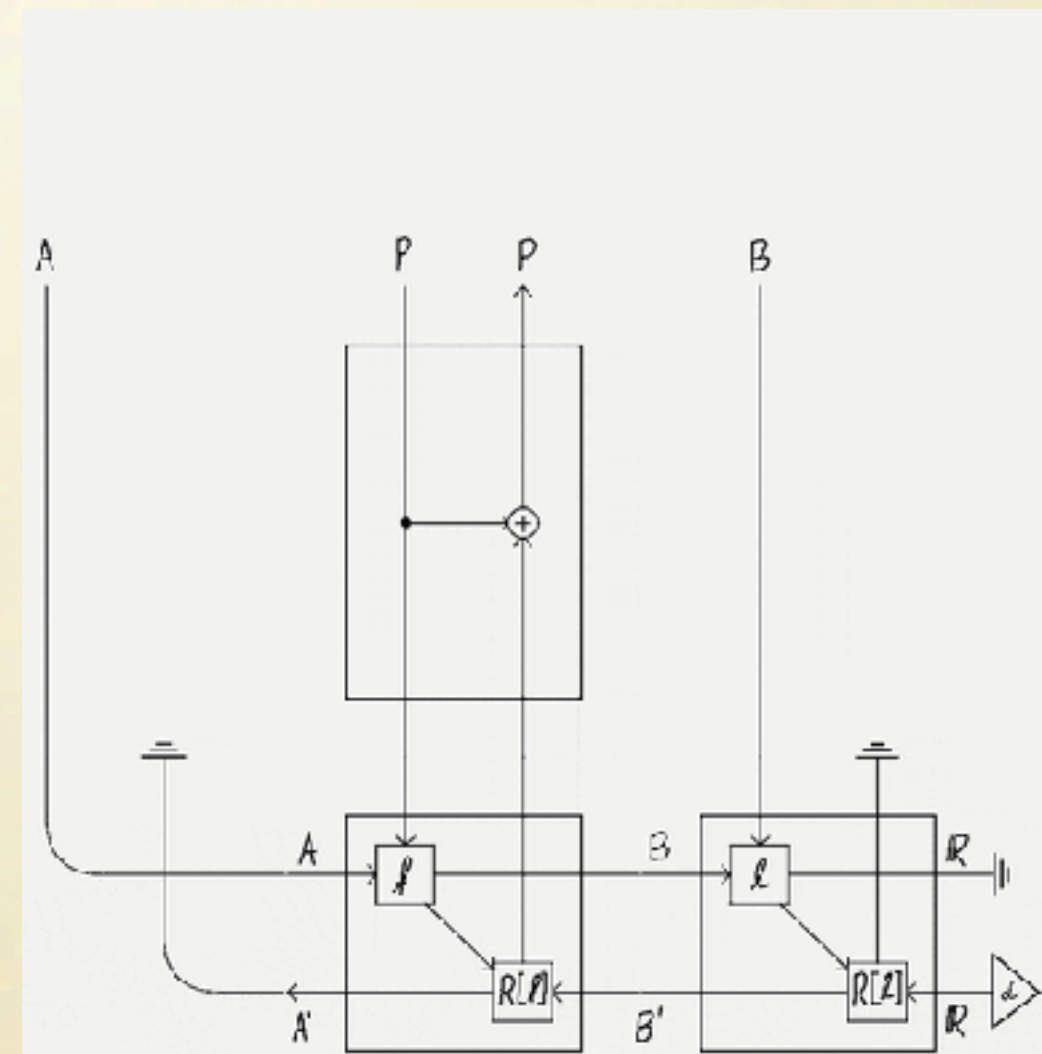
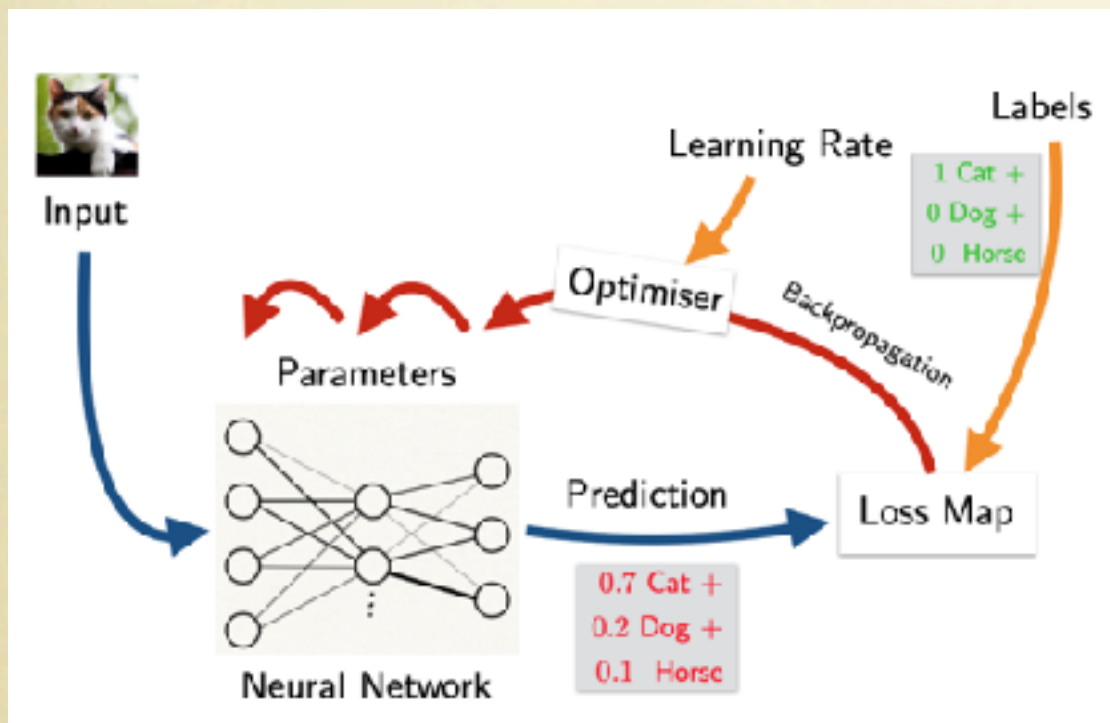
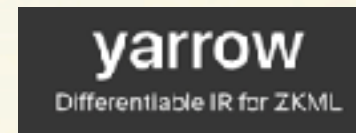
Example VIII: Deep Learning



Example VIII: Deep Learning



Example VIII: Deep Learning

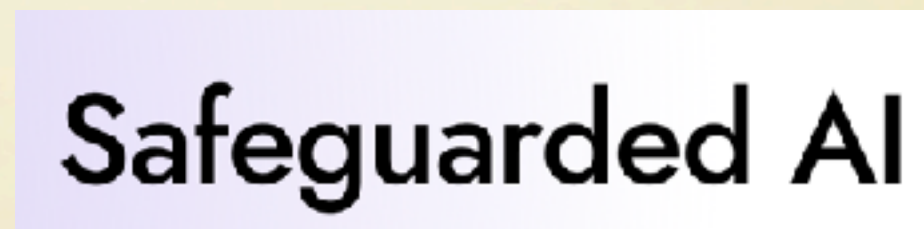
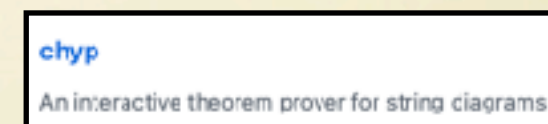
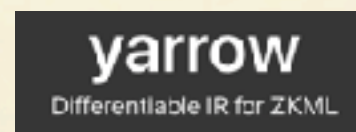
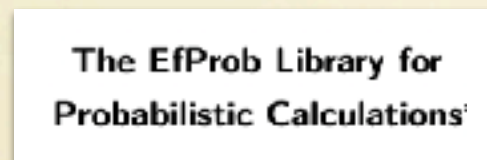
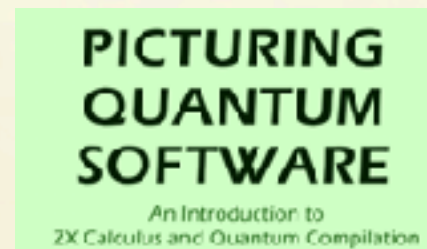
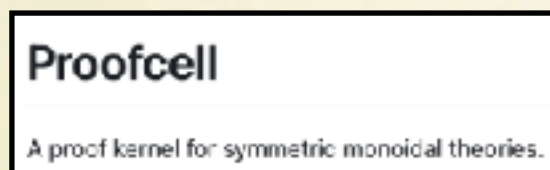


From Theory to Implementation

Programming Principles

System Design/Specification

Theorem Proving and Proof Checking



Breakthroughs: Quantum Circuit Optimisation, Circuit Compilation, Causal Inference/Discovery, Gradient-Descent Learning, ...

Part III

Conclusions

Summing Up

String diagrams intend to take the best of two worlds:

- They are a **formal syntax**, which can be reasoned about in a **compositional** (algebraic) manner.
- They are a graphical language, which highlights how components in a system **exchange resources**.

String diagrams are a relatively young methodology, which is starting just now to inform the development of programming practices, as well as tools of analysis (especially compilers).

Peirce Vindicated?

Many of Peirce's original insights find a new home in string diagram-based approaches in Theoretical Computer Science.

- **Relations** as unit of reasoning
- **Variable-free** language
- Computation based on **geometric** and **topological** manipulations
- A formal model should **reflect the real-world** phenomena it refers to.
- A formal model should align with how **humans** naturally **think**.



Want to know more?

- An Introduction to String Diagrams for Computer Scientists
arxiv.org/pdf/2305.08768.pdf
- String Diagrams for λ -calculi and Functional Computation
arxiv.org/pdf/2305.18945.pdf
- A Survey on Compositional Signal Flow Theory
<http://www.zanasi.com/fabio/files/paperIFIP21.pdf>
- Blog Tutorial: Graphical Linear Algebra
www.graphicallinearalgebra.net
- String Diagrammatic Approach to Peirce's Calculus of Relations
<https://doi.org/10.1016/j.tcs.2026.116189>